

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and communications technology

Software Design Document

Version 1.3

An Internet Media Store

Subject: ITSS Software Development

Group 6

Vũ Hải Đăng-20225962

Nguyễn Minh Khôi-20226050

Lê Đại Lâm-20225982

Phạm Thành Nam-20225989

Bùi Xuân Sơn-20226065

Hanoi, 06, 2025

<All notations inside the angle bracket are not part of this document, for its purpose is for extra instruction. When using this document, please erase all these notations and/or replace them with corresponding content as instructed>

<This document, written by Prof. NGUYEN Thi Thu Trang, is used as a case study for student with related courses. Any modifications and/or utilization without the consent of the author is strictly forbidden>

Table of Contents

1 Introduction.....	3
1.1 Objective.....	3
1.2 Scope.....	3
1.3 Glossary.....	4
1.4 References.....	6
2 Overall Description.....	7
2.1 General Overview.....	7
2.2 Assumptions/Constraints/Risks.....	18
2.1.1 Assumptions.....	18
2.1.2 Constraints.....	19
2.1.3 Risks.....	21
3. System Architecture and Architecture Design.....	22
3.1 Architectural Patterns.....	22
3.2 Interaction Diagrams.....	23
3.3 Analysis Class Diagrams.....	34
3.4 Unified Analysis Class Diagram.....	42
3.5 Security Software Architecture.....	42
4. Detailed Design.....	44
4.1 User Interface Design.....	44
4.1.1 Screen Configuration Standardization.....	44
4.1.2 Screen Transition Diagrams.....	45
4.1.3 Screen Specifications.....	45
4.2 Data Modeling.....	55
4.2.1 Conceptual Data Modeling.....	55
4.2.2 Database Design.....	55
4.3 Class Design.....	64
4.3.1 General Class Diagram.....	64
4.3.2 Class Diagrams.....	64
4.3.3 Class Design.....	67
5. Design Considerations.....	132
5.1 Goals and Guidelines.....	132
5.1.1 Performance Goals.....	132
5.1.2 Modularity and Maintainability.....	133
5.1.3 User Experience Guidelines.....	133
5.1.4 Development Guidelines.....	134
5.1.5 Integration Guidelines.....	134
5.1.6 Business Logic Guidelines.....	135
5.2 Architectural Strategies.....	136

5.2.1 Layered Architecture Strategy.....	136
5.2.2 Technology Stack Selection.....	136
5.2.3 Module-Based Architecture.....	137
5.2.4 Data Management Strategy.....	137
5.2.5 External Integration Strategy.....	138
5.2.6 Concurrency and Synchronization Strategy.....	138
5.2.7 Error Detection and Recovery Strategy.....	139
5.2.8 Security Architecture Strategy.....	139
5.2.9 User Interface Strategy.....	140
5.2.10 Extensibility Strategy.....	141
5.3 Design and Program Evaluation.....	141
5.3.1 Cohesion & SRP.....	141
5.3.2 Coupling & Other SOLID.....	148
5.4 Design Patterns.....	149

1 Introduction

The following subsections of the Software Design Document (SDD) provide an overview of the architectural and design specifications for AIMS: An Internet Media Store. This document translates the functional requirements defined in the Software Requirements Specification (SRS) into technical design components and implementation guidelines.

1.1 Objective

The objective of this Software Design Document is to provide detailed architectural and design specifications for the AIMS: An Internet Media Store system. This document serves as a technical blueprint that outlines the system's structural organization, component interactions, data models, and implementation strategies.

The intended audience includes:

- **Software Architects:** To validate the overall system architecture and design patterns
- **Software Engineers/Developers:** To implement the system components according to specified designs
- **Database Administrators:** To design and implement the data storage solutions
- **Technical Project Managers:** To oversee technical development progress and resource allocation
- **Quality Assurance Engineers:** To develop test strategies based on the architectural design
- **System Integrators:** To understand component interfaces and integration points
- **Technical Writers:** To create implementation documentation and API specifications

This document bridges the gap between requirements (defined in the SRS) and implementation, ensuring all technical stakeholders have a clear understanding of how the system will be constructed.

1.2 Scope

The AIMS: An Internet Media Store is a desktop-based e-commerce application designed for the management and sale of physical media products. This design document covers the complete technical architecture for:

- **Product Management Subsystem**

- Product catalog management with support for Books, CDs, LP records, and DVDs
- Inventory management with real-time stock tracking
- Price management with business rule enforcement
- Product attribute management based on media type
- **User Management Subsystem**
 - Role-based access control for Administrators, Product Managers, and Customers
 - User authentication using token-based security
 - User account lifecycle management
- **E-commerce Transaction Subsystem**
 - Shopping cart management with session persistence
 - Order processing workflow including standard and rush delivery options
 - Payment integration with VNPay payment gateway
 - Order status tracking and management
- **Integration Subsystem**
 - VNPay API integration for payment processing
 - Email notification system for order confirmations and user management
 - External shipping calculation services
- **Technical Scope:**
 - Multi-user desktop application supporting up to 1,000 concurrent users
 - Response time requirements: ≤ 2 seconds normal operation, ≤ 5 seconds peak hours
 - Uptime requirement: 300 hours continuous operation
 - Recovery time: ≤ 1 hour maximum downtime after failure
- **Out of Scope:**
 - Digital media product sales
 - Alternative payment methods beyond credit cards
 - Mobile application versions
 - Real-time chat support
 - Customer account creation (customers can purchase without accounts)

The system architecture will be designed for scalability, maintainability, and performance, with clear separation of concerns between business logic, data access, and presentation layers.

1.3 Glossary

No	Term	Explanation	Example	Note
1	MVC (Model-View-Controller)	Architectural pattern separating application logic into three interconnected components	User interface (View), Business logic (Controller), Data handling (Model)	Used for organizing code structure
2	API (Application Programming Interface)	Set of protocols and tools for building software applications	VNPay Payment API	Enables system integration
3	Session Management	Process of maintaining user state and cart data during software interaction	Shopping cart persistence during user session	Critical for e-commerce functionality
4	Database Transaction	Sequence of database operations performed as a single logical unit	Order placement with inventory update	Ensures data consistency
5	Repository Pattern	Design pattern that separates data access logic from business logic	ProductRepository, OrderRepository	Abstracts database operations
6	Factory Pattern	Design pattern for creating objects without specifying the exact class	MediaProductFactory for Books, CDs, DVDs	Creates objects based on media type
7	Observer Pattern	Design pattern where objects notify other objects about state changes	Inventory update notifications	Used for real-time updates
8	DTO (Data Transfer Object)	Object that carries data between processes	PaymentDTO, OrderDTO	Reduces method calls
9	Singleton Pattern	Design pattern ensuring a class has only one instance	DatabaseConnection, EmailService	Manages shared resources
10	Service Layer	Architectural layer containing business logic	OrderService, PaymentService	Separates business

				logic from data access
11	ORM (Object-Relational Mapping)	Technique for converting incompatible type systems	Entity classes mapped to database tables	Simplifies database operations
12	Dependency Injection	Design pattern for achieving Inversion of Control	Injecting PaymentService into OrderController	Improves testability and flexibility
13	RESTful Architecture	Architectural style for designing networked applications	HTTP methods for CRUD operations	Used for external API integration
14	Load Balancer	System that distributes network traffic across multiple servers	Round-robin request distribution	Supports 1,000 concurrent users
15	Cache	Temporary storage for frequently accessed data	Product catalog cache	Improves response time

1.4 References

Centers for Medicare & Medicaid Services. (n.d.). *System Design Document Template*. Retrieved from Centers for Medicare & Medicaid Services: <https://www.cms.gov/Research-Statistics-Data-and-Systems/CMS-Information-Technology/XLC/Downloads/SystemDesignDocument.docx>

2 Overall Description

2.1 General Overview

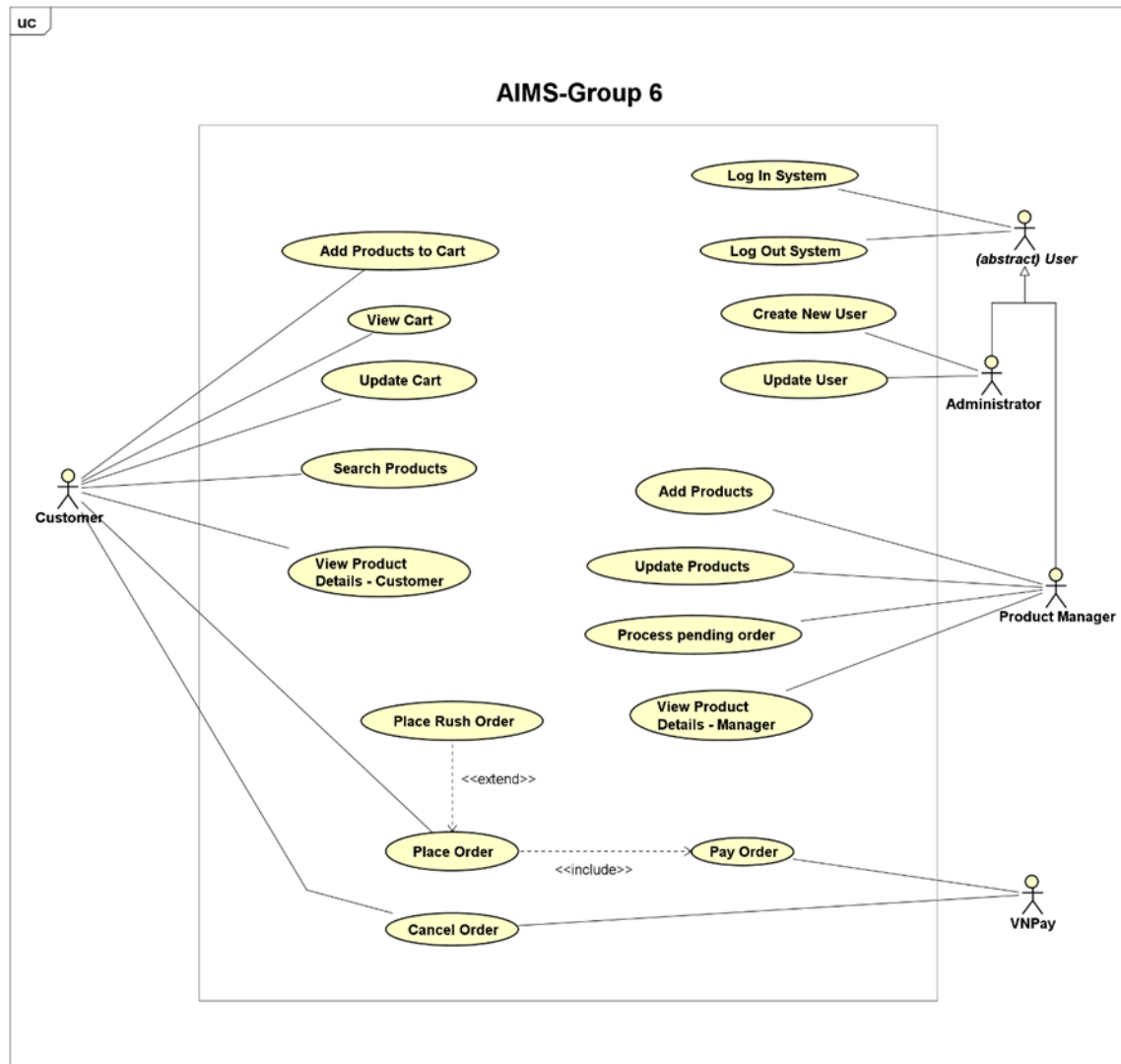
The system in question is an online media store designed to enable customers to purchase digital media products conveniently. This software functions as a comprehensive platform, catering not only to customers but also to product managers.

The system consists of three primary actors:

- **Customer:** Customers can look at, search for, and sort products in the store. To buy something, they must add or update items in their cart and provide delivery details. Once all information is correctly entered, they can proceed with payment via VNPay. Upon a successful transaction, AIMS will send an email confirmation along with the invoice. In certain cases, customers also have the option to place rush orders..
- **Product Manager:** Using the management interface provided by AIMS, product managers can efficiently oversee their store's inventory. They have full control over adding, removing, and updating product details. Additionally, they can manage pending orders by either approving or rejecting them.
- **Administrator:** Administrators hold the authority to manage users within the system. They can create new user accounts, update user information, and block or unblock users as needed.

A Use Case diagram shows how actors interact with different use cases in the system. It represents the system's functional requirements by illustrating the interactions between external and internal agents.

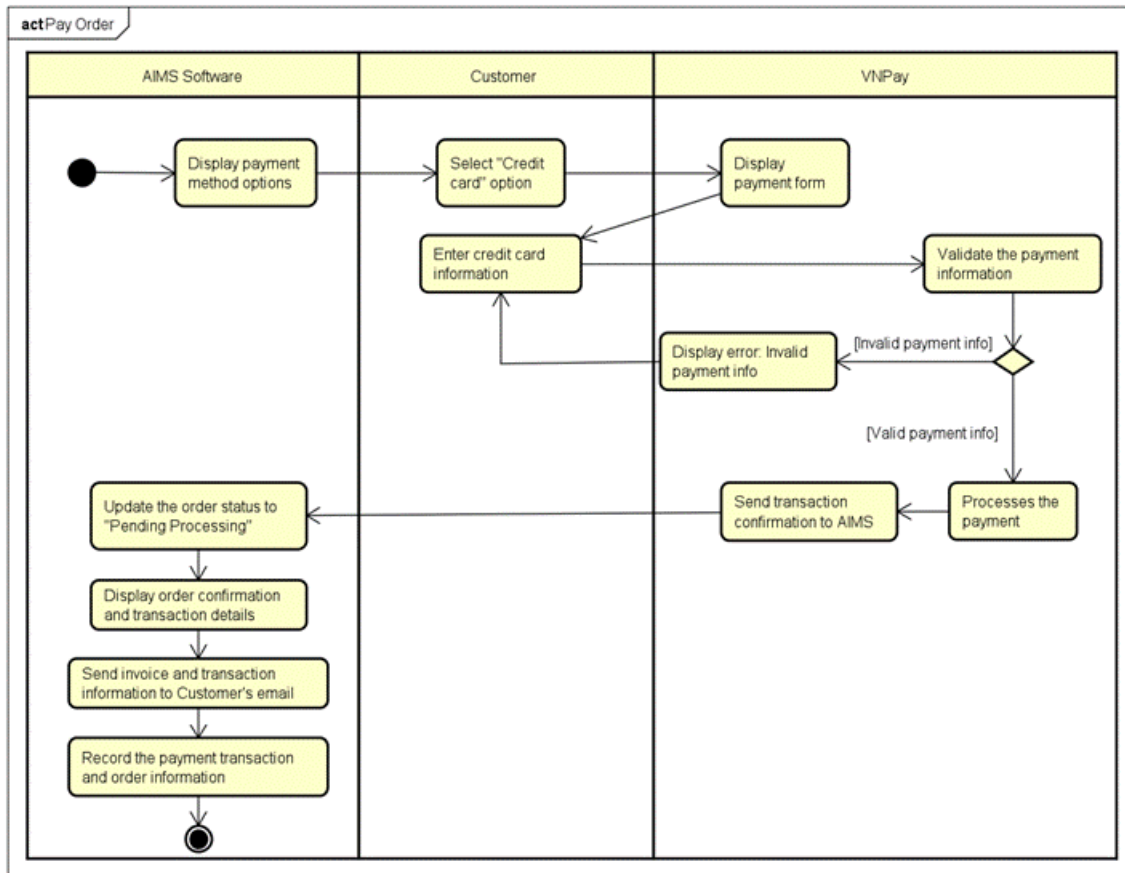
The diagram below presents the general use case diagram of AIMS software, highlighting the actors and their associated use cases.



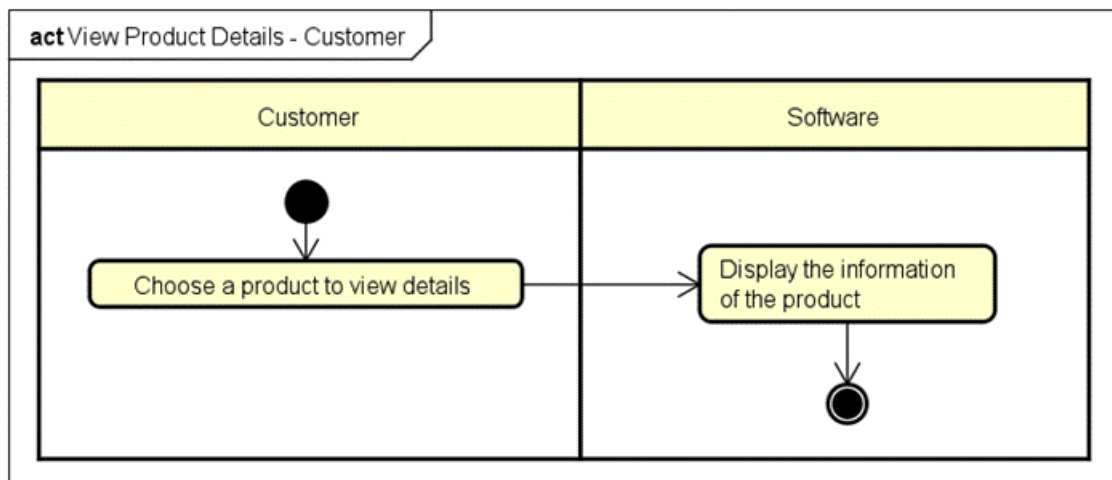
In the AIMS software, five core business operations are defined: "Place Order," "Place Rush Order," "Pay Order," "Cancel Order," and "Process Pending Order." Each of these operations is thoroughly illustrated using an activity diagram in its respective section. Below, we present the activity diagrams for all use cases in the AIMS software.

Activity diagrams are utilized to model business processes as they provide a structured visual representation of workflows, decision points, and resource allocation. These diagrams enable stakeholders to understand, analyze, and refine complex processes by depicting the sequence of activities, concurrency, error handling, and integration with other models. Additionally, they function as effective documentation tools, enhancing communication and collaboration among stakeholders.

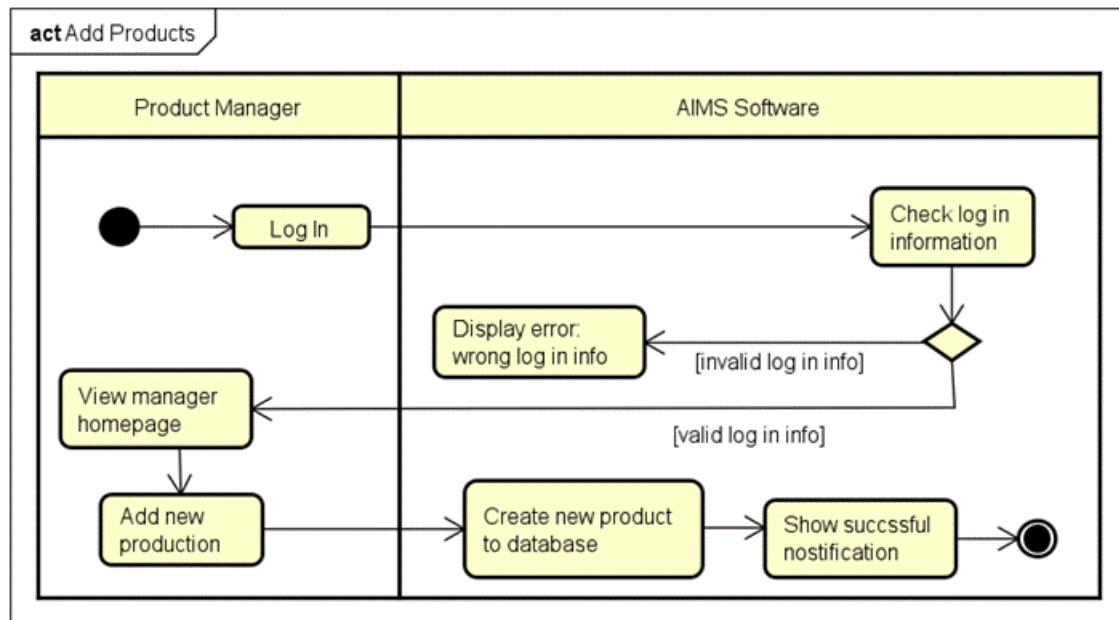
2.1.1 Business operation- Pay Order



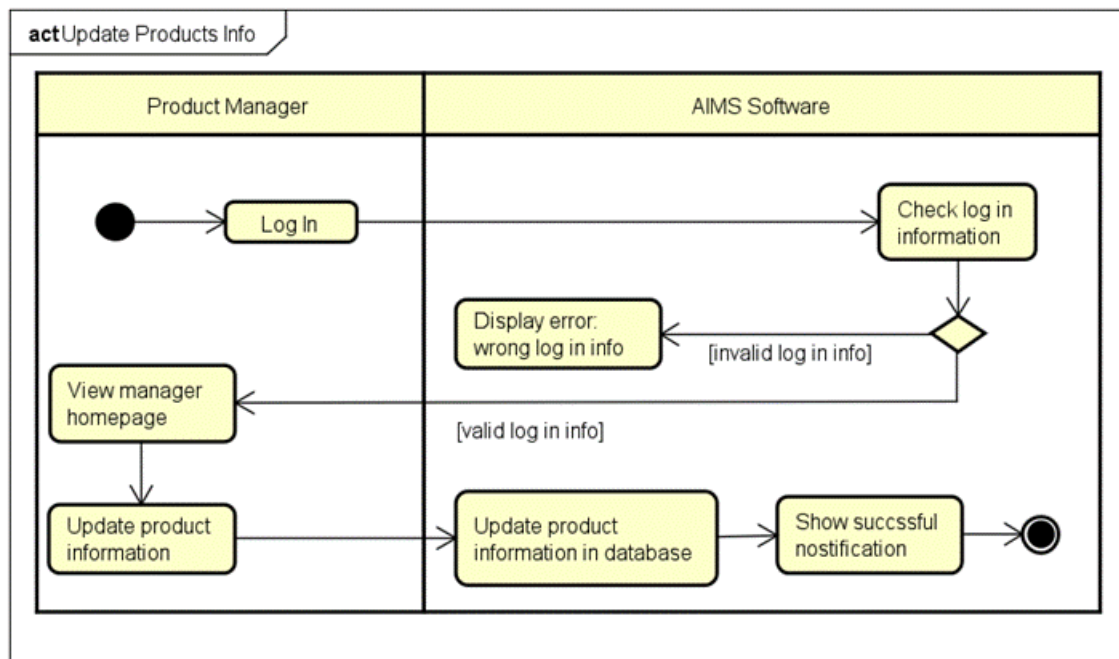
2.1.2 Business operation- View Product Detail



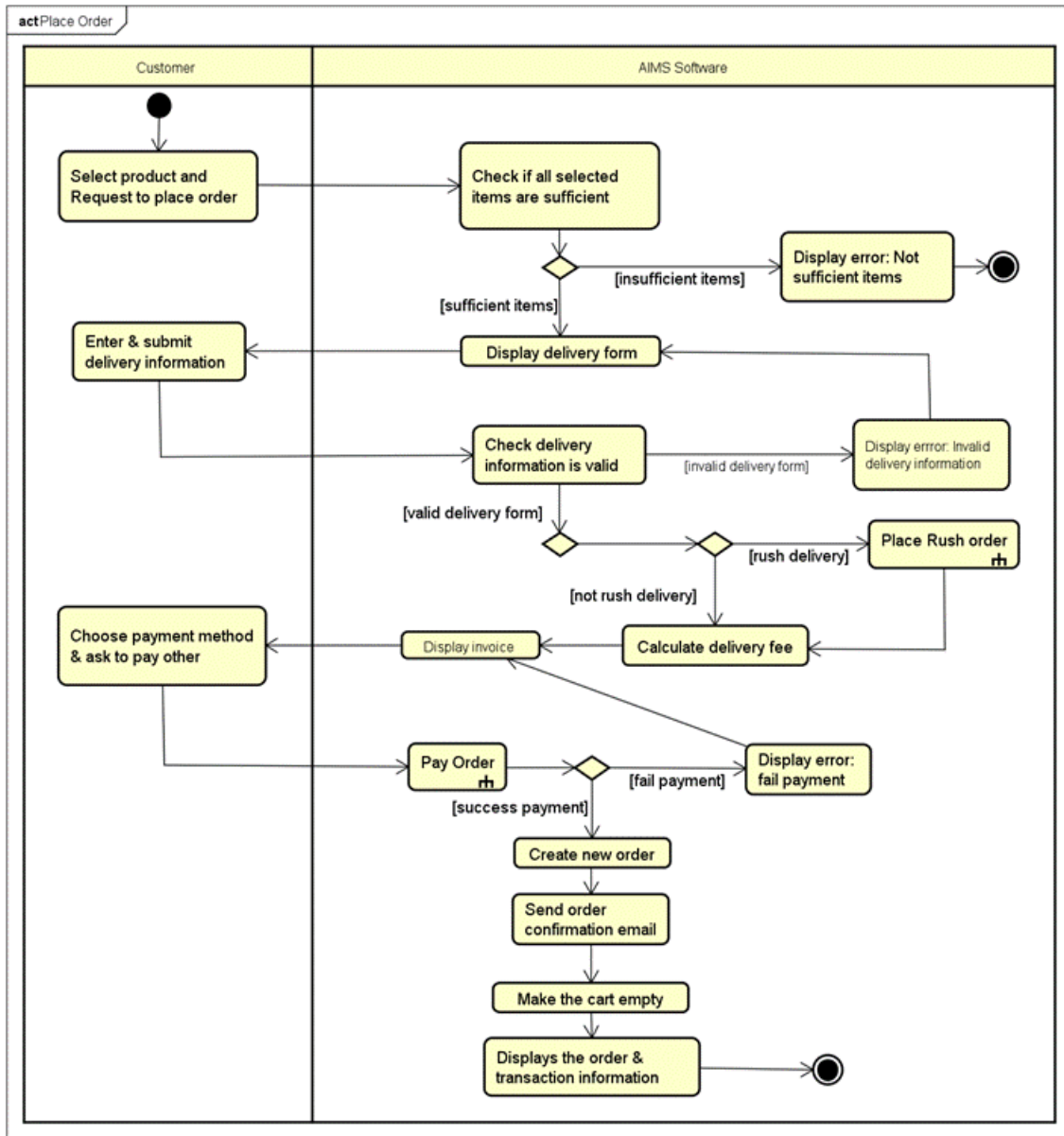
2.1.3 Business operation- Add Product



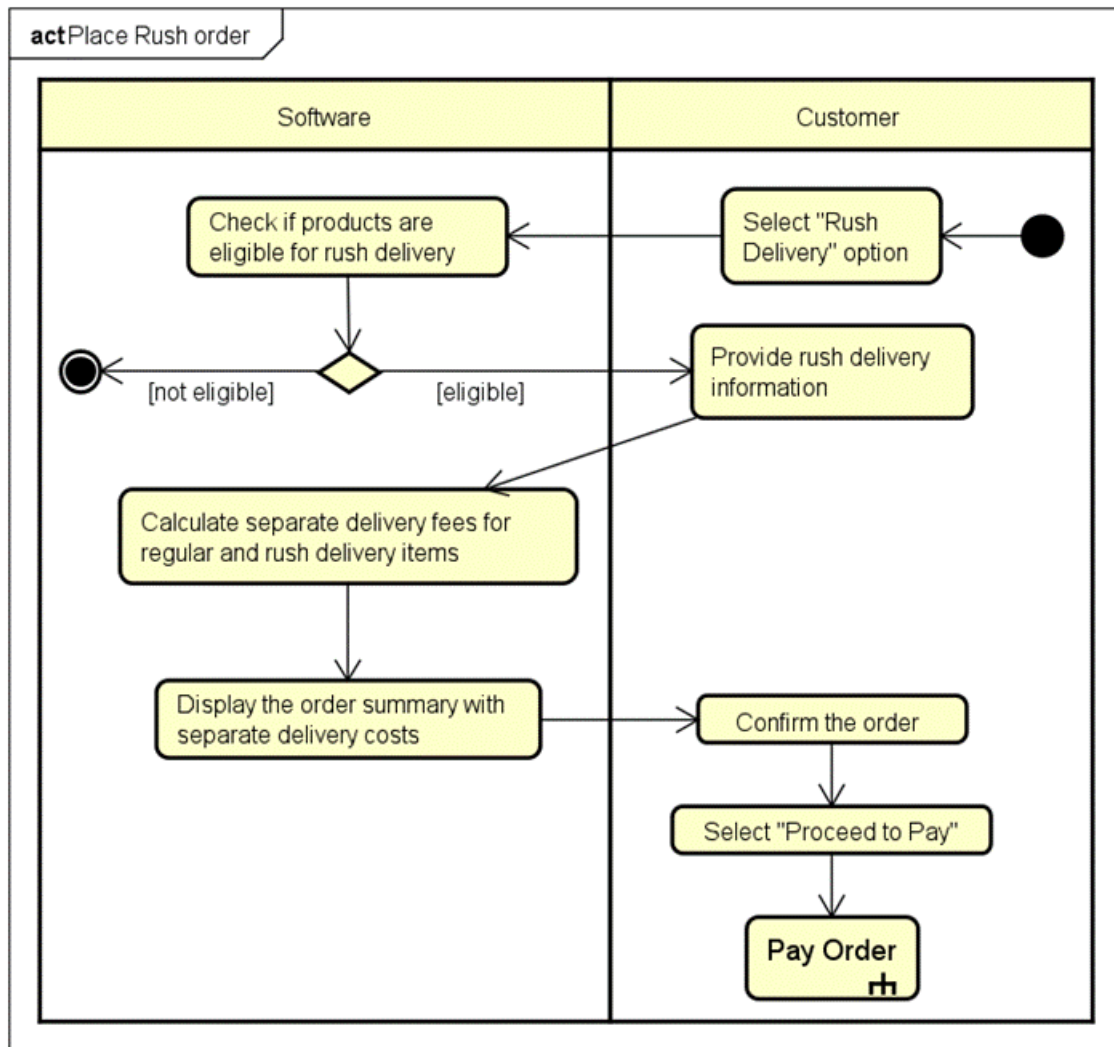
2.1.4 Business operation- Update Product



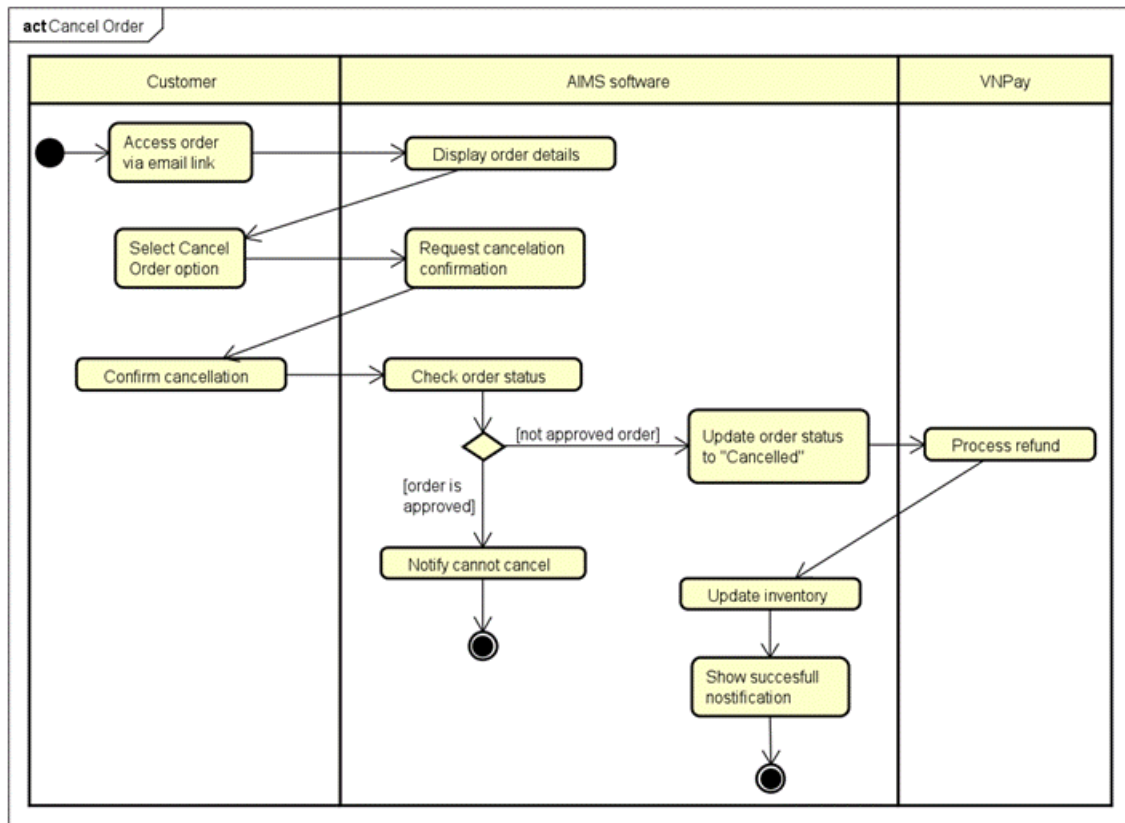
2.1.5 Business operation- Place Order



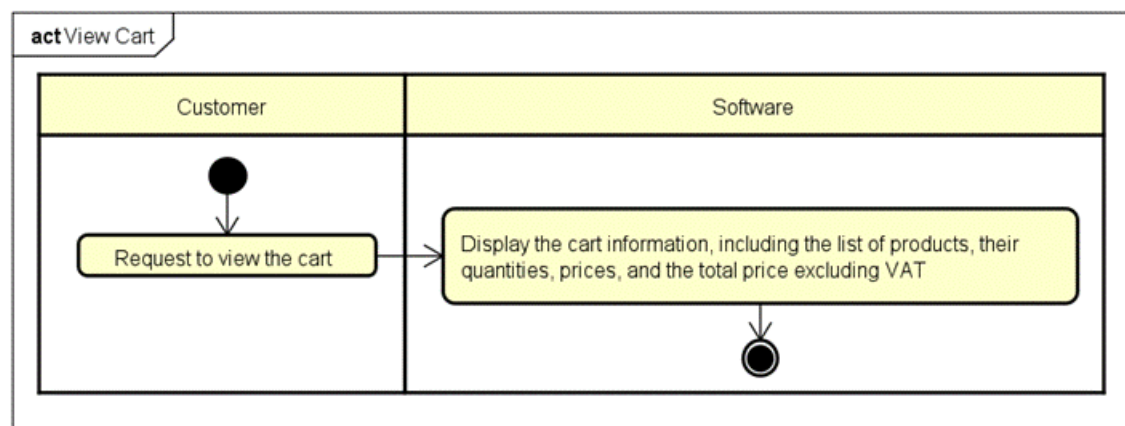
2.1.6 Business operation- Place Rush Order



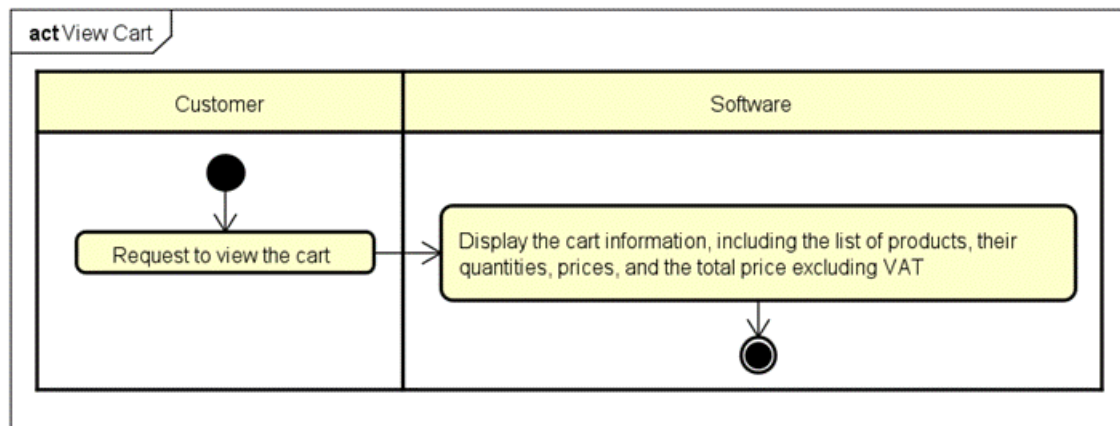
2.1.7 Business operation- Cancel Order



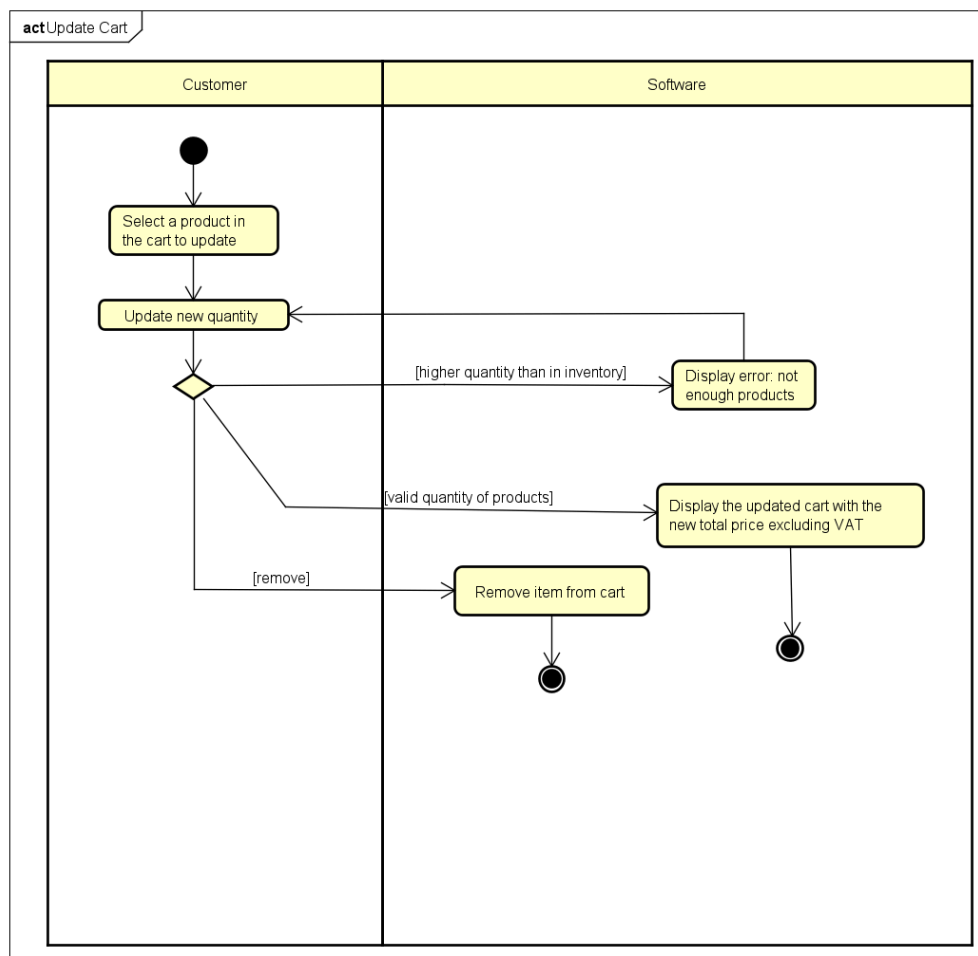
2.1.8 Business operation- Add product to Cart



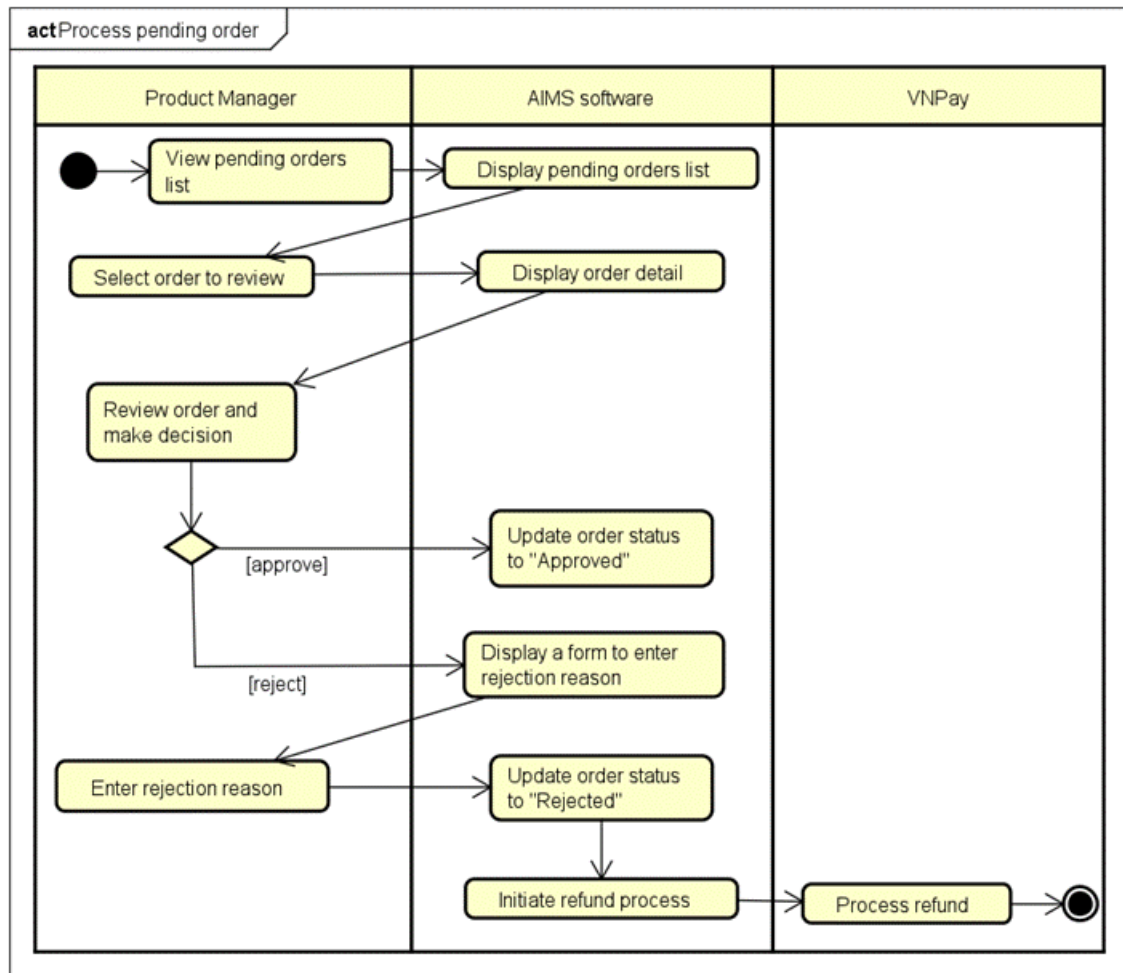
2.1.9 Business operation- View Cart



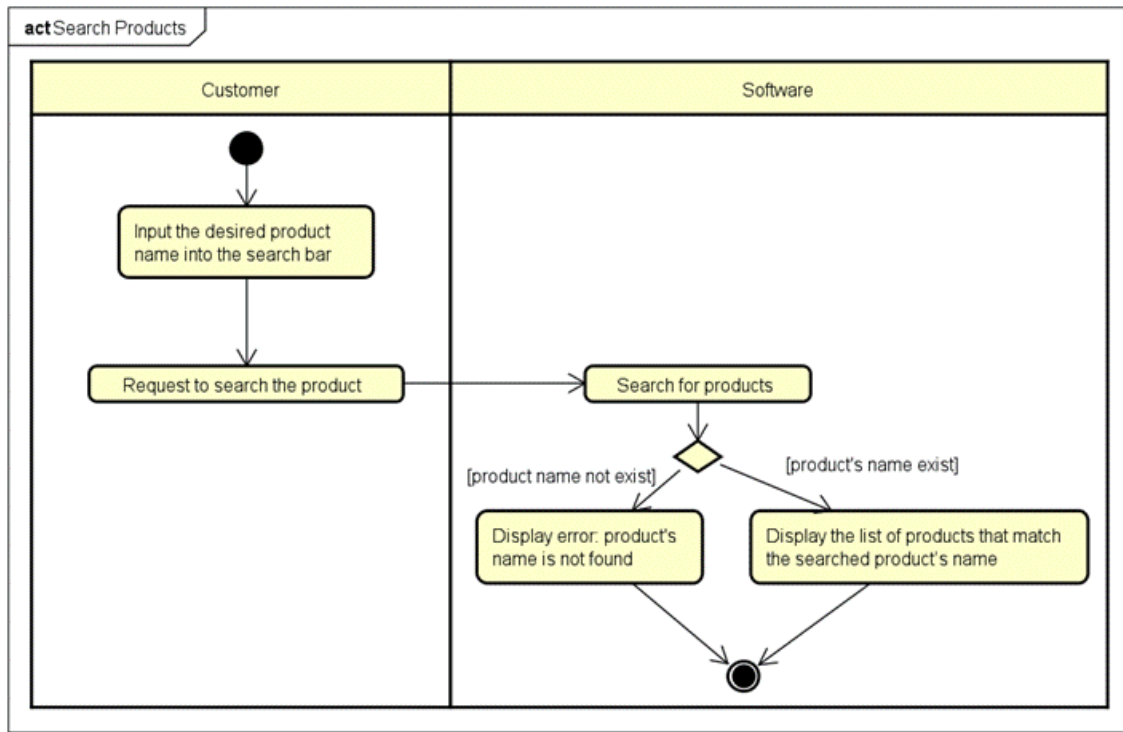
2.1.10 Business operation- Update Cart



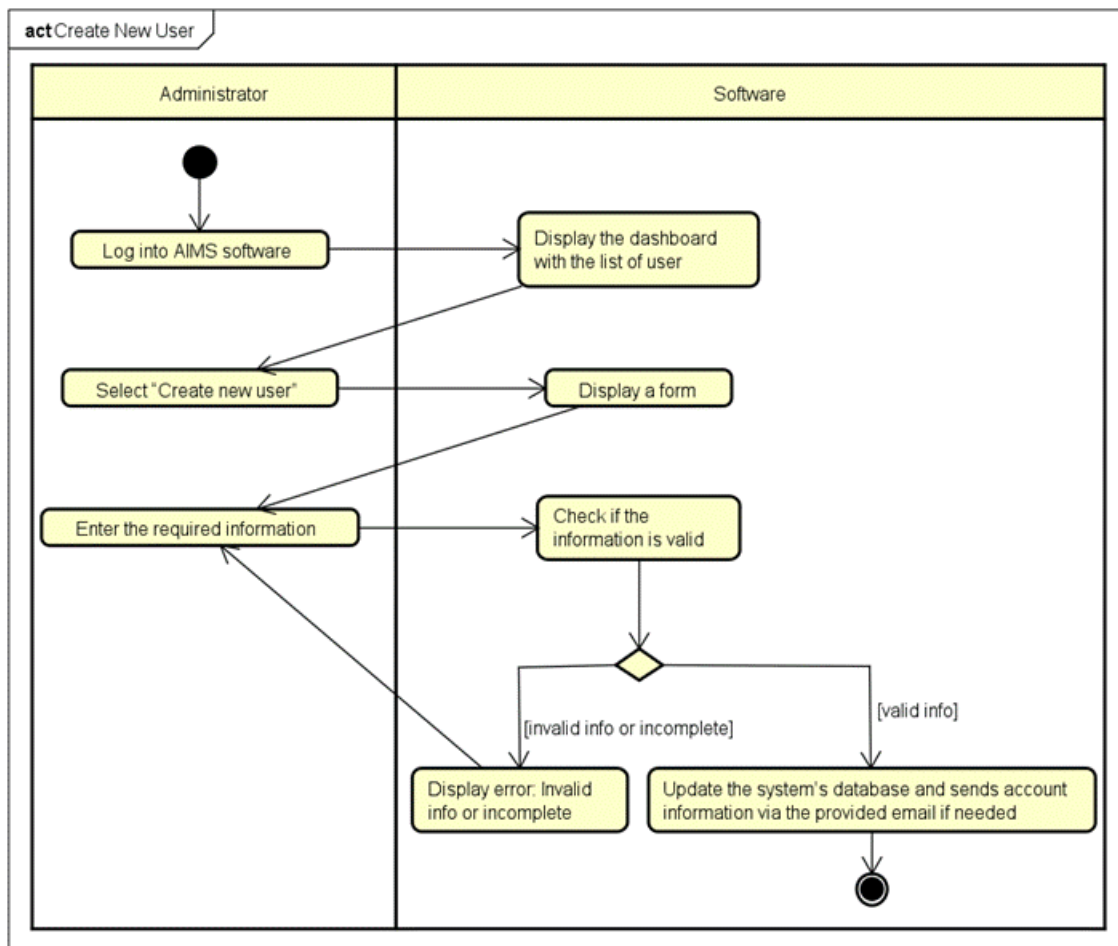
2.1.11 Business operation- Process Pending Order



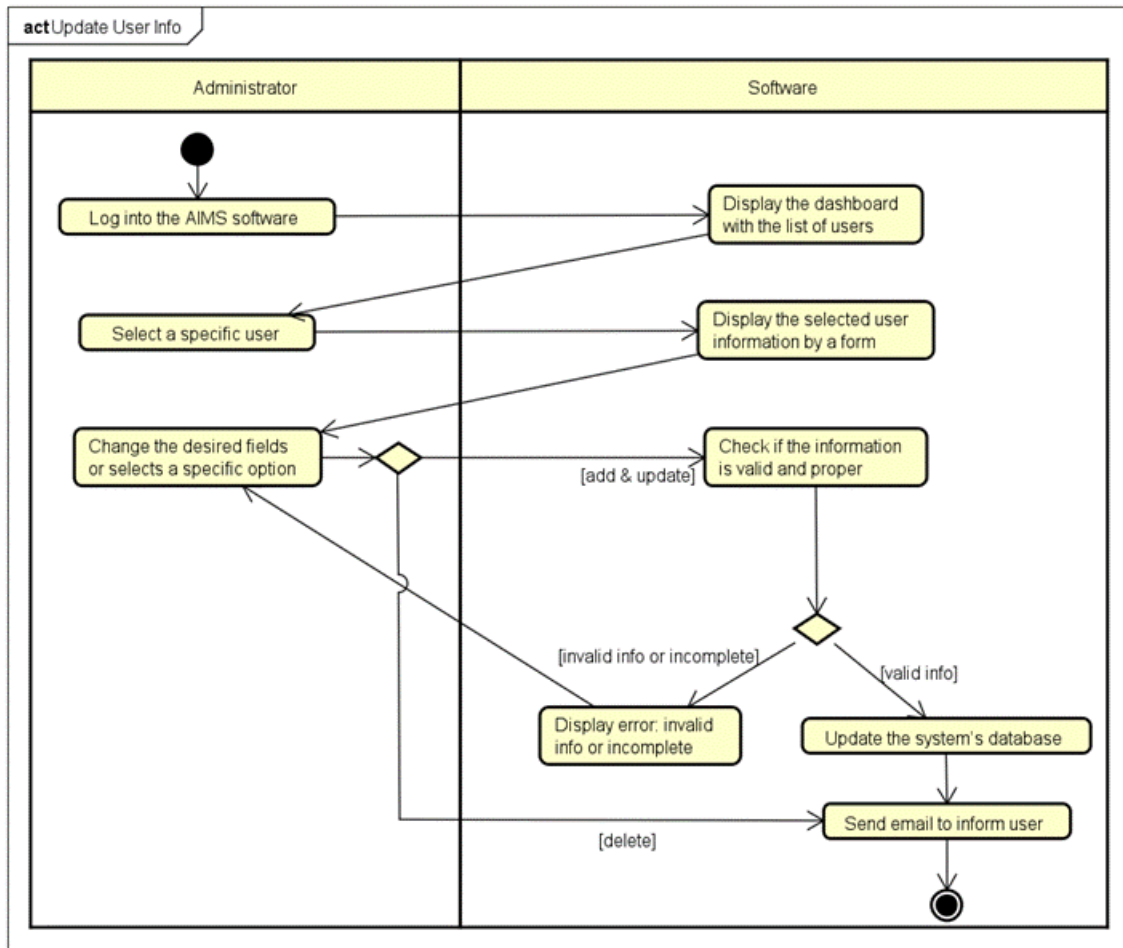
2.1.12 Business operation- Search Product



2.1.13 Business operation- Create User



2.1.14 Business operation- Update User



2.2 Assumptions/Constraints/Risks

2.1.1 Assumptions

- **Software and Hardware Dependencies:** The AIMS platform is intended to function efficiently within a standard Java application environment. A conventional relational database management system (such as SQLite) will be employed for data storage and management.
- **Operating System Compatibility:** The system will be primarily developed and deployed in a Windows environment; however, the final application must ensure cross-platform compatibility, particularly with both Windows and macOS operating systems.
- **User Profile Assumptions:** It is assumed that target users possess a basic level of digital literacy, including familiarity with navigating online marketplaces and

interacting with digital media platforms. Users are expected to have reliable internet connectivity and access to devices capable of streaming or downloading media content.

- **Functional Evolution** :Anticipated future enhancements may involve the addition of new media formats or the integration of alternative payment systems. Accordingly, the system must be designed with a modular and adaptable architecture. The platform architecture should facilitate scalability, allowing for the integration of new features with minimal structural modifications.

2.1.2 Constraints

The design and implementation of the system are subject to a range of global constraints that significantly influence decisions regarding hardware architecture, software development, communication protocols, and overall system functionality. These constraints, derived from technical, operational, legal, and environmental factors, are outlined below along with their associated impacts:

- **Hardware and Software Environment Constraints:** The system must be compatible with standard Java-based applications and operate efficiently within commonly used desktop environments (e.g., Windows and macOS). This constraint necessitates cross-platform compatibility in both the user interface and backend components, thereby influencing the choice of development frameworks and runtime environments.
- **End-User Environment Constraints:** The target user base is expected to possess only a basic level of digital proficiency and operate within diverse computing environments. This necessitates a user-friendly interface and robust error handling to accommodate varying levels of technical expertise and device capabilities.
- **Availability and Volatility of Resources:** Fluctuations in human resources and funding availability may affect development timelines and operational continuity. Moreover, reliance on third-party APIs for critical functions such as payment processing and media delivery requires the system to incorporate fallback mechanisms and modular integration to ensure resilience.
- **Standards Compliance and Licensing:** The system must adhere to relevant open-source software licenses and digital media distribution agreements. These legal constraints influence the selection of third-party libraries, content handling procedures, and deployment strategies to avoid intellectual property violations.
- **Interoperability and Interface Requirements:** Integration with third-party services, such as payment gateways (e.g., VNPay) and external media content providers, demands conformance to specified communication protocols and data

formats. This imposes architectural constraints on API design, error management, and data exchange mechanisms.

- **Data Repository and Distribution Requirements:** A centralized SQLite database is employed to manage user accounts, transaction histories, and media inventory. Given the localized nature of SQLite, the system must implement optimized query execution, efficient indexing, and data access patterns to ensure performance, particularly during concurrent operations. While content distribution is not cloud-based, media must be managed efficiently for local playback or controlled download.
- **Security and Regulatory Compliance:** The system must safeguard user data and financial transactions in compliance with standard security protocols (e.g., HTTPS, encryption, secure authentication). This impacts system architecture by requiring the integration of secure communication channels and robust access control mechanisms.
- **Capacity Limitations:** As a desktop-based solution, the application must be optimized for limited system memory and processing power. Efficient memory management, resource allocation, and JavaFX performance tuning are essential to support concurrent user actions, background processes, and media operations without degrading user experience.
- **Performance Requirements:** Users expect quick application startup, smooth navigation, and uninterrupted media playback. To meet these expectations, the system must employ caching strategies, preloading techniques, and efficient media handling logic to minimize latency and ensure responsiveness under varying system loads.
- **Network Communication Constraints:** The application depends on a stable internet connection for payment processing and media-related operations. However, it must also tolerate network variability through retry logic, connection timeout handling, and asynchronous processing to ensure graceful degradation and uninterrupted user interaction.
- **Verification and Validation Requirements:** The system must undergo rigorous validation through automated unit tests, integration tests, and user acceptance testing. This is critical to ensure that all components—from the JavaFX front end to the SQLite database and VNPay API integration—function correctly and securely under expected operational conditions.
- **Quality Assurance Measures:** To ensure long-term reliability and maintainability, the system should adopt continuous integration and delivery

(CI/CD) practices, regular code reviews, and scheduled maintenance cycles. Performance monitoring and user feedback mechanisms will also be incorporated to guide iterative improvements and adaptation over time.

2.1.3 Risks

- **Third-Party Dependency Risks:**
 - **Identified Risk:** The system relies on third-party services for critical functions such as payment processing and media content delivery, which introduces potential points of failure or service disruption.
 - **Mitigation Strategy:** To reduce dependency-related vulnerabilities, the system architecture will support integration with multiple service providers. In addition, contingency protocols will be developed to ensure business continuity in the event of third-party service failures.
- **Human Resource Risks:**
 - **Identified Risk:** A shortage of qualified developers or the loss of key technical personnel may adversely affect project timelines and product quality.
 - **Mitigation Strategy:** This risk will be addressed through competitive recruitment processes, investment in ongoing staff development, and initiatives focused on employee retention and knowledge transfer.
- **Technical and Scalability Risks:**
 - **Identified Risk:** As the user base expands, the system may encounter performance degradation or infrastructure limitations.
 - **Mitigation Strategy:** The platform will be designed using scalable cloud-based infrastructure, incorporating techniques such as horizontal scaling and load balancing to accommodate increased demand while maintaining performance standards.

3. System Architecture and Architecture Design

In this section, we propose the general overview of System Architecture Design. This starts from defining the Architectural Patterns, then we design the interaction diagram using sequence diagrams for the proposed architectural patterns. From the interaction diagrams, we can have the analysis class diagrams, which are the fundamental steps for the detailed-level class diagrams for implementation.

3.1 Architectural Patterns

The Entity-Control-Boundary (ECB) architecture design pattern is a software design approach that helps in organizing and structuring code to enhance maintainability, scalability, and clarity. This pattern divides the system into three primary types of components: entities, controls, and boundaries.

Entities represent the core business logic and data of the application. These components encapsulate the main data structures and the operations directly related to the core functionality of the system. Entities are typically designed to be reusable and independent of the specific use cases. They model real-world objects and are responsible for managing and maintaining their state and behavior. For example, in a banking application, entities might include classes such as *Account*, *Customer*, and *Transaction*.

Controls are responsible for coordinating the flow of information between entities and boundaries. They manage the application's use cases, orchestrating the interactions between entities and ensuring the correct sequence of operations. Controls contain the logic that drives the application's processes, ensuring that user inputs are properly processed and that the necessary business rules are applied. For instance, a *PlaceOrder* control might handle the logic for executing a financial transaction, validating inputs, and updating account balances accordingly.

Boundaries define the interfaces through which the system interacts with external actors, such as users, other systems, or external services. They serve as the point of communication between the application and its environment, encapsulating the specifics of input and output mechanisms. Boundaries translate user actions into requests that the control components can handle and present the results back to the user. Examples of boundaries include user interface components like forms and buttons, web service interfaces, or API endpoints, which serve as subsystems in interface design.

We select this architecture pattern because many advantages it delivers:

- **Separation of Concerns:** By clearly dividing the system into entities, controls, and boundaries, the ECB pattern ensures a clean separation of concerns, making the codebase more modular and easier to understand.

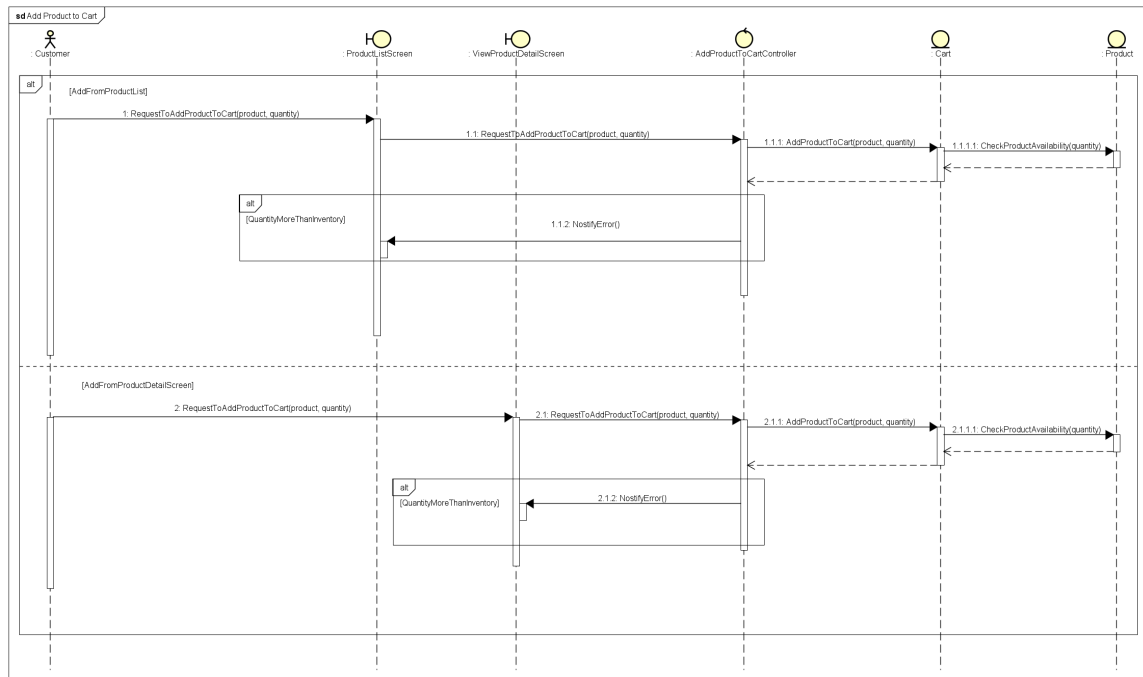
- **Maintainability:** Changes in one part of the system (e.g., user interface changes) do not directly impact the core business logic encapsulated in entities, thereby reducing the risk of introducing bugs when making modifications.

- **Scalability:** The modular nature of the ECB pattern facilitates scalability. New features can be added by creating new controls and boundaries without altering existing entities.

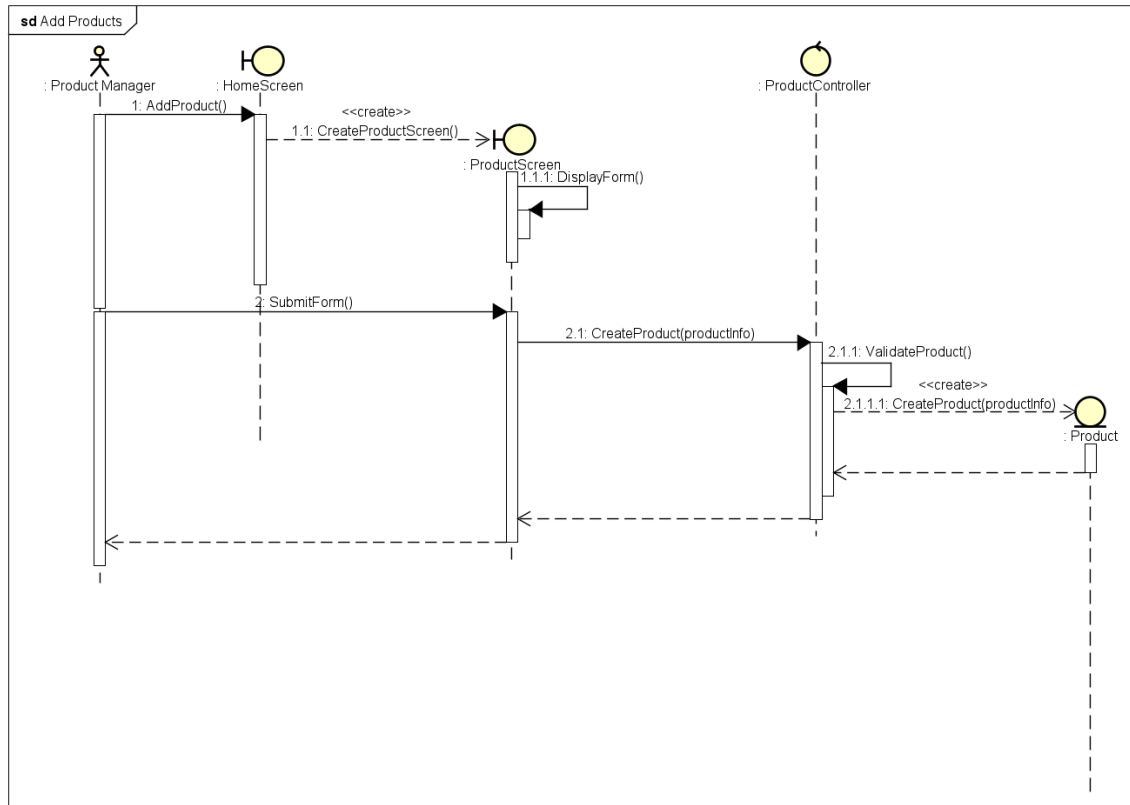
- **Testability:** The clear separation between entities, controls, and boundaries enhances testability. Business logic can be tested independently of the user interface, and mock boundaries can be used to simulate interactions during testing.

3.2 Interaction Diagrams

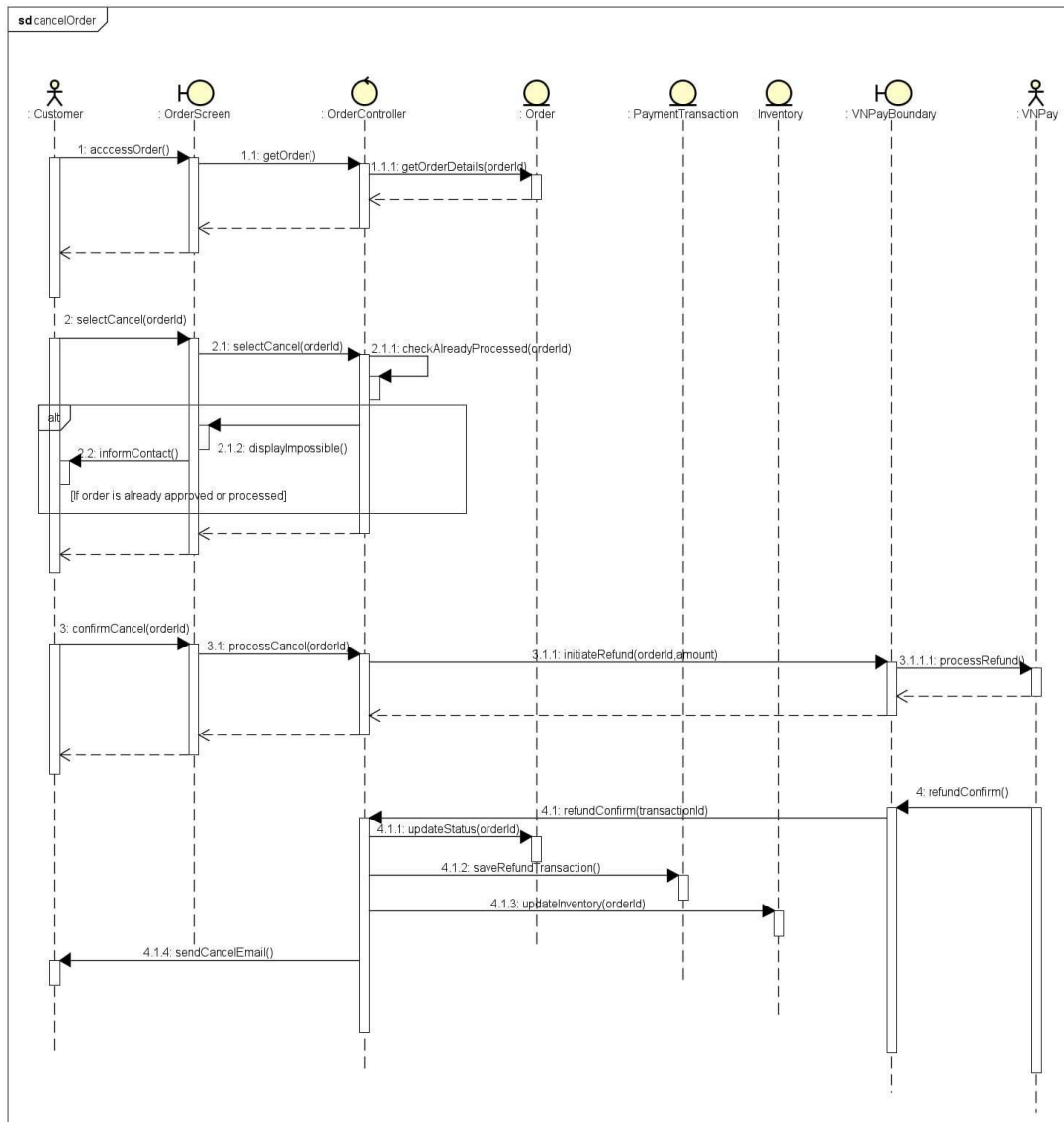
1. Add Product to Cart



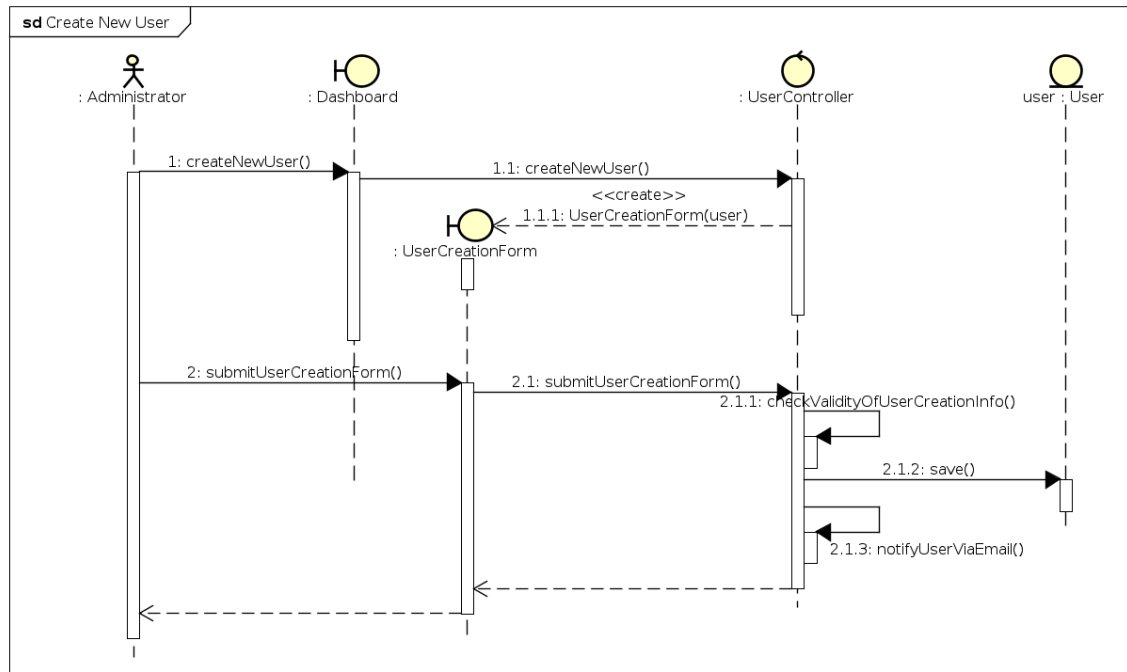
2. Add Products



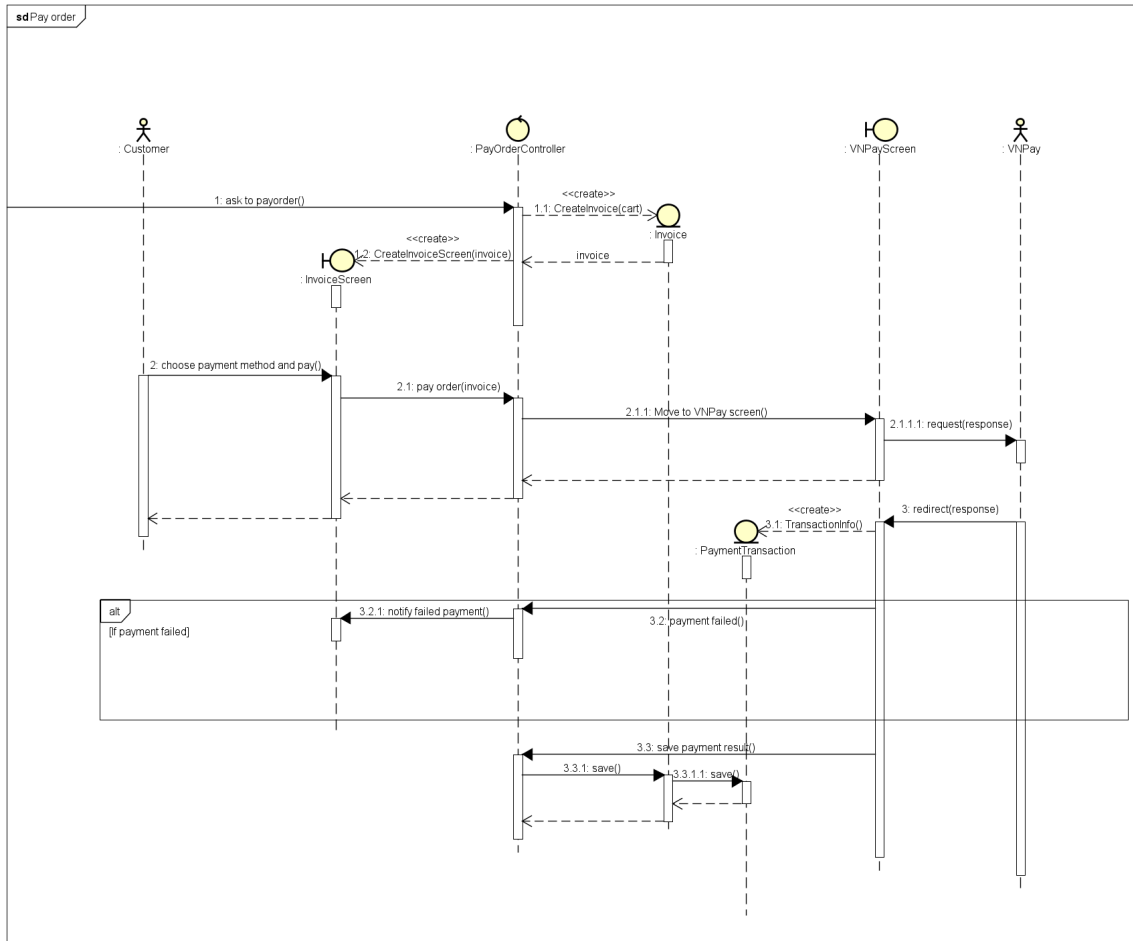
3. Cancel Order



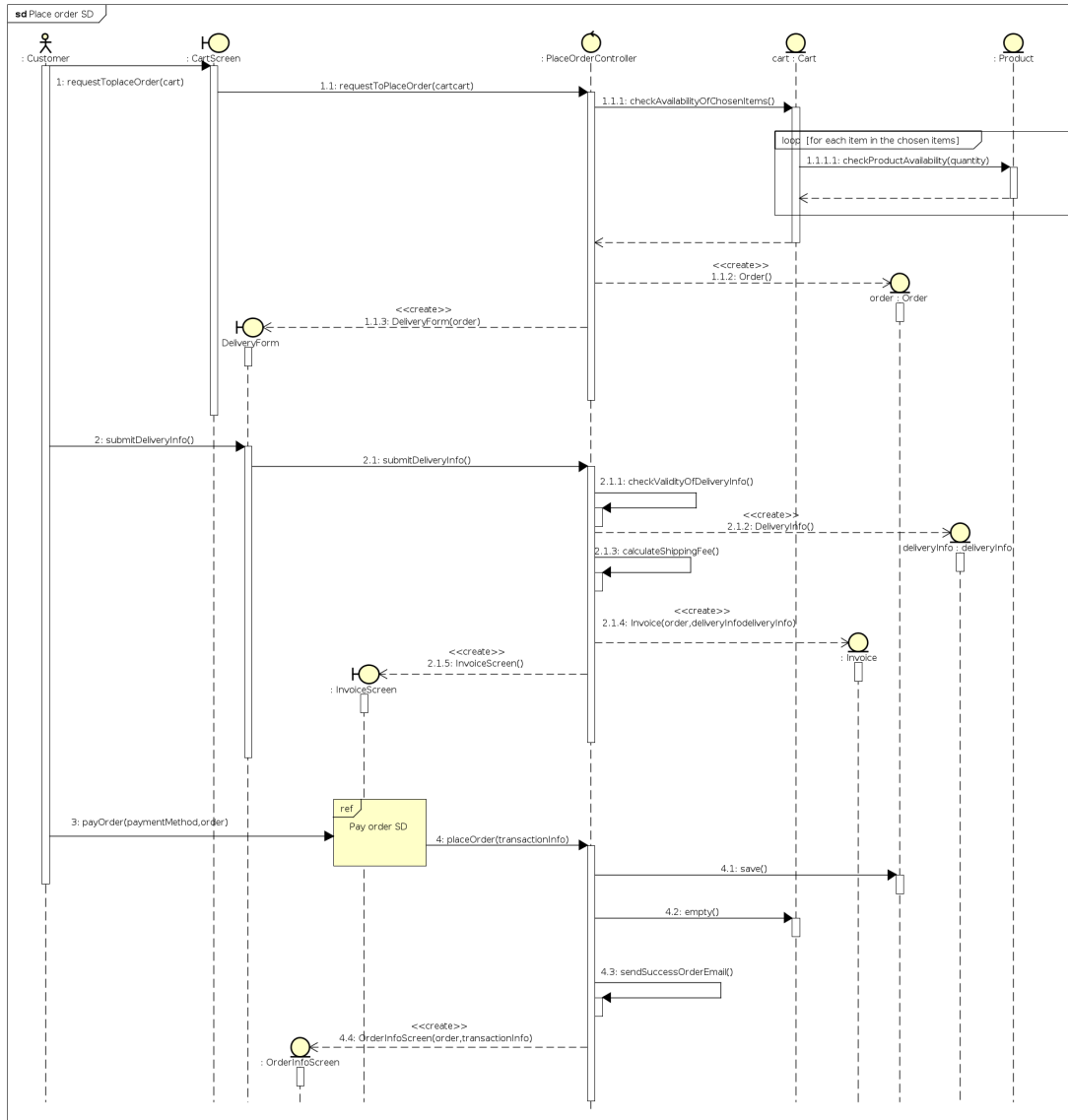
4. Create New User



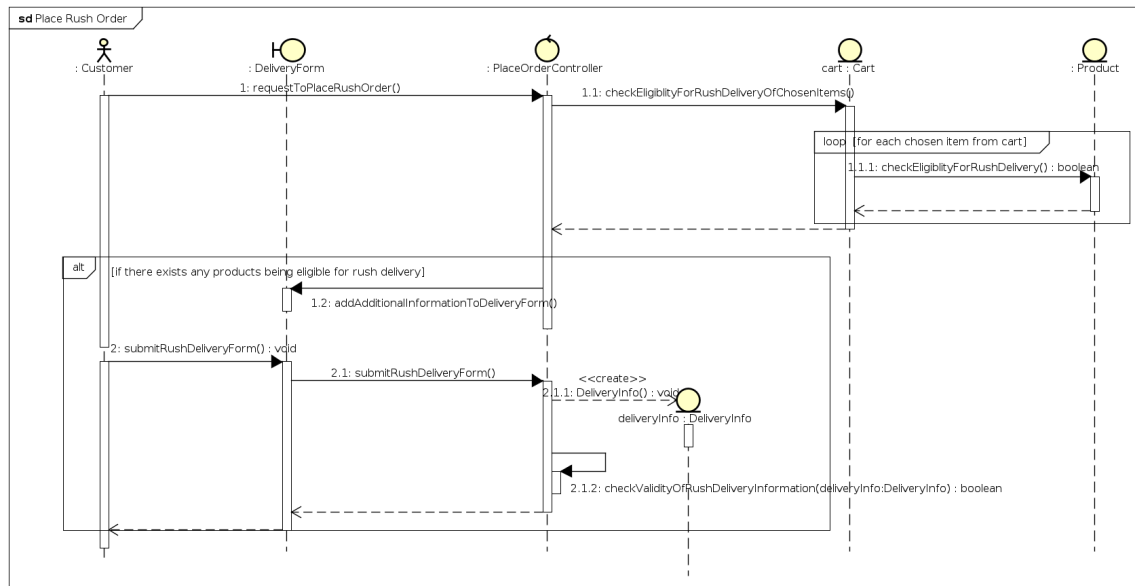
5. Pay Order



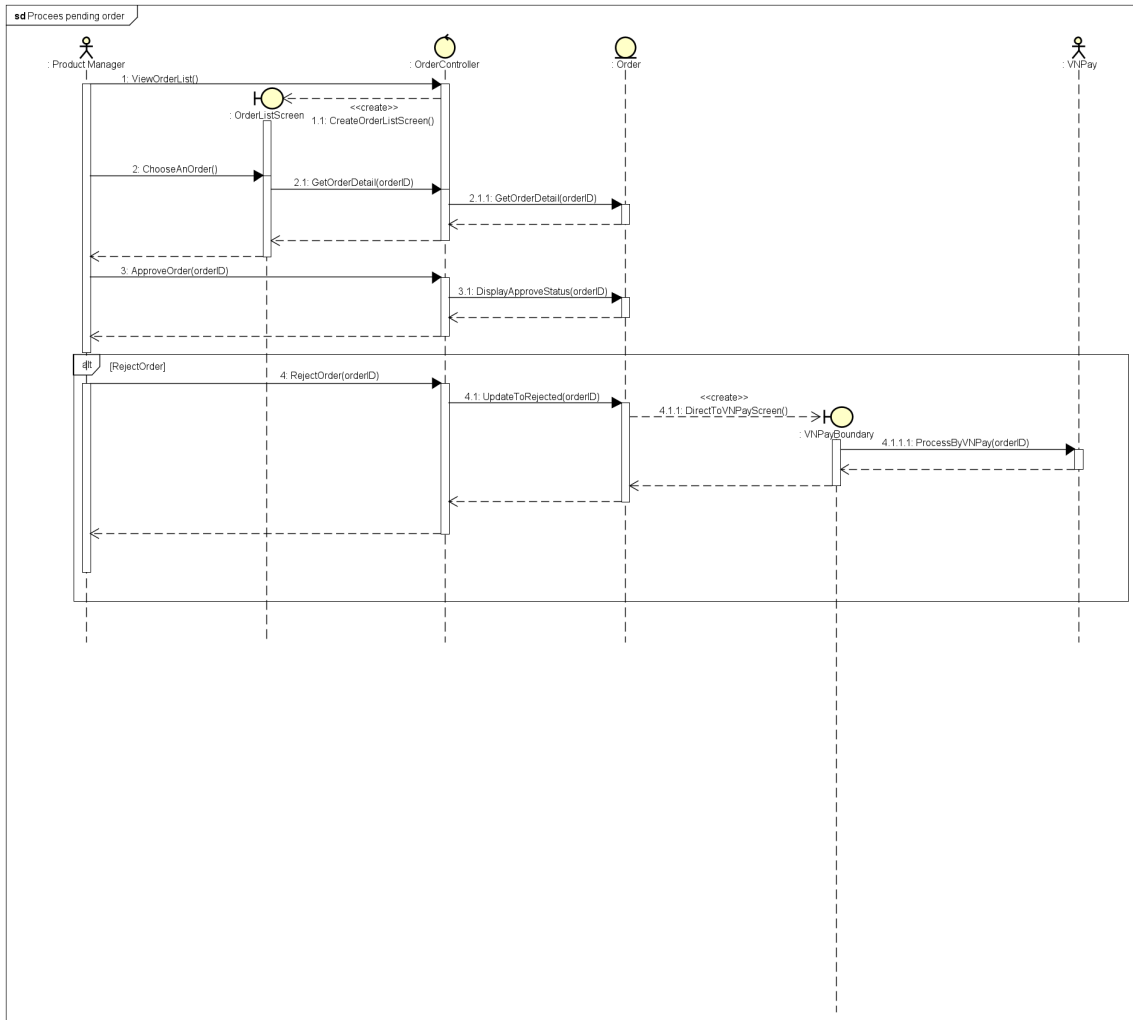
6. Place Order



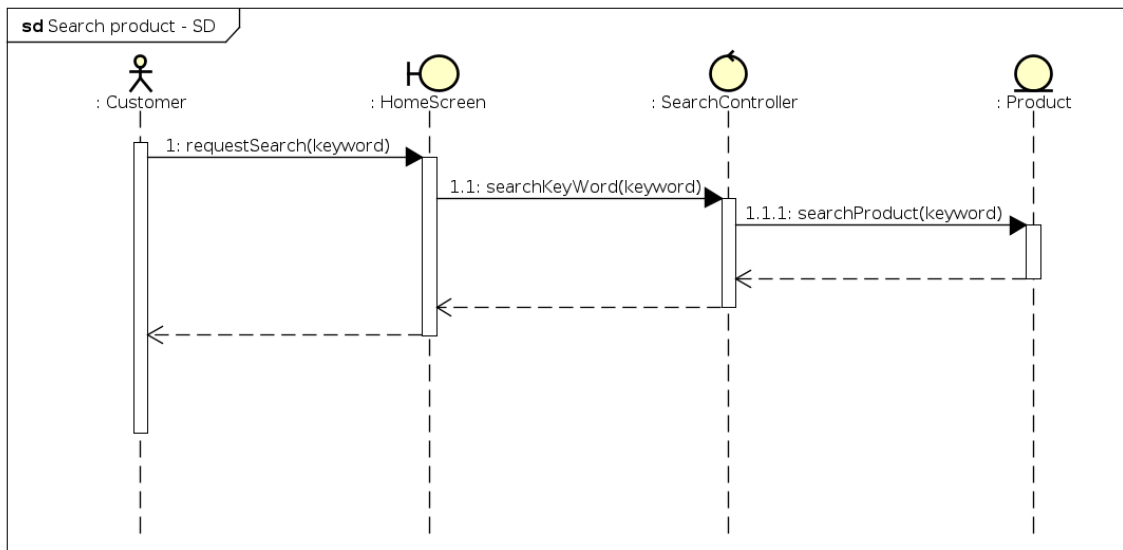
7. Place Rush Order



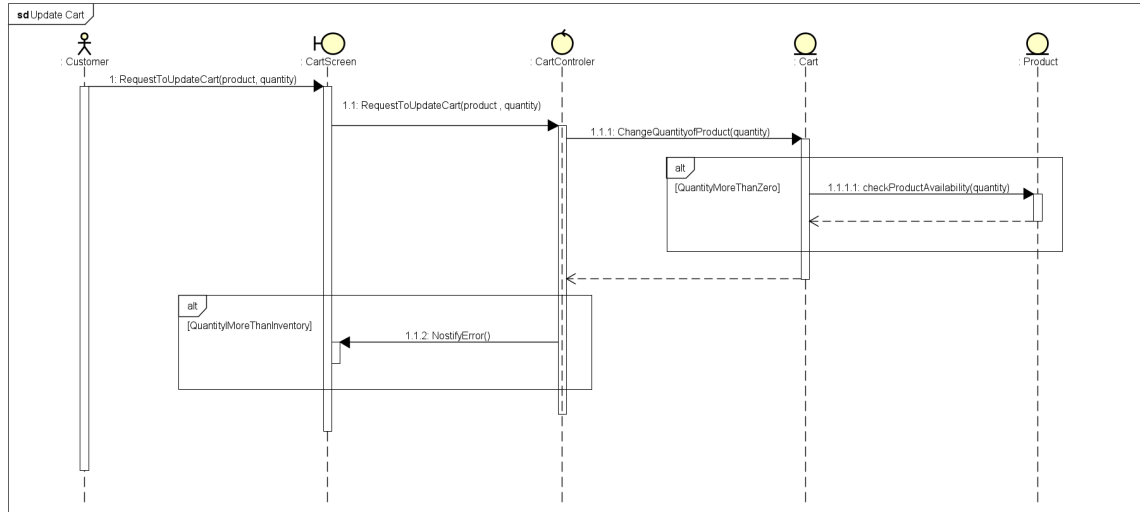
8. Process pending order



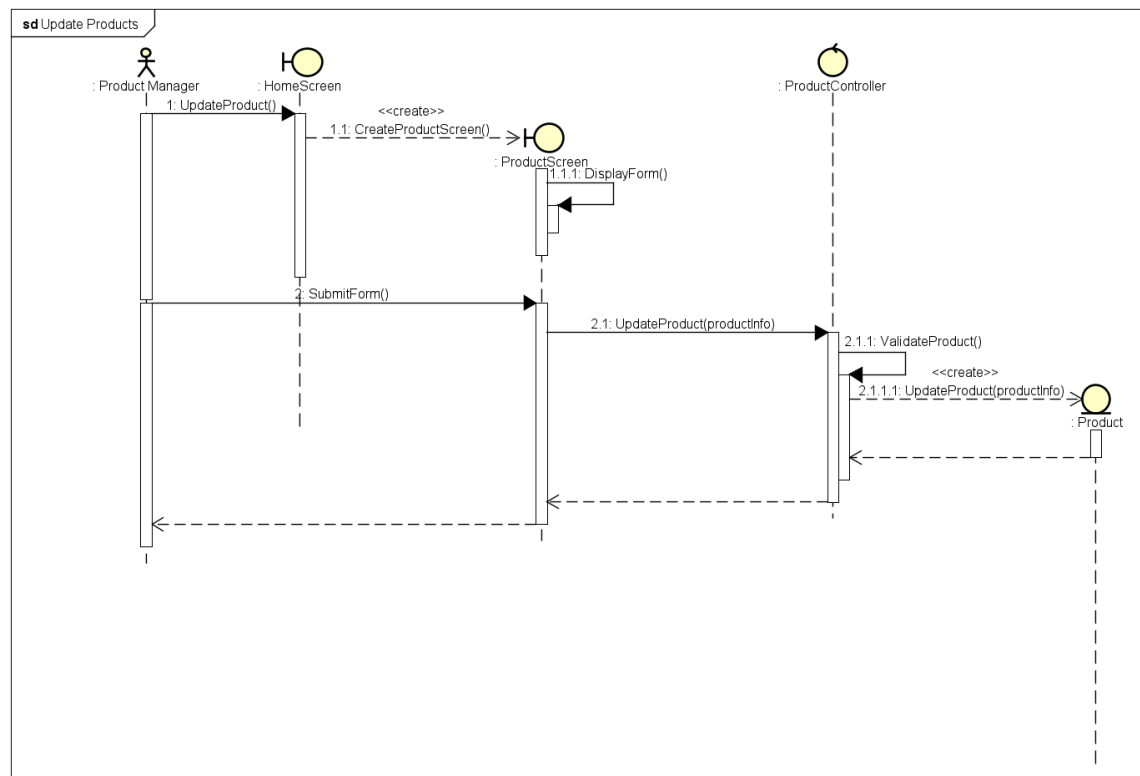
9. Search product



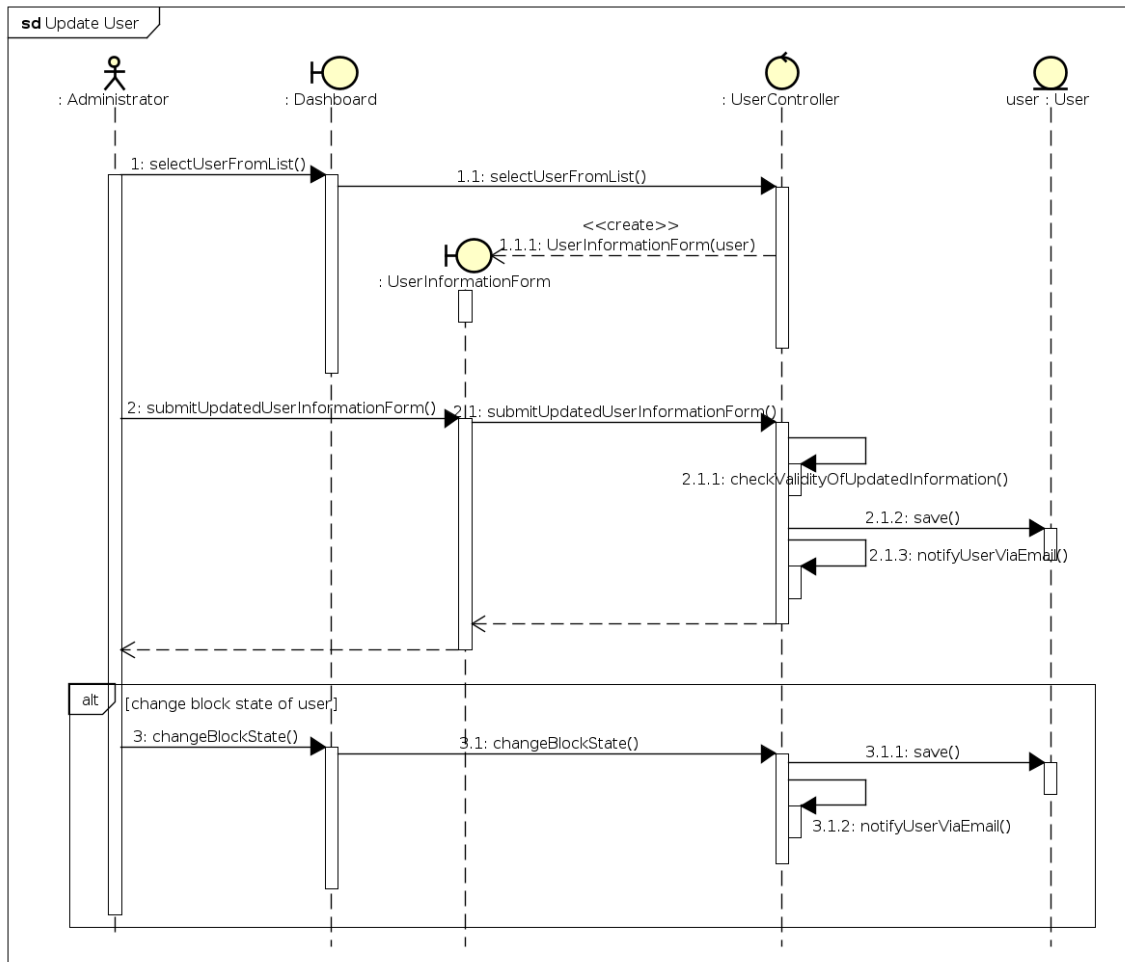
10. Update Cart



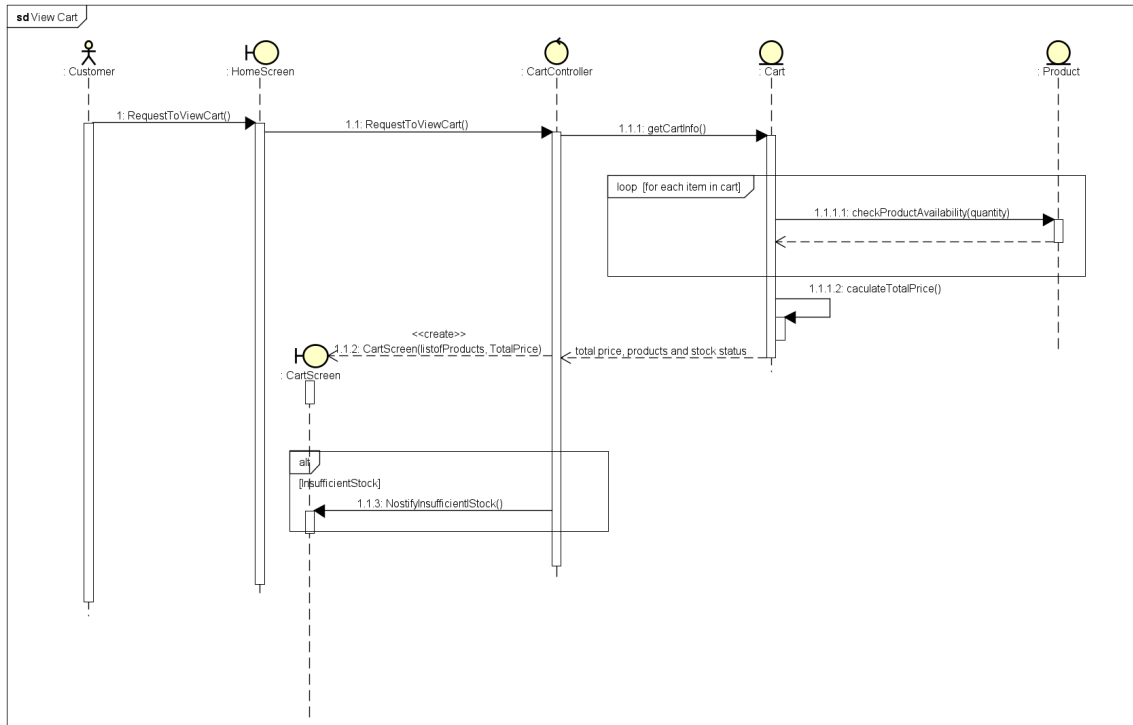
11. Update Products



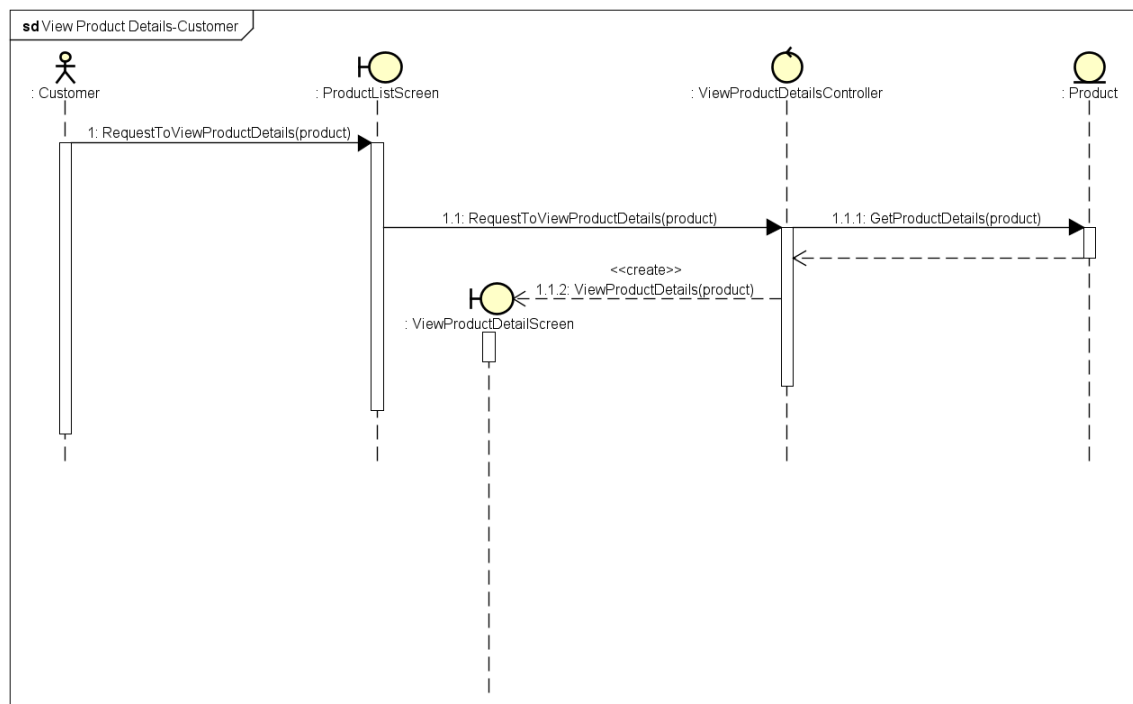
12. Update User



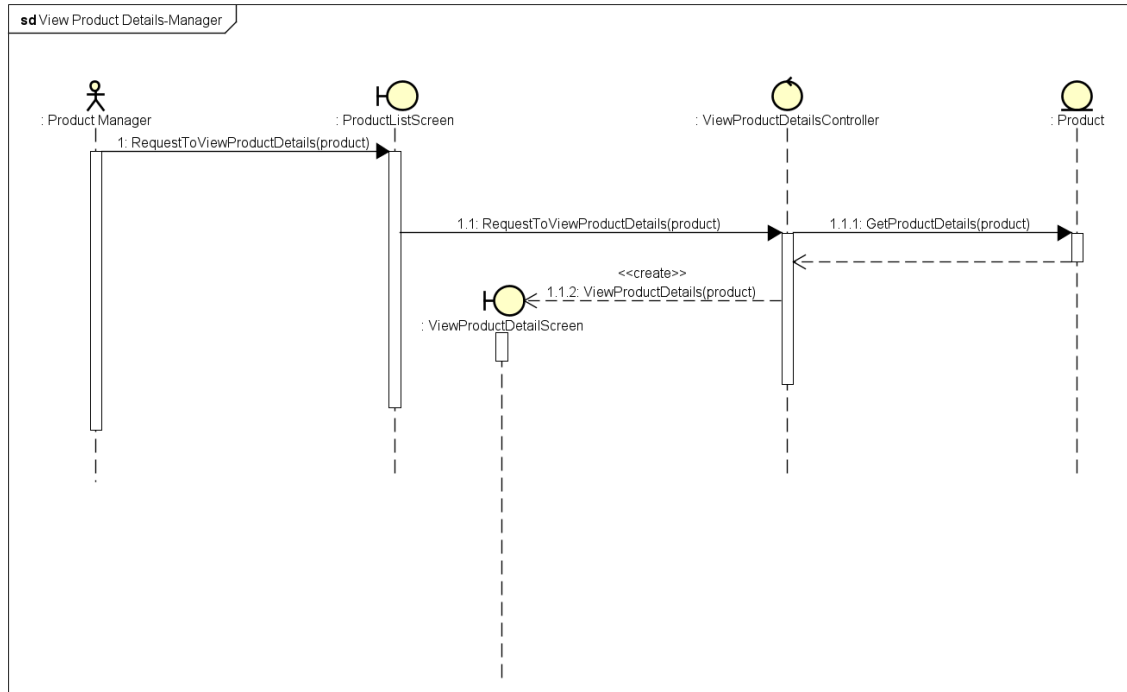
13. View cart



14. View Product Details - Customer

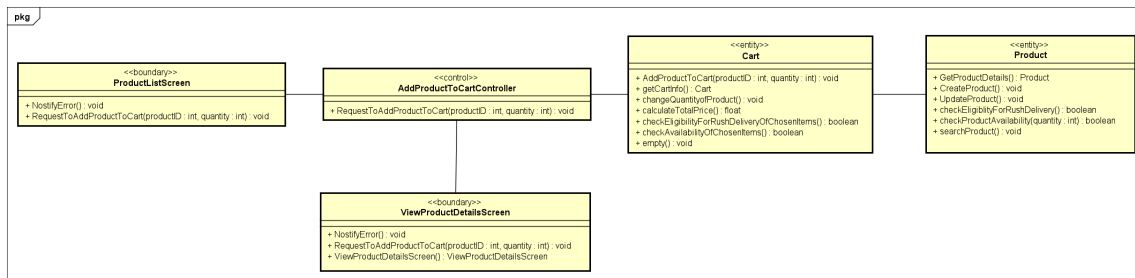


15. View Product Details – Manager

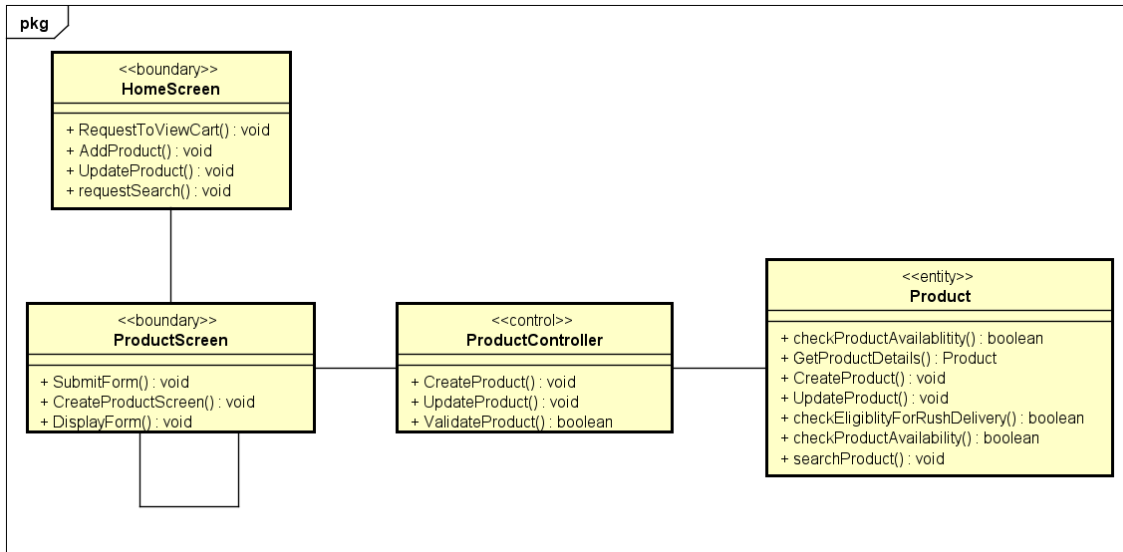


3.3 Analysis Class Diagrams

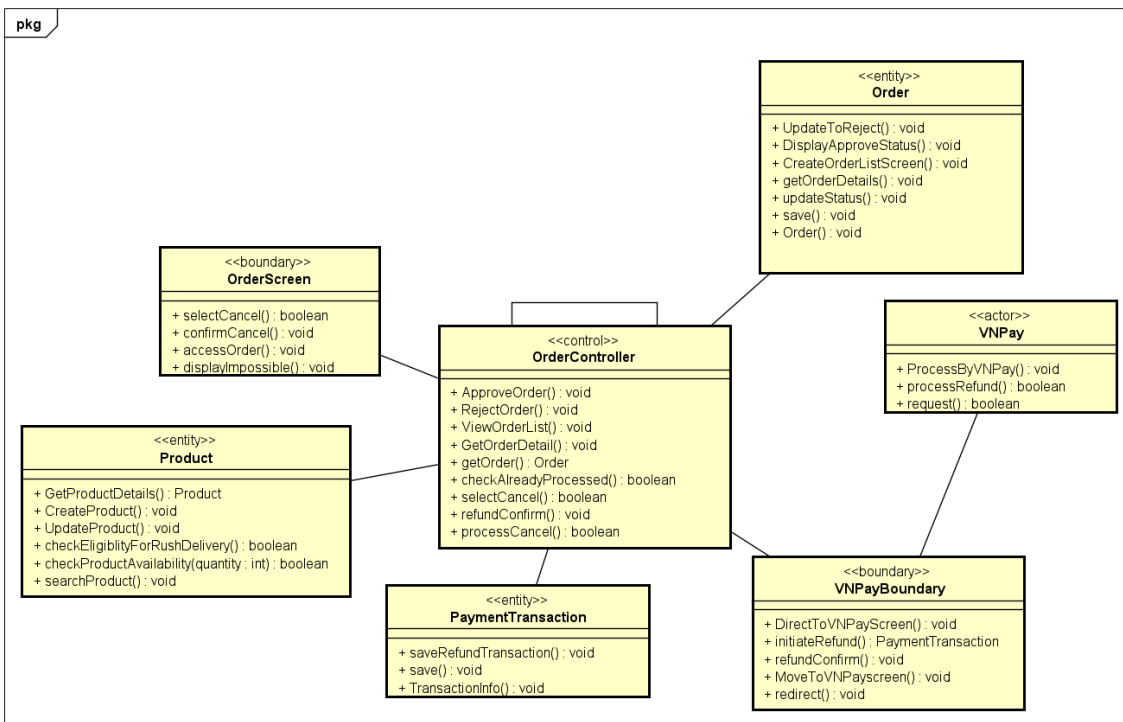
1. Add Products to Cart



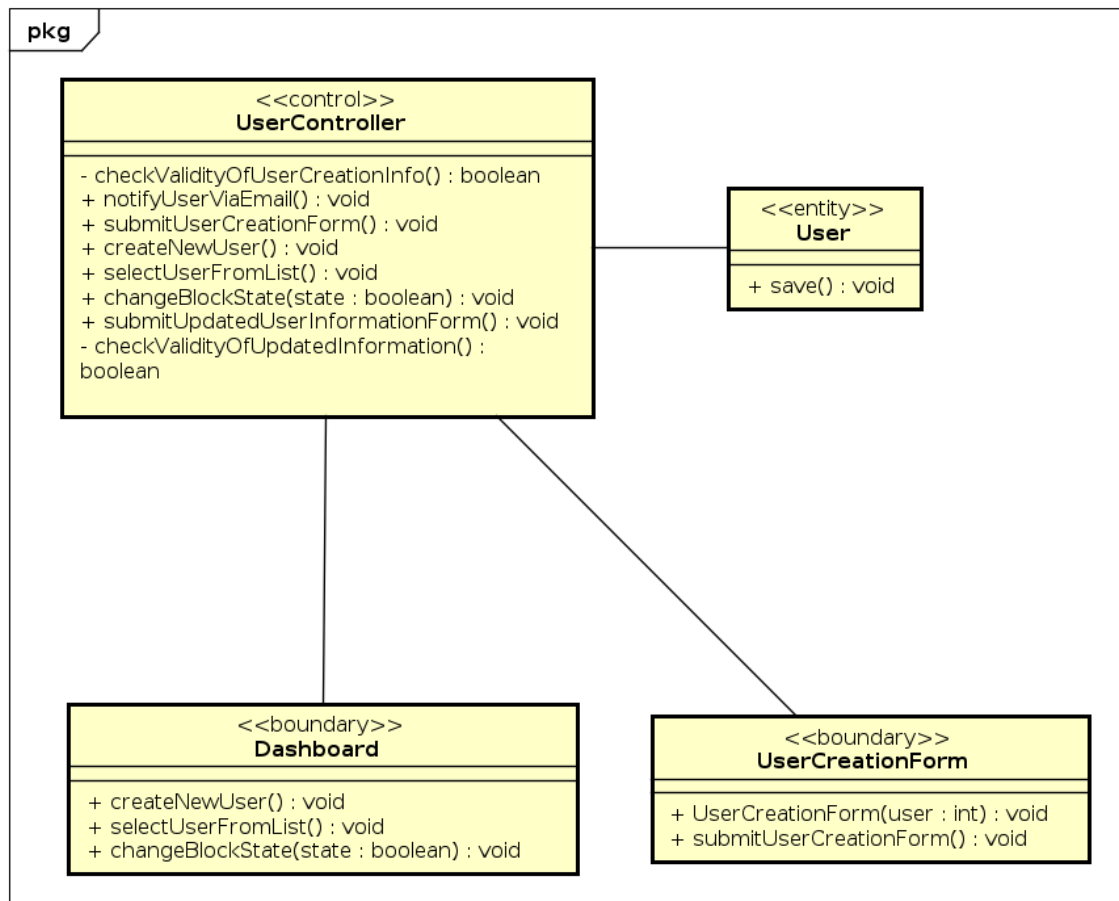
2. Add Products



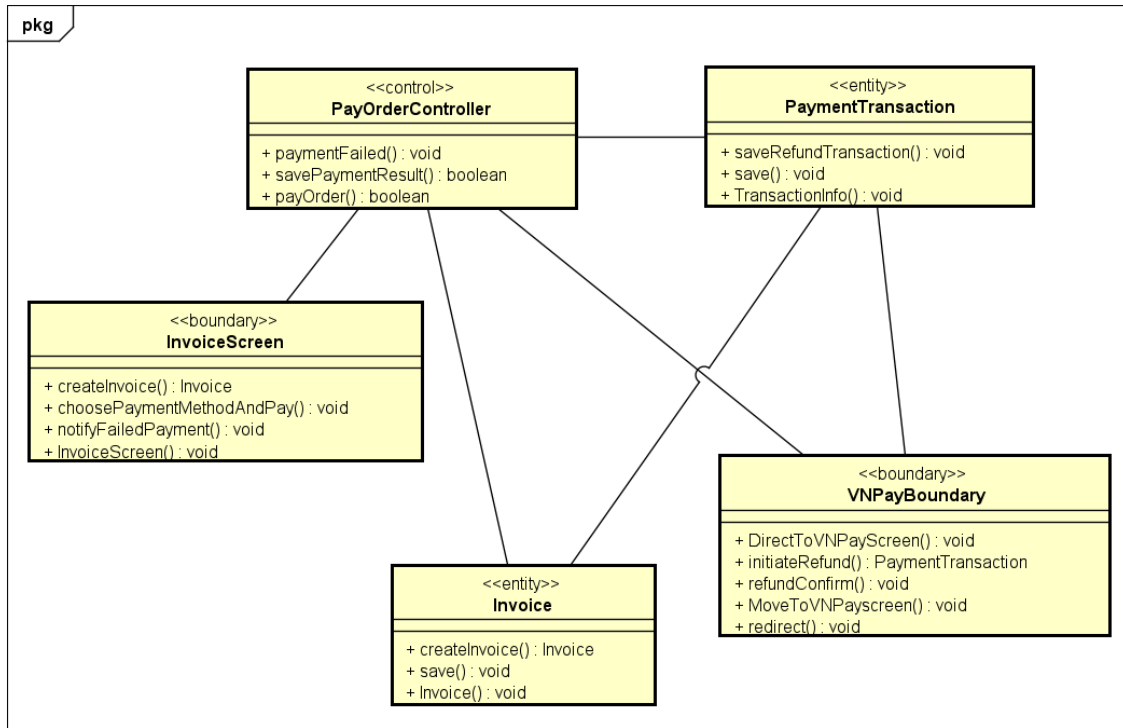
3. Cancel Order



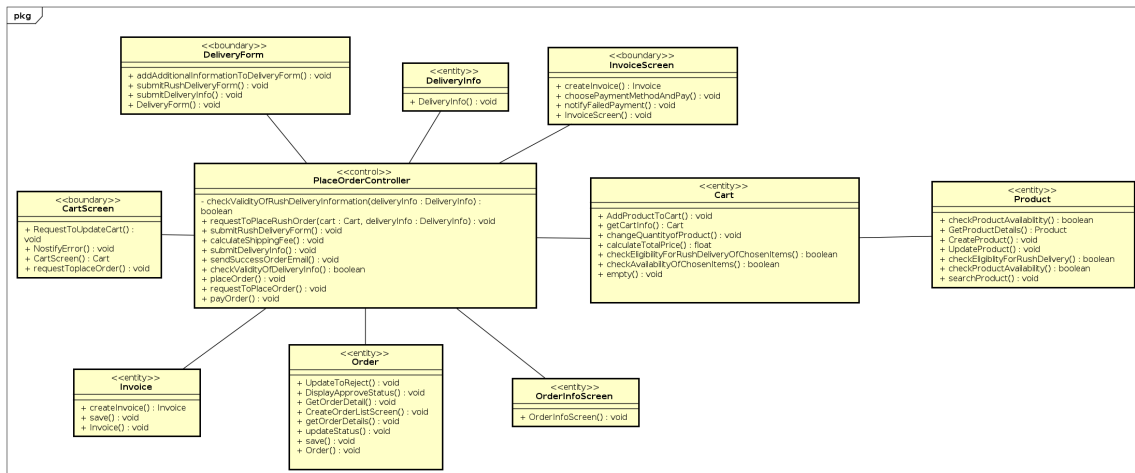
4. Create New User



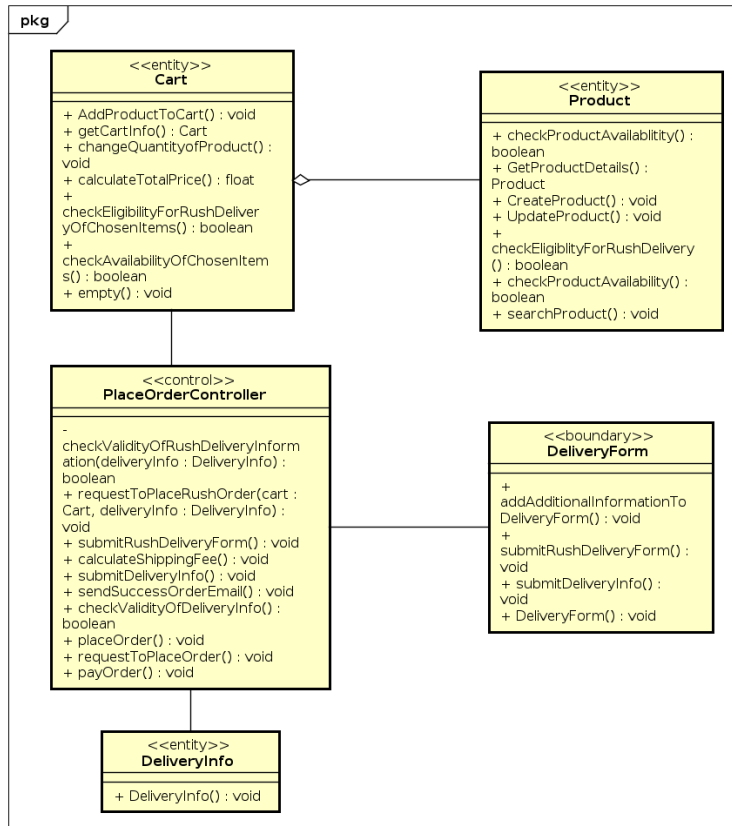
5. Pay Order



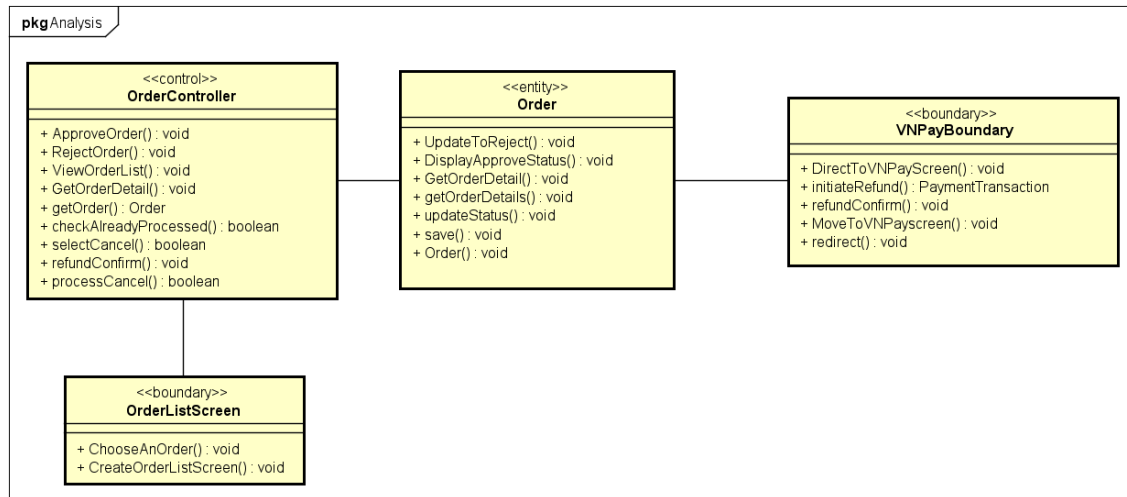
6. Place Order



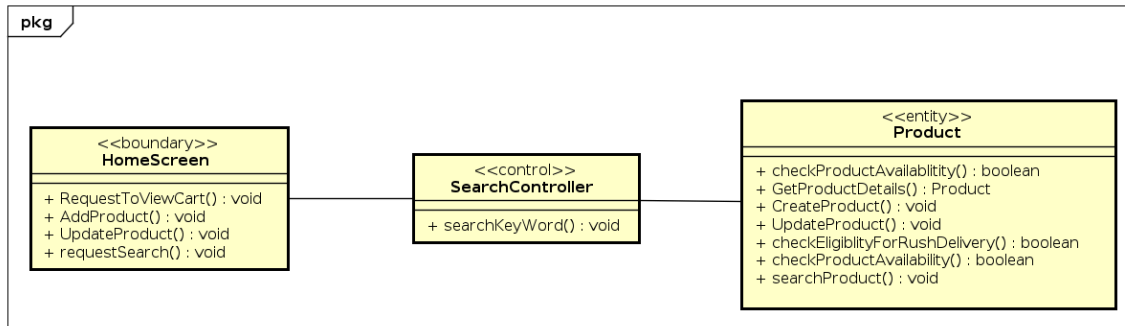
7. Place Rush Order



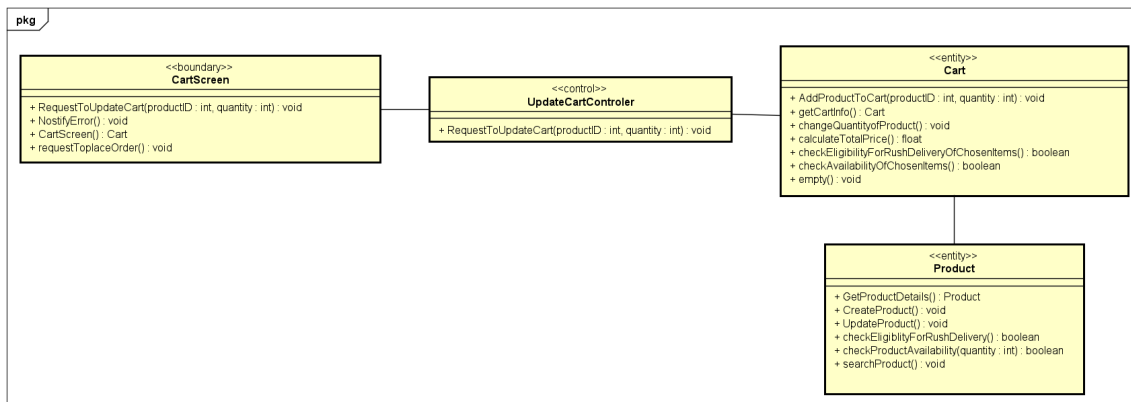
8. Process pending order



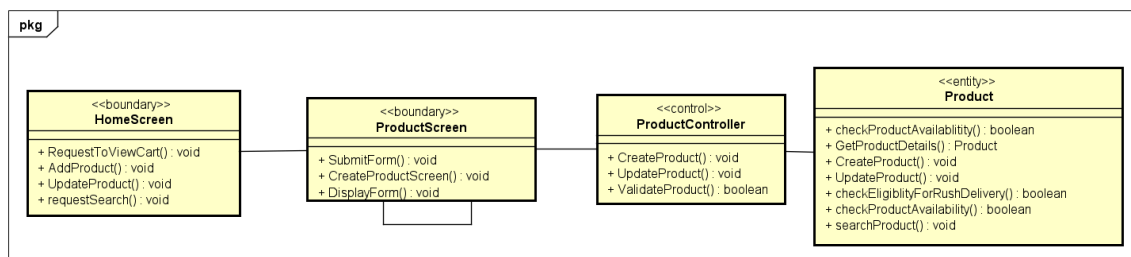
9. Search Products



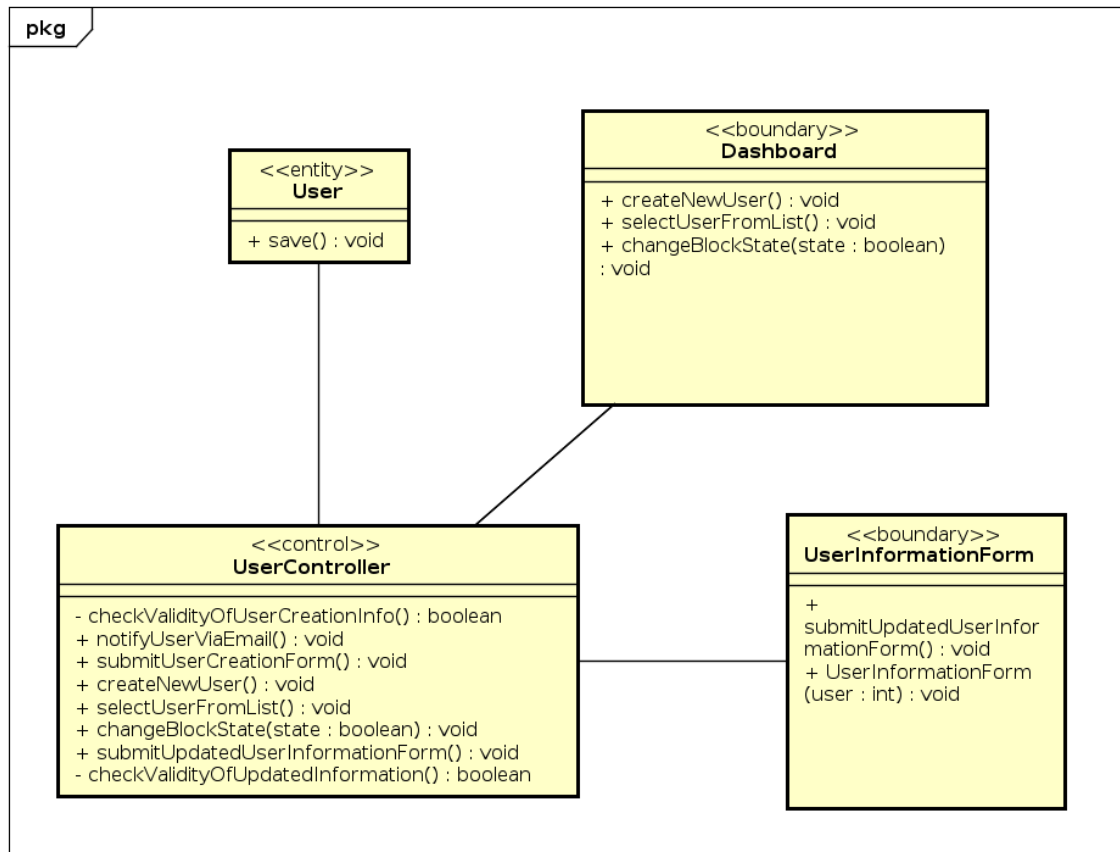
10. Update Cart



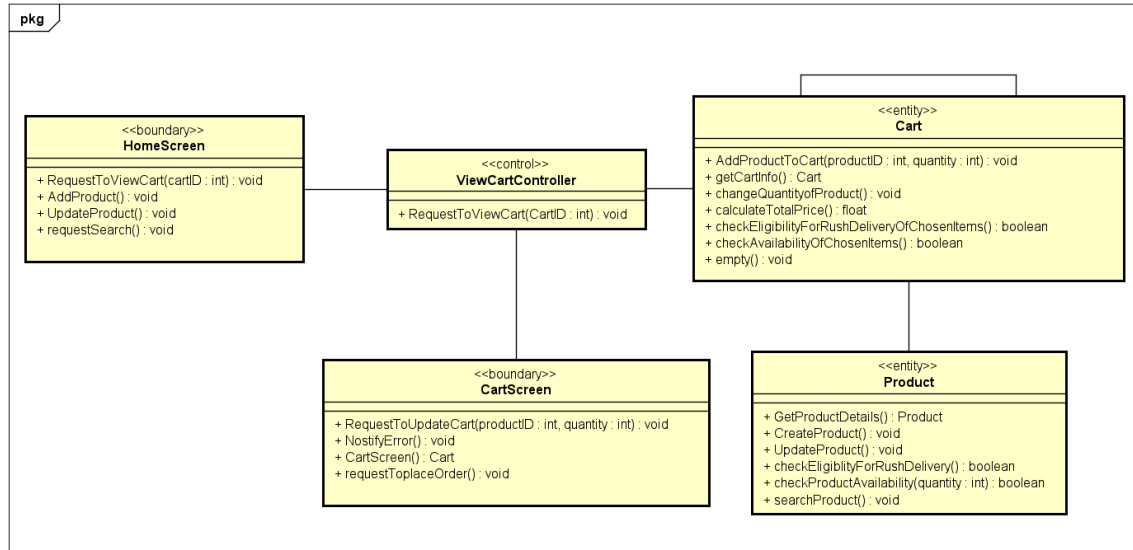
11. Update Products



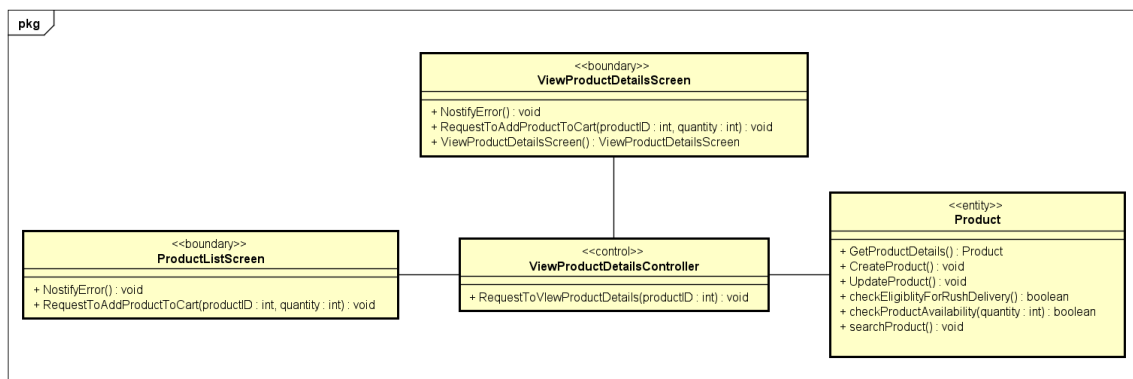
12. Update User



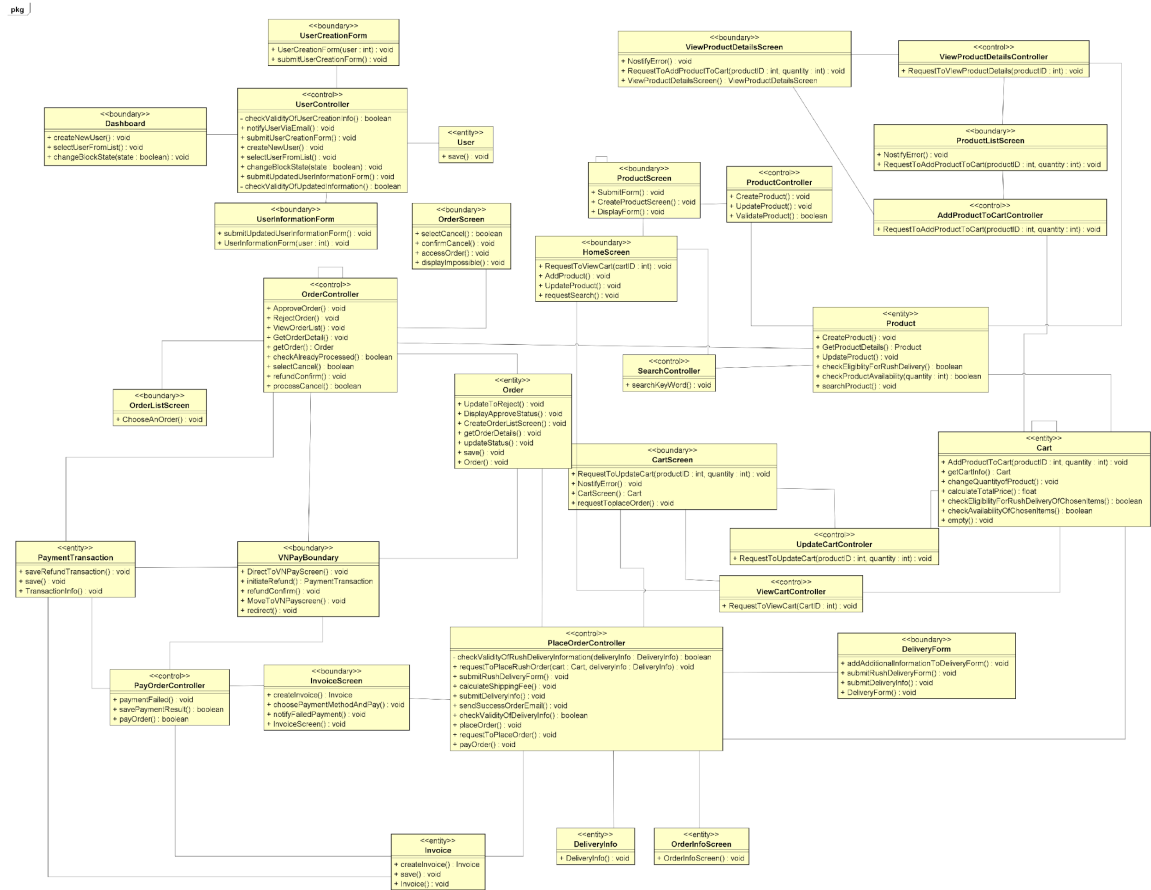
13. View Cart



14. View Product Details



3.4 Unified Analysis Class Diagram



3.5 Security Software Architecture

The security architecture of the ITSS E-commerce application follows a layered approach, ensuring that each tier of the system enforces specific security responsibilities. The architecture separates the concerns across the **Presentation Tier**, **Business Logic Tier**, and **Data Access Tier**, applying best practices to protect the system against common threats such as unauthorized access, injection attacks, and data leakage.

a. Presentation Tier (ReactJS)

Role: Provides the user interface and handles user interactions.

Security Measures:

- Input validation on forms to prevent malformed data.
- Protection against XSS by escaping dynamic content.
- Token/session storage in secure locations (e.g., HttpOnly cookies or localStorage, if applicable).

- Uses HTTPS (recommended in production) to secure communication with the backend.
- Route guarding to prevent unauthorized users from accessing protected views.

b. Business Logic Tier (Spring Boot)

Role: Processes application logic and enforces security policies.

Security Measures:

- **Authentication and Authorization** using Spring Security.
- **Role-Based Access Control (RBAC)** to restrict access to specific endpoints.
- Passwords securely hashed (e.g., with BCrypt) before storage.
- CSRF protection enabled (if session-based authentication is used).
- API endpoints protected using annotations (e.g., `@PreAuthorize`, `@Secured`).
- Validates incoming requests using `@Valid` and input constraints.

c. Data Access Tier (Database - SQLite)

Role: Stores and retrieves application data.

Security Measures:

- Secure storage of sensitive data (e.g., hashed passwords).
- ORM layer (Spring Data JPA) used to prevent SQL injection.
- Access to the database is restricted to the backend only.
- Proper database file permissions set to prevent unauthorized access.

4. Detailed Design

4.1 User Interface Design

4.1.1 Screen Configuration Standardization

a. General Layout

The ITSS Store web interface adopts a clean, responsive design tailored for standard desktop resolutions, starting from 1366×768 pixels. The layout emphasizes simplicity and visual clarity, with clearly separated sections that guide the user through the store’s main functions. The interface is built to ensure a consistent experience across devices, combining accessibility with modern aesthetics.

b. Header (Top Navigation Bar)

At the very top of the screen, a fixed navigation bar provides access to essential functions. On the left side, the ITSS Store logo serves as a recognizable anchor for branding. Centered within the bar is a wide search field, accompanied by a distinct blue “Search” button that encourages product discovery. To the right, users can view their cart via a shopping cart icon with a notification badge, and access account-related options with clear “Login” and “Sign Up” buttons. The layout ensures all elements are horizontally aligned and easy to interact with.

c. Welcome Section (Hero Banner)

The central feature of the homepage is a visually prominent welcome section, designed to immediately engage the user. A bold heading—“Welcome to ITSS Store”—is centered on a soft pastel background, creating a welcoming and modern look. Beneath the heading is a short sentence inviting users to explore the collection of books, CDs, and DVDs. This section also features two clearly visible call-to-action buttons: “Shop Now” in blue and “Browse Categories” in a bordered white style. All components are centrally aligned to maintain focus and visual balance.

d. Category Section

Just below the welcome area, the “Shop by Category” section presents the store’s offerings in a structured and accessible format. The section title is bold and centered, introducing a three-column layout that features individual product categories: Books, CDs, and DVDs. Each category is represented with a large, playful icon, a concise title, and a short tagline that provides context—such as “Discover amazing stories” for books. The grid layout ensures clarity and helps users quickly identify and navigate to the content they are interested in.

e. Typography and Fonts

Text throughout the ITSS Store interface is styled using a consistent visual hierarchy. The default font is Arial, chosen for its readability and wide support across web browsers. Headings and section titles may also use Newsreader or a similar serif font for stylistic contrast. The base font size is approximately 24 pixels, which provides a comfortable reading experience on modern screens. Visual contrast is maintained with black or dark gray text on light backgrounds, and blue is used strategically for call-to-action elements.

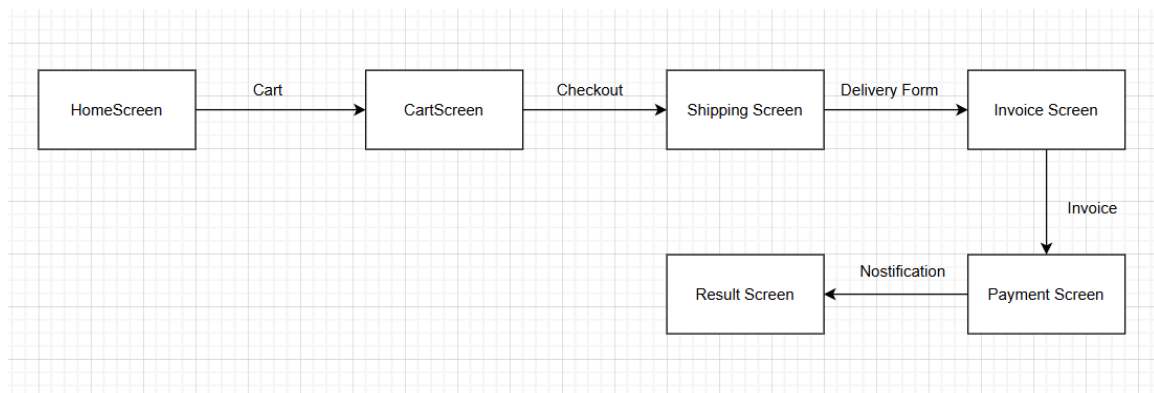
g. Input & Interaction Behavior

Interactive elements such as the search bar and buttons are intuitive and responsive. Input fields validate basic conditions, such as preventing empty submissions in the search bar. Navigation is handled entirely through on-screen elements, such as buttons and links, with no reliance on keyboard shortcuts. While the provided screenshot does not show modal popups, it is expected that any additional windows (e.g., help or login) appear as modal dialogs that temporarily disable interaction with the main content until closed.

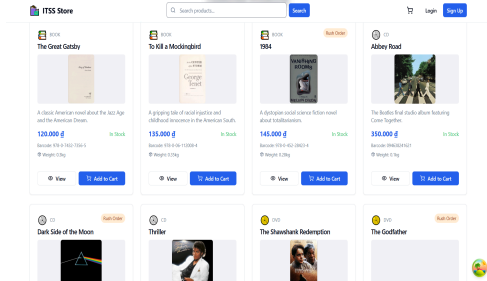
h. Error Handling

The system is designed to provide clear and immediate feedback in case of user errors or invalid input. For example, if a user attempts to perform a search without entering text, an error message should be displayed either next to the input field or as a toast notification. These messages are concise and informative, allowing the user to understand the problem and correct it quickly. This approach ensures a smooth and frustration-free user experience.

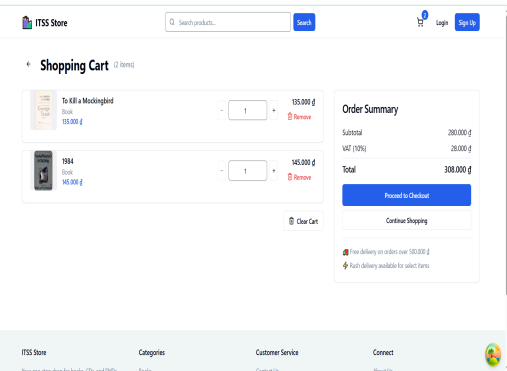
4.1.2 Screen Transition Diagrams

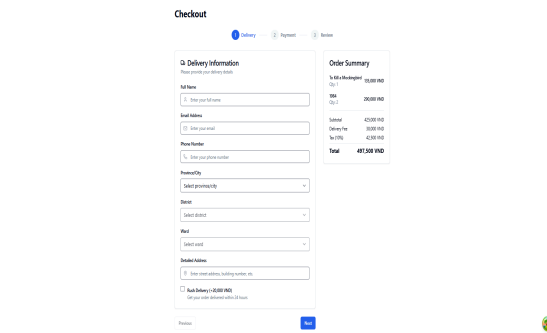


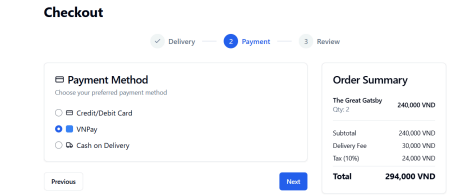
4.1.3 Screen Specifications

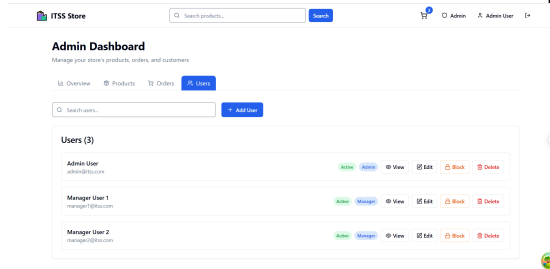
AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	HomeScreen	05/06/2025	Nguyễn Minh Khôi 20226050	Vũ Hải Đăng 20225962	Lê Đại Lâm 20225982
		Control	Operation	Function	
		"Add to Cart" button	Click to add product to cart	Enables users to purchase selected products	
		"View" button	Click to view product details	Navigates to detailed product view	
		Cart icon	Click to access the shopping cart	View and proceed to checkout for selected items	
		Area for display product	Displays product details: name, description, barcode, weight, price	Allows users to view and select products	
		"Login" / "Sign Up" buttons	Click to log in or register	Provides user account access	

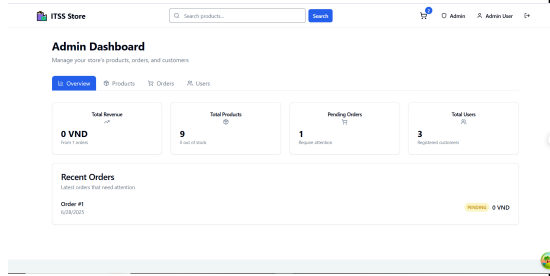
AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	View Product Detail Screen	05/06/2025	Vũ Hải Đăng 20225962	Lê Đại Lâm 20225982	Nguyễn Minh Khôi 20226050
		Control	Operation	Function	
		Back button	Click to return to previous page	Navigate back to the Home or Listing screen	
		Area for product information section	Initial	Provides key product data to the user	
		Quantity selector (- 1 +)	Click	Enables selection of desired purchase amount	
		"Add to Cart" button	Click	Adds selected item(s) to user's shopping cart	

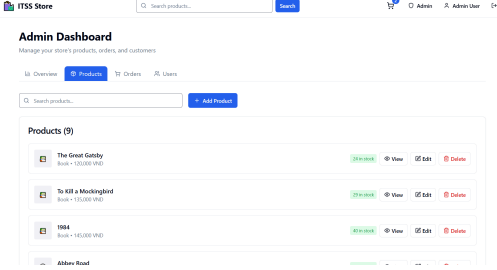
AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	CartScreen	05/06/2025	Lê Đại Lâm 20225982	Bùi Xuân Sơn 20226065	Phạm Thành Nam 20225989
		Control	Operation	Function	
		Quantity stepper (±)	Click to increase/decrease item quantity	Allows users to adjust the quantity of each product in the cart.	
		Remove button	Click to remove product from cart	Removes the selected product from the cart.	
		"Clear Cart" button	Click to remove all products	Empties the cart in the current session.	

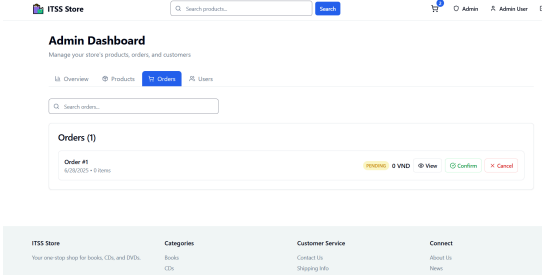
AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Delivery Info Screen	05/06/2025	Phạm Thành Nam 20225989	Lê Đại Lâm 20225982	Vũ Hải Đăng 20225962
		Control	Operation	Function	
		Next button	Validates form and proceeds to Payment step	Initiates delivery info validation and shipping fee recalculation before showing payment options.	
		Order Summary pane	Displays selected items and calculated prices	Summarizes subtotal, delivery fee, 10% VAT, and total amount before payment step.	
		Rush Delivery checkbox	User optionally enables rush delivery	Adds 10,000–20,000 ^d fee depending on configuration; only available in inner city (per AIMS spec).	

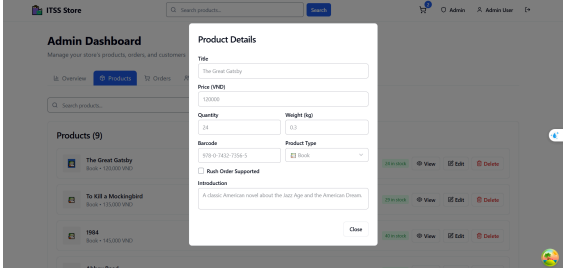
AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Screen	05/06/2025	Vũ Hải Đăng 20225962	Phạm Thành Nam 2022598	Bùi Xuân Sơn 20226065
		Control	Operation	Function	
		Payment method options	Click	Allow user to choose between Credit/Debit, VNPAY, or Cash on Delivery	
		Order Summary panel	Initial	Show order details including item names, quantities, subtotal, delivery fee, tax, and total	
		“Previous” button	Click	Return to the previous step (Delivery)	
		“Next” button	Click	Proceed to the next step (Review)	

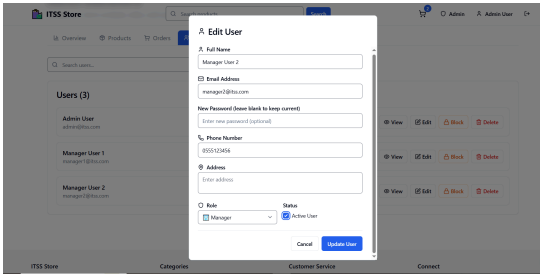
AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Manager User Screen	05/06/2025	Bùi Xuân Sơn 20226065	Vũ Hải Đăng 20225962	Phạm Thành Nam 20225989
		Control	Operation	Function	
		Search Users bar	Enter keywords to filter users	Allows admin to quickly search and locate specific users.	
		Add User" button	Click to open user creation form	Enables admin to create a new user account (email, roles, password, etc.).	
		"Edit" button	Click to edit user info	Allows admin to update user profile, email, or assign/change roles.	
		Block" button	Click to block user	Disables account access; prevents login or any privileged actions.	
		"Delete" button	Click to remove user	Deletes the user account from the system.	
		Top bar "Admin" icon	Click to return to admin homepage	Navigates to the main Admin Dashboard (overview panel).	
		View button	Click to view user information	Opens detailed information about the user (roles, status, contact)	

AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Admin Dashboard overview screen	05/06/2025	Nguyễn Minh Khôi 20226050	Lê Đại Lâm 20225982	Phạm Thành Nam 20225989
		Control	Operation	Function	
		Total Revenue card	Display only	Shows accumulated revenue and number of orders; useful for business monitoring.	
		Total Products card	Display only	Indicates the number of products in the store, and stock status summary.	
		Recent Orders list	Shows latest orders requiring attention	Allows quick access to newest orders; clicking may navigate to full order details.	

AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Manage Product Screen	05/06/2025	Lê Đại Lâm 20225982	Nguyễn Minh Khôi 20226050	Bùi Xuân Sơn 20226065
		Control	Operation	Function	
		Search Products bar	Type keywords to search products	Allows the admin to filter the product list by name, category, etc.	
		"Add Product" button	Click to open product creation form	Lets the admin add a new product with all required info (value, price, barcode, category, quantity...).	
		"Edit" button	Click to edit product	Allows editing product info (e.g., price, stock), subject to AIMS constraints (e.g., max 2 price edits/day).	
		Delete button	Click to delete product	Allows deletion of up to 10 products at a time; actions are logged as per AIMS rules.	
		Stock status label	Display only (e.g., "29 in stock")	Shows current inventory count of each product.	
		"View" button	Click to view product details	Opens detailed information of the selected product	

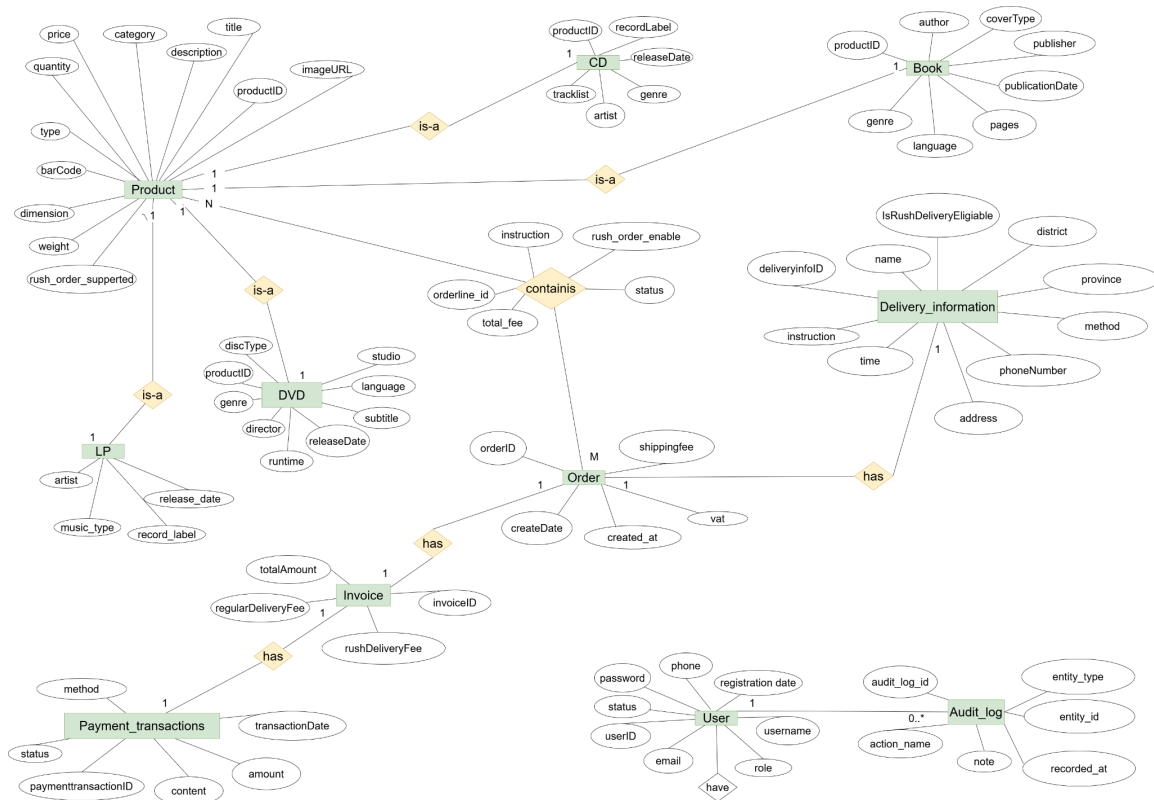
AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Order Manager Screen	05/06/2025	Phạm Thành Nam 20225989	Bùi Xuân Sơn 20226065	Vũ Hải Đăng 20225962
		Control	Operation	Function	
		Search bar (Search orders...)	Input	Search for orders by ID, date, or customer	
		Order card (e.g., Order #1)	Display	Show basic order info like ID, date, item count	
		Status badge (PENDING)	Initial	Indicates the current status of the order	
		Total amount label (0 VND)	Display	Displays total price of the order	
		“View” button	Click	Opens the detailed view of the order	
		“Confirm” button	Click	Updates order status to "Confirmed" (approved)	
		“Cancel” button	Click	Cancels the order and updates status accordingly	

AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Manage Product Screen	05/06/2025	Vũ Hải Đăng 20225962	Nguyễn Minh Khôi 20226050	Lê Đại Lâm 20225982
		Control	Operation	Function	
		Title input field	Input	Enter or edit product name	
		Price (VND) input	Input	Enter or update product price	
		Quantity input	Input	Set available quantity in stock	
		Weight (kg) input	Input	Set product weight	
		Barcode input	Input	Enter or view product barcode	
		Product Type dropdown	Select	Choose product category (e.g., Book, CD, DVD)	
		Rush Order Supported checkbox	Toggle	Mark product as eligible for rush order	
		Introduction textarea	Input	Provide a short product description	
		Close button	Click	Close the product details modal without saving	

AIMS Software		Date of creation	Approved by	Reviewed by	Person in charge
Screen specification	Manage User Screen	05/06/2025	Bùi Xuân Sơn 20226065	Phạm Thành Nam 20225989	Vũ Hải Đăng 20225962
		Control	Operation	Function	
		Full Name input	Input	Allows the user to edit the full name of the account	
		Email Address input	Input	Allows editing of the email associated with the user	
		Password input field	Input	Allows setting a new password (optional)	
		Phone Number input	Input	Allows updating the user's phone number	
		Address input	Input	Allows user to input/edit address	
		Role dropdown	Select	Allows switching between roles (e.g., Admin, Manager)	
		Active User checkbox	Toggle	Enables or disables user account	
		Cancel button	Click	Cancels the editing process and closes the form	
		Update User button	Click	Cancels the editing process and closes the form	

4.2 Data Modeling

4.2.1 Conceptual Data Modeling



4.2.2 Database Design

4.2.2.1 Database Management System

We decided to use SQLite as a Database Management System.

SQLite3 is a lightweight, serverless, self-contained RDBMS that stores all data in a single file. It's ideal for mobile apps, embedded systems, and applications requiring fast, low-overhead data management. It offers ACID compliance and cross-platform support.

4.2.2.2 Database Diagram



4.2.2.3 Database Detail Design

Table 1. Product

#	PK	FK	Column Name	Data type	Mandato ry	Description
1	x		product_id	INTEGER	Yes	Auto-generated primary key for the product
2			type	VARCHAR	No	Category or type of the product
3			barcode	VARCHAR	No	Unique barcode identifier for the product
4			condition	VARCHAR	No	Condition of the product (e.g., new, used)
5			created_at	TIMESTAM P	No	Timestamp when the product record was created
6			dimensions	VARCHAR	No	Physical dimensions of the product

7			image_url	VARCHAR	No	URL linking to the product image
8			import_date	VARCHAR	No	Date the product was imported into the system
9			introduction	TEXT	No	Brief introduction or description of the product
10			price	INTEGER	No	Price of the product
11			quantity	INTEGER	No	Number of available units in stock
12			rush_order_supported	BOOLEAN	No	Indicates whether rush (urgent) orders are supported for this product
13			title	VARCHAR	No	Title or name of the product
14			updated_at	TIMESTAMP	No	Timestamp when the product record was last updated
15			weight	FLOAT	No	Weight of the product

Table 2. CD

#	P K	F K	Column Name	Data type	Mandato ry	Description
1			artists	Varchar(500)	No	List of performing artists (comma-separated)
2			genre	Varchar(100)	No	Music genre (e.g., Pop, Rock, Jazz)
3			record_label	Varchar(200)	No	Record label that published the CD
4			release_date	Varchar(255)	No	CD release date
5			track_list	TEXT	No	Comma-separated list of tracks
6	x	x	product_id	BigInt	Yes	Primary key and FK to Product table

Table 3. Book

#	P K	F K	Column Name	Data type	Mandatory	Description
1	x		book_id	Integer	Yes	ID, auto increment
2			authors	Varchar(500)	No	List of authors
3			cover_type	Varchar(50)	No	Hardcover, Paperback, etc.
4			genre	Varchar(100)	No	Book genre (e.g., Fiction, Drama)
5			page_count	Integer	No	Total number of pages
6			publication_date	Varchar(255)	No	Date of publication
7			publishers	Varchar(200)	No	Publisher name(s)
8		x	product_id	BigInt	Yes	FK to Product table (assumed)

Table 4. DVD

#	P K	F K	Column Name	Data type	Mandatory	Description
1			directors	Varchar(500)	No	List of directors (comma-separated)
2			duration_minutes	Integer	No	Total duration in minutes
3			dvd_type	Varchar(50)	No	Type of DVD (e.g., Standard, Blu-ray)
4			genre	Varchar(100)	No	Genre of the movie
5			rating	Varchar(10)	No	Rating (e.g., G, PG, PG-13, R)
6			release_date	Varchar(255)	No	Release date of the DVD
7			studio	Varchar(200)	No	Studio that produced or released the DVD
8	x	x	product_id	BigInt	Yes	Primary key and FK to Product table

Table 5. DeliveryInfo

#	P K	F K	Column Name	Data type	Mandato ry	Description
1	x		delivery_id	Integer	Yes	Primary key, unique ID for delivery
2			address	TEXT	Yes	Full delivery address
3			delivery_fee	Integer	No	Shipping cost
4			delivery_message	TEXT	No	Optional message from customer
5			district	Varchar(100)	Yes	District name
6			email	Varchar(255)	Yes	Recipient's email address
7			name	Varchar(255)	Yes	Recipient's full name
8			phone	Varchar(20)	Yes	Phone number
9			province	Varchar(100)	Yes	Province/City
10			ward	Varchar(100)	Yes	Ward/Commune

Table 6. Invoice

#	P K	F K	Column Name	Data type	Mandato ry	Description
1	x		invoice_id	Integer	Yes	Primary key, unique invoice ID
2			created_at	Timestamp	No	Time when invoice was generated
3			description	TEXT	No	Optional note or description
4			paid_at	Timestamp	No	Time when invoice was paid
5			payment_method	Varchar(50)	No	Payment method (e.g., CASH, CARD, VNPay, etc.)
6			payment_status	Varchar(255)	No	Enum: PENDING, PAID, FAILED, REFUNDED, CANCELLED

7		x	order_id	BigInt	Yes	Unique FK to Order table
---	--	---	----------	--------	-----	--------------------------

Table 7. Payment Transaction

#	P K	F K	Column Name	Data type	Mandato ry	Description
1	x		transaction_id	INTEGER	Yes	Auto-generated primary key for the payment transaction
2		x	invoice_id	INTEGER	No	Foreign key referencing Invoice.invoice_id
3			amount	INTEGER	No	Amount of money involved in the transaction
4			created_at	TIMESTAMP	No	Timestamp when the transaction was created
5			failure_reason	VARCHAR	No	Reason for transaction failure, if any
6			payment_method	VARCHAR	No	Method used for the payment (e.g., Credit Card, PayPal)
7			processed_at	TIMESTAMP	No	Timestamp when the transaction was processed
8			status	VARCHAR	No	Current status of the transaction (e.g., success, failed, pending)

Table 8. User

#	P K	F K	Column Name	Data type	Mandato ry	Description
1	x		user_id	INTEGER	Yes	Auto-generated primary key for the user
2			email	VARCHAR	No	User's email address
3			is_active	BOOLEAN	No	Indicates whether the user account is active
4			name	VARCHAR	No	Full name of the user

5			password	VARCHAR	No	Encrypted user password
6			phone	VARCHAR	No	Contact phone number of the user
7			registration_date	TIMESTAMP	No	Timestamp when the user registered
8			role	VARCHAR	No	Role assigned to the user (e.g., admin, customer, staff)
9			salary	FLOAT	No	Salary of the user, applicable for employee roles

Table 9. LP record

#	P K	F K	Column Name	Data type	Mandato ry	Description
1	x	x	product_id	INTEGER	Yes	Primary key and foreign key referencing Product.product_id
2			artist	VARCHA R	No	Name of the artist or band
3			music_type	VARCHA R	No	Genre or type of music (e.g., rock, jazz, classical)
4			record_label	VARCHA R	No	Record label that produced or distributed the LP
5			release_date	VARCHA R	No	Release date of the LP
6			rpm	INTEGER	No	Rotations per minute (e.g., 33, 45) of the vinyl record
7			size_inches	FLOAT	No	Size of the vinyl record in inches (e.g., 7", 12")
8			sleeve_condition	VARCHA R	No	Condition of the LP sleeve or cover (e.g., Mint, VG+, Good)
9			tracklist	TEXT	No	Full list of tracks included on the LP

10			vinyl_condition	VARCHAR	No	Physical condition of the vinyl (e.g., Near Mint, Very Good, Fair)
----	--	--	-----------------	---------	----	--

Table 10. OrderItems

#	P K	F K	Column Name	Data type	Mandatory	Description
1	x		order_id	INTEGER	Yes	Auto-generated primary key for the order item
2		x	delivery_id	INTEGER	No	Foreign key referencing DeliveryInformation.delivery_id
3			created_at	TIMESTAMP	No	Timestamp when the order item was created
4			status	VARCHAR	No	Current status of the order (e.g., pending, shipped, delivered, canceled)
5			total_after_vat	INTEGER	No	Total amount of the order item after VAT is applied
6			total_before_vat	INTEGER	No	Total amount of the order item before VAT is applied
7			updated_at	TIMESTAMP	No	Timestamp of the last update to the order item
8			vat_percentage	INTEGER	No	VAT percentage applied to the order item

Table 11. Audit Log

#	P K	F K	Column Name	Data type	Mandatory	Description
1	x		audit_log_id	Integer	Yes	Primary key, auto-increment log ID
2			action_name	Varchar(255)	Yes	Name of the action performed
3			action_type	Varchar(255)	No	Enum: CREATE, UPDATE, DELETE, etc.
4			entity_id	BigInt	No	ID of the affected entity

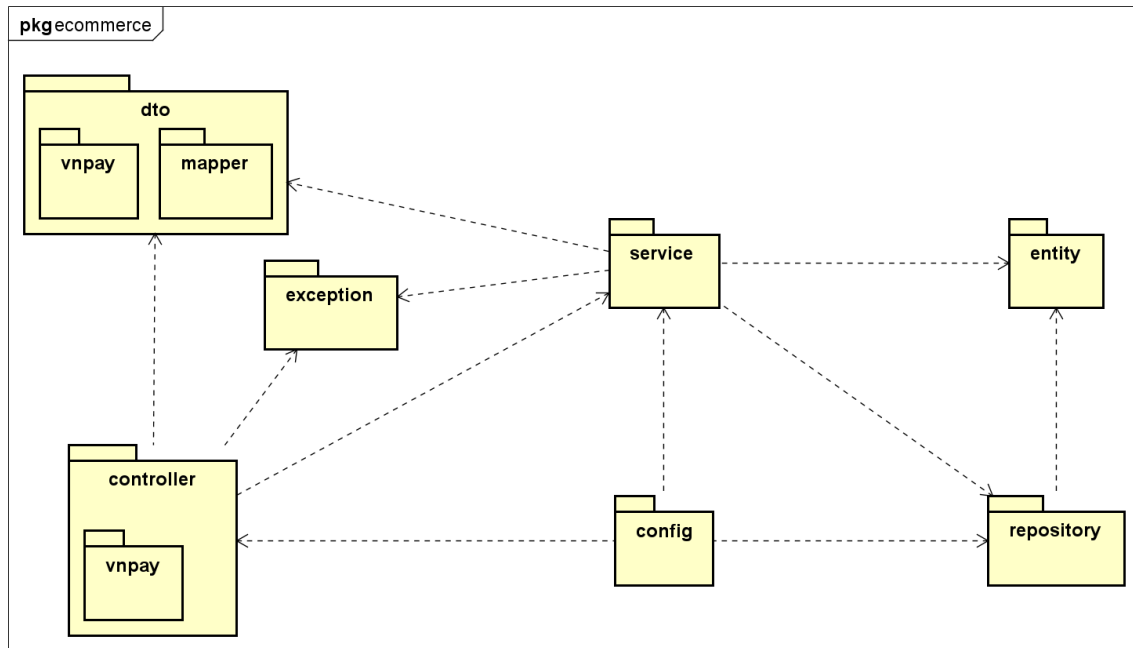
5			entity_type	Varchar(50)	No	Type of entity affected
6			note	TEXT	No	Optional note about the action
7			recorded_at	Timestamp	No	Timestamp when the action was recorded
8		x	user_id	BigInt	No	ID of the user who performed the action

Table 12. OrderItem

#	PK	FK	Column Name	Data type	Mandatory	Description
1	x		order_item_id	INTEGER	Yes	Auto-generated primary key for the individual order item
2		x	order_id	INTEGER	No	Foreign key referencing OrderItems.order_id
3		x	product_id	INTEGER	No	Foreign key referencing Product.product_id
4			delivery_time	VARCHAR	No	Estimated or requested delivery time for this item
5			instructions	TEXT	No	Special instructions related to the delivery or handling of the item
6			quantity	INTEGER	No	Number of units ordered for this product
7			rush_order_using	BOOLEAN	No	Indicates whether a rush (urgent) order service is being used
8			status	VARCHAR	No	Status of the item (e.g., processing, shipped, delivered)
9			total_fee	INTEGER	No	Total fee charged for this item (may include rush fee or other charges)

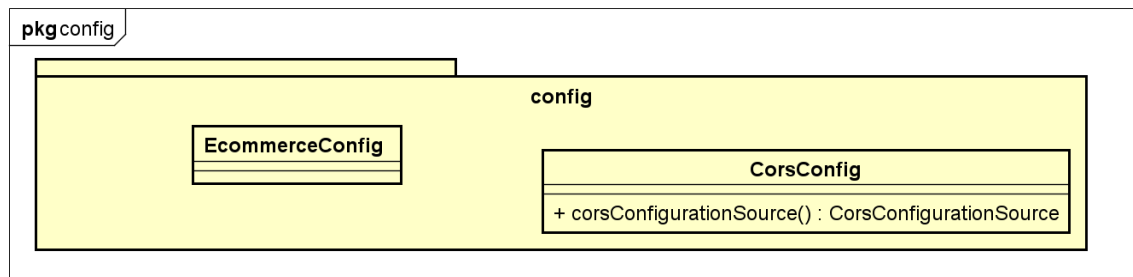
4.3 Class Design

4.3.1 General Class Diagram

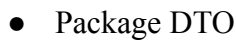


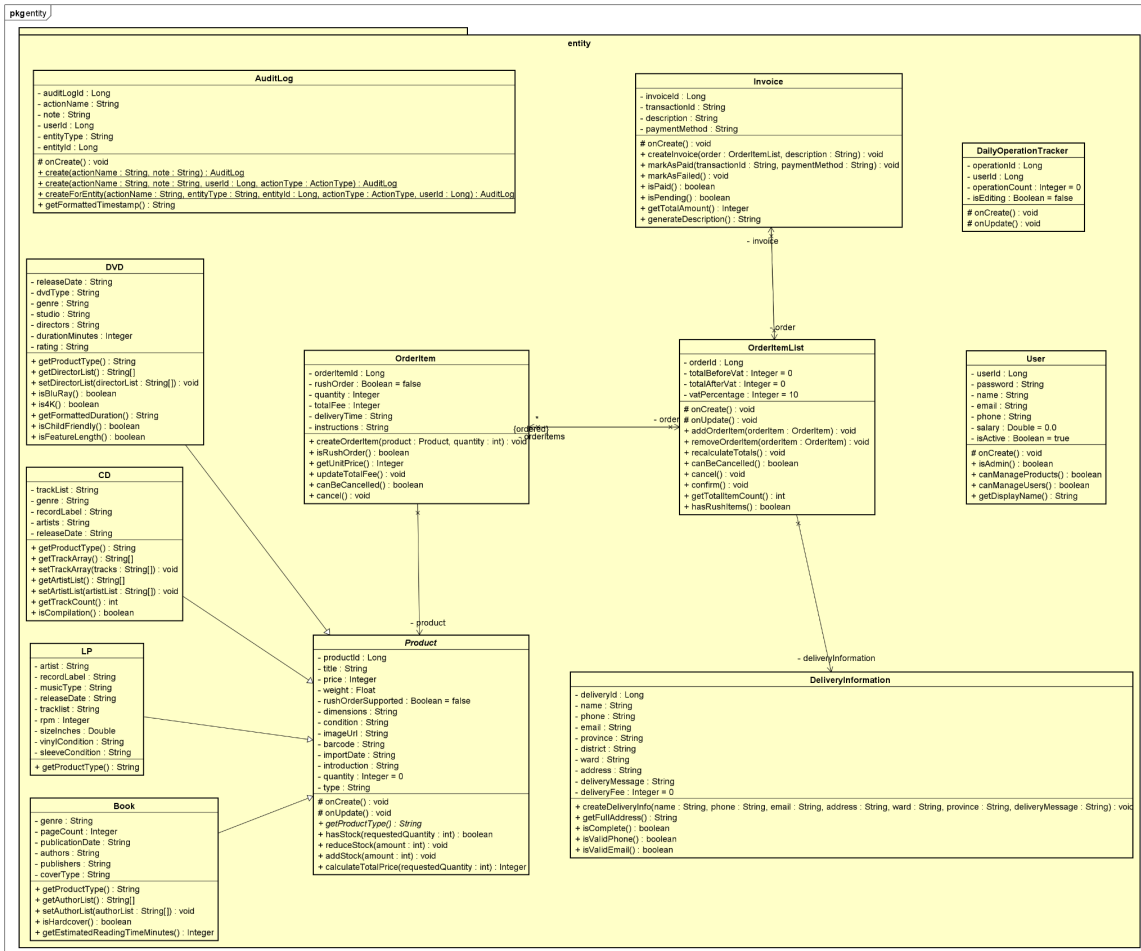
4.3.2 Class Diagrams

- Package Config

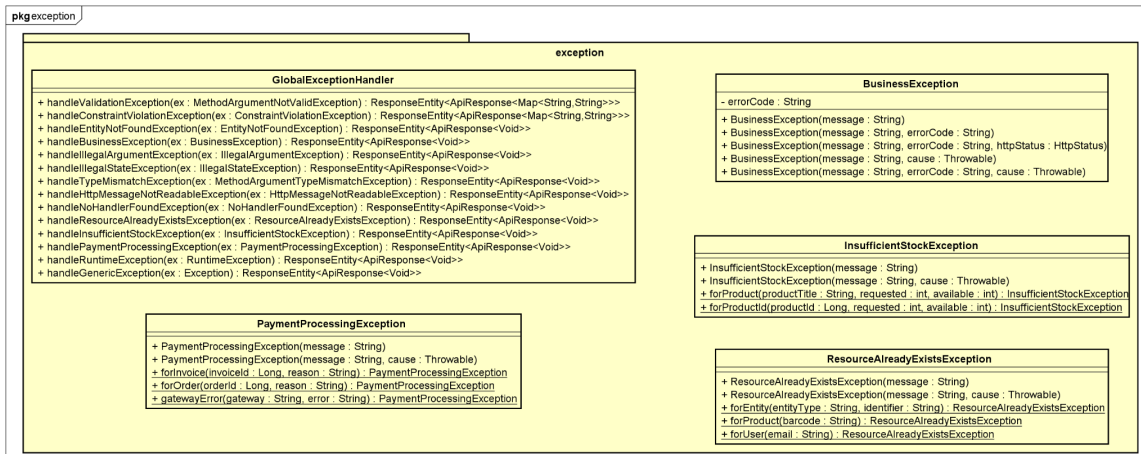


- Package Controller

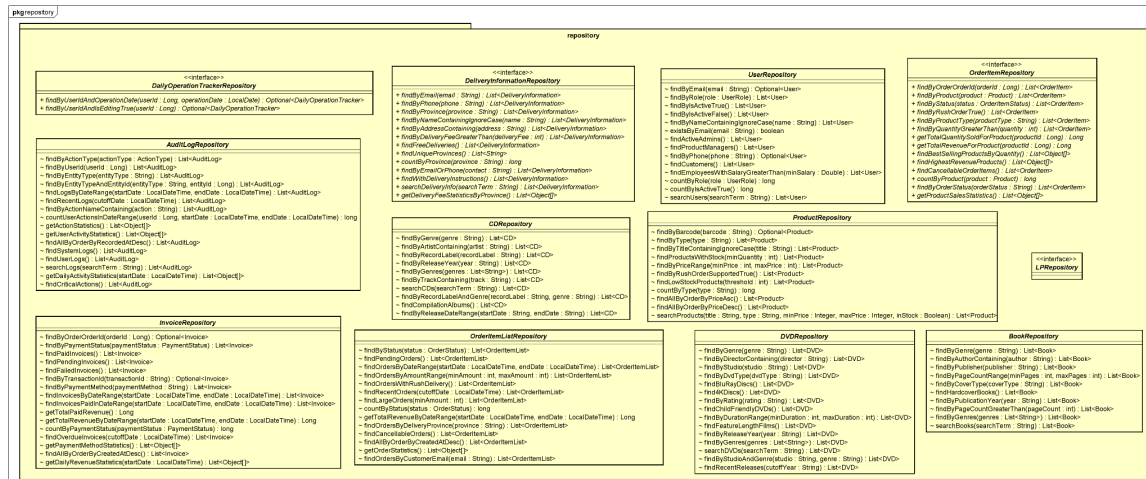




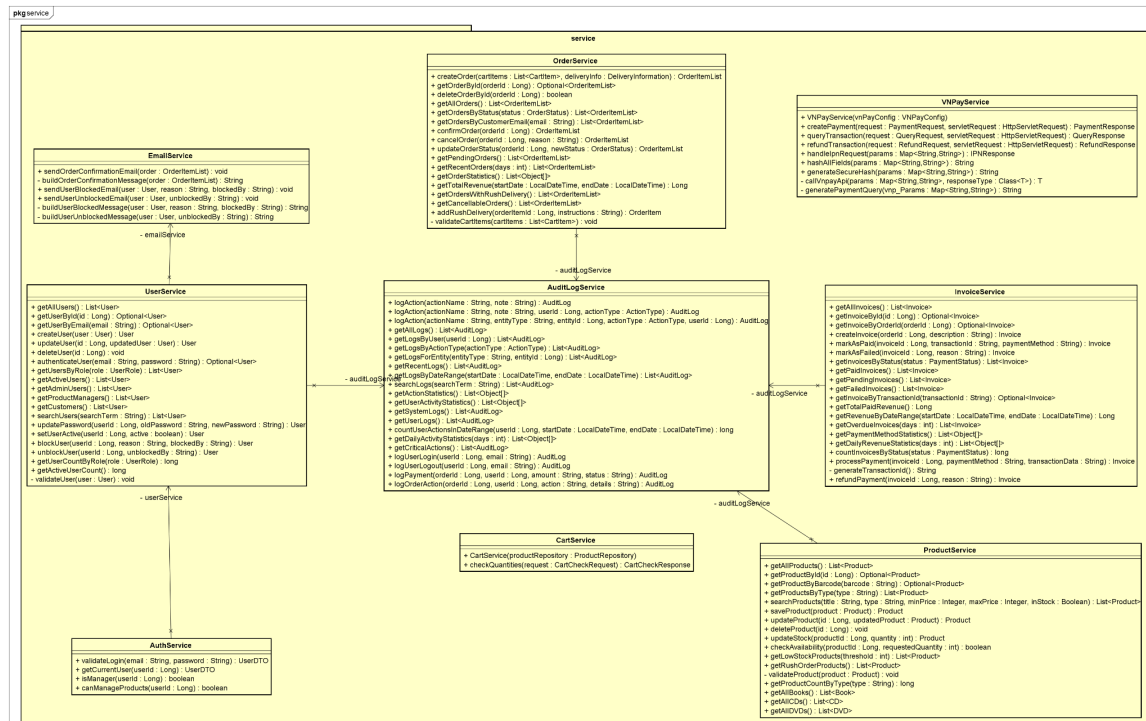
● Package Exception



● Package Repository



● Package Service



4.3.3 Class Design

4.3.3.1 Product

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	Long	Auto-generated	Product primary key ID (unique identifier)

2	title	String	-	Product title or name
3	category	String	-	Category to which the product belongs
4	price	Integer	-	Price per unit
5	productValue	Integer	-	Reference value to validate price (used for isPriceValid)
6	weight	Float	null	Weight of product
7	rushOrderSupported	Boolean	FALSE	Whether rush delivery is supported
8	dimensions	String	null	Dimensions of physical product (e.g. "20x15x3 cm")
9	condition	String	null	Product condition (New, Used – Like New, etc.)
10	imageUrl	String	null	URL to the product image
11	barcode	String	null	Unique product barcode
12	importDate	String	null	Import date
13	introduction	String (TEXT)	null	Product introduction or description
14	quantity	Integer	0	Available quantity in stock
15	type	String	Auto-managed	Discriminator column for subclass (book, cd, dvd, etc.)
16	createdAt	LocalDateTime	Auto-set	Timestamp when product is created
17	updatedAt	LocalDateTime	Auto-set	Timestamp when product is last updated

Table 2. Operation design

#	Name	Type	Description (purpose)
1	getProductType()	abstract String	Abstract method to return the product's type (must be implemented by subclass)
2	hasStock()	boolean hasStock(int quantity)	Check whether there is sufficient stock
3	reduceStock()	void reduceStock(int amount)	Subtracts amount from stock; throws error if not enough
4	addStock()	void addStock(int amount)	Adds amount to stock
5	calculateTotalPrice()	Integer calculateTotalPrice(int)	Returns total price for given quantity
6	isPriceValid()	boolean isPriceValid()	Validates price is between 30%-150% of product value

7	onCreate()	@PrePersist void onCreate()	Auto-set timestamps on creation
8	onUpdate()	@PreUpdate void onUpdate()	Auto-update updatedAt timestamp on any modification

a, Parameter:

checkProductAvailability(quantity: int): Boolean

- quantity: An integer representing the number of products to check if the available inventory is sufficient.
- Return: true if the available inventory is sufficient, otherwise false.

calculateTotalPrice(requestedQuantity: int): Integer

- requestedQuantity: The number of units to calculate the total price for.
- Return: The total price (unit price $\times$ requested quantity).

reduceStock(amount: int): void

- amount: The number of units to subtract from current inventory.
- Return: void.

addStock(amount: int): void

- amount: The number of units to add to the current inventory.
- Return: void

isPriceValid(): Boolean

- Return: true if the price is within 30%–150% of the productValue, otherwise false.

b, Exception:

reduceStock(amount: int): void

- IllegalStateException: If the current quantity is less than the amount to reduce.

c, Method:

How to use parameters/attributes:

checkProductAvailability(quantity: int): Boolean

- Purpose: This method checks whether the available inventory is sufficient for the requested quantity of a product.
- Usage: If the available stock is at least equal to the requested quantity, the method returns true; otherwise, it returns false.

calculateTotalPrice(requestedQuantity: int): Integer

- Purpose: This method calculates the total price of a given quantity of product.
- Usage: Multiply unit price by requested quantity to get total.

reduceStock(amount: int): void

- Purpose: This method reduces stock quantity by the specified amount.
- Usage: Subtract amount from inventory. Throws exception if not enough stock

addStock(amount: int): void

- Purpose: This method adds stock quantity by the specified amount.
- Usage: Increases current inventory by amount.

isPriceValid(): Boolean

- Purpose: This method checks if product price is within acceptable bounds.
- Usage: Compare price against 30%–150% of productValue.

d, State:

- If stock > requested quantity → Available.
- If stock < requested quantity → Not Available.
- If product_Id is invalid → ProductNotFoundException is raised.

4.3.3.2 Book

#	Name	Data type	Default value	Description
1	genre	String	null	Book genre (e.g., Fiction, Science, History...)
2	pageCount	Integer	null	Total number of pages in the book
3	publicationDate	String	null	Date when the book was published
4	authors	String	null	Comma-separated list of authors
5	publishers	String	null	Publisher of the book
6	coverType	String	null	Type of cover (e.g., hardcover, paperback)

Table 2. Operation design

#	Name	Type	Description (purpose)
1	getProductType()	String	Returns "book" to identify this subclass
2	getAuthorList()	String[]	Converts authors string to an array of author names
3	setAuthorList(String[])	void	Joins array of authors into string and assigns to authors
4	isHardcover()	boolean	Returns true if coverType equals "hardcover" (case-insensitive)
5	getEstimatedReadingTimeMinutes()	Integer	Estimates reading time based on pageCount (assumes 250 words/page, 200 wpm)

a. Parameter

getAuthorList(): String[]

- **Return:** An array of authors obtained by splitting the authors string using commas.

setAuthorList(authorList: String[]): void

- authorList: An array of author names to be stored in the authors string.
- **Return:** void

isHardcover(): boolean

- **Return:** true if the book has a hardcover; otherwise false.

getEstimatedReadingTimeMinutes(): Integer

- **Return:** An estimated reading time (in minutes), calculated using pageCount.
- Formula: $\text{pageCount} \times 250 \div 200$ (250 words per page, 200 words per minute)

b. Exception

The Book class does **not throw any explicit exceptions**.

All methods are guarded with null checks. There is **no exception raised** within this class.

c. Method

getAuthorList(): String[]

- **Purpose:** Converts the authors string into an array of author names.
- **Usage:** Used to retrieve and manipulate authors individually.

setAuthorList(String[] authorList): void

- **Purpose:** Stores an array of author names into the authors field as a comma-separated string.
- **Usage:** Converts array → string and sets it.

isHardcover(): boolean

- **Purpose:** Checks if the book cover type is hardcover.
- **Usage:** Compares coverType to "hardcover" case-insensitively.

getEstimatedReadingTimeMinutes(): Integer

- **Purpose:** Estimates how long it would take to read the book.
- **Usage:** Uses page count to compute time using average reading speed (250 words/page at 200 wpm).

4.3.3.3 CD

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	trackList	String	null	List of tracks in the CD, stored as a comma-separated string
2	genre	String	null	Music genre (e.g., Pop, Rock, Jazz)
3	recordLabel	String	null	Record label that published the CD
4	artists	String	null	List of performing artists, comma-separated
5	releaseDate	String	null	CD release date

Table 2. Operation design

#	Name	Type	Description (purpose)
1	getProductType()	String	Returns "cd" as the product type (overridden from Product)

2	getTrackArray()	String[]	Splits the trackList string into an array of tracks
3	setTrackArray()	void setTrackArray(String[])	Joins an array of tracks into a string and sets it to trackList
4	getArtistList()	String[]	Splits the artists string into an array of artist names
5	setArtistList()	void setArtistList(String[])	Joins an array of artist names into a string and sets it to artists
6	getTrackCount()	int	Returns the number of tracks in the trackList
7	isCompilation()	boolean	Returns true if the album has more than 3 artists (treated as a compilation)

a. Parameter

getTrackArray(): String[]

- Return: An array of tracks obtained by splitting the trackList string using commas.

setTrackArray(tracks: String[]): void

- tracks: An array of track names to be joined and stored in trackList.
- Return: void

getArtistList(): String[]

- Return: An array of artist names obtained by splitting the artists string using commas.

setArtistList(artistList: String[]): void

- artistList: An array of artist names to be joined and stored in artists.
- Return: void

getTrackCount(): int

- Return: Number of tracks (length of getTrackArray() result).

isCompilation(): boolean

- Return: true if the album contains more than 3 artists, otherwise false.

b. Exception

The CD class does not throw any custom exceptions explicitly.

c. Method

getTrackArray(): String[]

- Purpose: Convert trackList string into an array of individual tracks.
- Usage: Used to retrieve track list as a manipulable array.

setTrackArray(String[] tracks): void

- Purpose: Store an array of tracks as a single comma-separated string.
- Usage: Converts array → string and assigns it to trackList.

getArtistList(): String[]

- Purpose: Convert artists string into an array of individual artist names.
- Usage: Used to retrieve artists as an array.

setArtistList(String[] artistList): void

- Purpose: Store an array of artists as a single comma-separated string.
- Usage: Converts array → string and assigns it to artists.

getTrackCount(): int

- Purpose: Count number of tracks listed in the CD.
- Usage: Return the length of the array returned by getTrackArray().

isCompilation(): boolean

- Purpose: Determine if the CD is a compilation (i.e., multiple artists).
- Usage: Returns true if more than 3 artists are listed.

4.3.3.4 DVD

Table 1. Attribute design

#	Name	Data Type	Default Value	Description
1	releaseDate	String	null	DVD release date.
2	dvdType	String	null	Type of disc (e.g., DVD, Blu-ray, 4K Ultra HD, etc.).
3	genre	String	null	Genre of the movie (e.g., Action, Horror, Comedy).

4	studio	String	null	Studio or publisher of the DVD.
5	directors	String	null	List of directors, stored as a comma-separated string.
6	durationMinutes	Integer	null	Duration of the movie in minutes.
7	rating	String	null	Content rating (e.g., G, PG, PG-13, R).

Table 2. Operation design

#	Name	Type	Description (Purpose)
1	getProductType()	String	Returns "dvd" as the product type (overridden from Product).
2	getDirectorList()	String[]	Splits the directors string into an array of director names.
3	setDirectorList()	void	Joins an array of directors into a comma-separated string and stores it.
4	isBluRay()	boolean	Returns true if the DVD is a Blu-ray disc.
5	is4K()	boolean	Returns true if the DVD is a 4K Ultra HD disc.
6	getFormattedDuration()	String	Converts durationMinutes into hours and minutes format (e.g., "1h 45m").
7	isChildFriendly()	boolean	Returns true if the DVD is rated G or PG.
8	isFeatureLength()	boolean	Returns true if the duration is 75 minutes or more.

a. Parameter & Return

- **getDirectorList(): String[]**
 - **Return:** An array of director names split from the directors string using a comma delimiter.

- **setDirectorList(directorList: String[]): void**
 - **Parameter:** directorList — an array of director names.
 - **Return:** None. Stores the array as a single comma-separated string into the directors field.
- **isBluRay(): boolean**
 - **Return:** true if dvdType contains "blu" (case-insensitive), otherwise false.
- **is4K(): boolean**
 - **Return:** true if dvdType contains "4k" (case-insensitive), otherwise false.
- **getFormattedDuration(): String**
 - **Return:** A formatted duration string like "2h 5m". Returns "Unknown" if the value is null or invalid.
- **isChildFriendly(): boolean**
 - **Return:** true if the rating is "G" or "PG", otherwise false.
- **isFeatureLength(): boolean**
 - **Return:** true if durationMinutes is 75 or more, otherwise false.

b. Exception

The DVD class does **not throw any custom exceptions** explicitly. All methods assume well-formed input and handle null values gracefully.

c. Method Details

- **getProductType(): String**
 - Purpose:** Returns the specific type of the product.
 - Usage:** This method overrides the getProductType() method from the superclass Product, and consistently returns "dvd" to indicate that this object represents a DVD product.
- **getDirectorList(): String[]**
 - Purpose:** Converts the directors string field into an array of individual director names.
 - Usage:** This method allows access to the list of directors in an array format, making it easier to process or display them separately. It uses a comma (,) and optional whitespace as the delimiter.
- **setDirectorList(String[] directorList): void**
 - Purpose:** Takes an array of director names and stores them as a single comma-separated string.
 - Usage:** This method is used to serialize an array of director names into a string format suitable for database storage in the directors field.
- **isBluRay(): boolean**
 - Purpose:** Checks if the DVD is a Blu-ray disc.

Usage: Returns true if the dvdType field contains the substring "blu" (case-insensitive). Useful for identifying the disc format.

- **is4K(): boolean**

Purpose: Determines whether the DVD is a 4K Ultra HD disc.

Usage: Returns true if the dvdType field contains "4k" (case-insensitive). Helps distinguish high-resolution formats.

- **getFormattedDuration(): String**

Purpose: Converts the integer duration in minutes into a human-readable string format (hours and minutes).

Usage: If the duration is valid, returns a string like "2h 15m"; otherwise, returns "Unknown". This improves display readability for end-users.

- **isChildFriendly(): boolean**

Purpose: Determines if the DVD is suitable for children.

Usage: Returns true if the rating field is "G" or "PG". Useful for filtering or recommending family-friendly content.

- **isFeatureLength(): boolean**

Purpose: Checks if the DVD qualifies as a feature-length film.

Usage: Returns true if durationMinutes is 75 or more. Commonly used to classify full-length movies versus short films.

4.3.3.5 LP

Table 1. Attribute design

#	Name	Data Type	Default Value	Description
1	artist	String	null	Name of the performing artist or band.
2	recordLabel	String	null	Name of the record label that published the LP.
3	musicType	String	null	Genre or category of the music (e.g., Jazz, Rock, Classical).
4	releaseDate	String	null	Release date of the LP.
5	tracklist	String	null	List of tracks stored as a comma-separated string.
6	rpm	Integer	null	Speed of the record in revolutions per minute (e.g., 33, 45, 78).
7	sizeInches	Double	null	Physical diameter of the vinyl in inches (e.g., 7", 10", 12").
8	vinylCondition	String	null	Condition of the vinyl record (e.g., Mint, Very Good Plus, etc.).

9	sleeveCondition	String	null	Condition of the LP sleeve or cover.
---	-----------------	--------	------	--------------------------------------

Table 2. Operation design

#	Name	Type	Description (Purpose)
1	getProductType()	String	Returns "lp" to indicate this is an LP product (overridden from Product).

a. Parameter & Return

- **getProductType(): String**
 - **Return:** Returns the string "lp" as the product type.

b. Exception

The LP class does **not throw any custom exceptions** explicitly. All methods assume valid or null-safe input and handle them accordingly.

c. Method Details

- **getProductType(): String**
Purpose: Identifies the specific product type.
Usage: This method overrides the superclass Product's method to return the string "lp", which designates that the current product is a vinyl LP (Long Play record). This value can be used in type filtering, product categorization, or serialization logic.

4.3.3.6 Invoice

Table 1. Attribute design

#	Name	Data Type	Default Value	Description
1	invoiceId	Long	Auto-generated	Unique identifier for each invoice (Primary Key).
2	order	OrderItemList	-	Associated order that this invoice belongs to (One-to-One relation).
3	transactionId	String	null	Identifier returned by the payment gateway after successful transaction.
4	description	String (TEXT)	null	Detailed description of the invoice

				content.
5	createdAt	LocalDateTime	Auto-set (@PrePersist)	Timestamp when invoice is created.
6	paymentStatus	PaymentStatus	PENDING	Current payment status (PENDING, PAID, etc.).
7	paymentMethod	String	null	Payment method used (e.g., credit card, VNPay).
8	paidAt	LocalDateTime	null	Timestamp when the payment was completed.

Table 2. Operation design

#	Name	Type	Description
1	createInvoice()	void createInvoice(OrderItemList, String)	Initializes invoice with associated order and sets payment to PENDING.
2	markAsPaid()	void markAsPaid(String, String)	Updates invoice status to PAID, stores transaction ID and payment method.
3	markAsFailed()	void markAsFailed()	Marks invoice as FAILED.
4	isPaid()	boolean isPaid()	Returns true if status is PAID.
5	isPending()	boolean isPending()	Returns true if status is PENDING.
6	getTotalAmount()	Integer getTotalAmount()	Returns the total amount of the associated order (after VAT).
7	generateDescription()	String generateDescription()	Returns a textual summary of the invoice with order info.
8	onCreate()	@PrePersist void onCreate()	Auto-set createdAt timestamp before persisting.

a. Parameter

createInvoice(order: OrderItemList, description: String)

- Initializes invoice with order and description. Sets payment status to PENDING.

markAsPaid(transactionId: String, paymentMethod: String)

- Updates payment status to PAID, sets paidAt to now.

markAsFailed()

- Marks the payment as FAILED.

isPaid()

- Returns true if paymentStatus == PAID.

isPending()

- Returns true if paymentStatus == PENDING.

getTotalAmount()

- Returns order.getTotalAfterVat() if order is not null; else returns 0.

generateDescription()

- Returns description in format: "Invoice for order #<id> containing <n> item(s)".

b. Exception

No explicit exceptions are thrown in this class. However, getTotalAmount() and generateDescription() gracefully handle null order references.

c. Method

createInvoice()

- Purpose: Initializes the invoice entity.
- Usage: Called when a new invoice is to be created for a given order.

markAsPaid()

- Purpose: Marks an invoice as successfully paid.
- Usage: After payment confirmation, store transaction info and timestamp.

markAsFailed()

- Purpose: Records failed payment attempt.
- Usage: Update paymentStatus to FAILED.

isPaid()

- Purpose: Check if invoice is paid.
- Usage: Used to verify completion of transaction.

isPending()

- Purpose: Check if payment is still pending.
- Usage: Used to block duplicate or premature updates.

getTotalAmount()

- Purpose: Retrieve total invoice cost from associated order.
- Usage: Display invoice total; safe fallback if no order is attached.

generateDescription()

- Purpose: Summarize invoice with order ID and item count.
- Usage: For UI or logs.

d. State

- If `paymentStatus == PENDING` → Pending payment
- If `paymentStatus == PAID` → Payment completed
- If `paymentStatus == FAILED` → Payment failed
- If `paymentStatus == REFUNDED` → Payment refunded
- If `paymentStatus == CANCELLED` → Payment cancelled
- If `order == null` → No associated order

4.3.3.7 OrderItem

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	Long	Auto-generated	Unique ID for the order line
2	order	OrderItemList	-	Parent order to which this line belongs (many-to-one)
3	product	Product	-	Product associated with this line item
4	status	OrderItemStatus (enum)	PENDING	Status of the item in the order flow
5	rushOrder	Boolean	FALSE	Whether rush delivery is applied for this item
6	quantity	Integer	Required	Number of product units in this line
7	totalFee	Integer	Required	Total price for this line item (unit price × quantity)
8	deliveryTime	String	null	Time for rush delivery, if applicable
9	instructions	String (TEXT)	null	Special delivery instructions or notes

Table 2. Operation design

#	Name	Type	Description (purpose)
1	createOrderItem()	void	Initialize order line with product and quantity
2	isRushOrder()	boolean	Check if rush delivery is applied
3	getUnitPrice()	Integer	Calculate and return unit price
4	updateTotalFee()	void	Recalculate totalFee based on product's price and current quantity
5	canBeCancelled()	boolean	Determine whether the order item can be cancelled
6	cancel()	void	Cancel this order item if it is eligible

a. Parameter

createOrderItem(product: Product, quantity: int): void

- Initializes this item with product and quantity
- Return: void

isRushOrder(): boolean

- Return: true if rush delivery is enabled

getUnitPrice(): Integer

- Return: totalFee / quantity, or 0 if quantity == 0

updateTotalFee(): void

- Return: void — sets totalFee = product.calculateTotalPrice(quantity)

canBeCancelled(): boolean

- Return: true if current status is PENDING or CONFIRMED

cancel(): void

- If canBeCancelled() is false → throws IllegalStateException
- Otherwise sets status = CANCELLED

b. Exception

cancel(): Throws IllegalStateException if status is not PENDING or CONFIRMED

c. Method

createOrderItem(product, quantity)

- Purpose: Initialize order line with required data and compute totalFee
- Usage: Called when creating order from cart items

isRushOrder()

- Purpose: Determine rush delivery eligibility
- Usage: Used in delivery fee calculation and UI display

getUnitPrice()

- Purpose: Derive per-unit cost from total
- Usage: Display per-product price

updateTotalFee()

- Purpose: Recalculate based on updated price or quantity
- Usage: Price change/update handling

cancel()

- Purpose: Cancel order line item
- Usage: Refund or change-of-mind scenario

d.State

- If status == PENDING or CONFIRMED → Can be cancelled
- If status == SHIPPED or DELIVERED or PROCESSING → Cannot be cancelled
- If rushOrder == true → Rush delivery applies
- If quantity == 0 or null → Unit price = 0
- If cancel() called while status == CANCELLED or not cancellable → Exception: Not allowed

4.3.3.8 OrderItemList

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	Long	Auto-generated	Unique identifier of the order (primary key)
2	totalBeforeVat	Integer	0	Total fee of all order items before VAT
3	totalAfterVat	Integer	0	Total fee after applying VAT

4	status	OrderStatus (enum)	PENDING	Status of the order
5	deliveryInfo	DeliveryInformation	null	Associated delivery information
6	vatPercentage	Integer	10	VAT rate (%) used to calculate totalAfterVat
7	createdAt	LocalDateTime	Auto-set	Timestamp when order was created
8	updatedAt	LocalDateTime	Auto-set	Timestamp when order was last updated
9	orderItems	List<OrderItem>	Empty list	List of line items in the order
10	invoice	Invoice	null	Invoice generated for this order

Table 2. Operation design

#	Name	Type	Description (purpose)
1	addOrderItem()	void	Add a line item to the order
2	removeOrderItem()	void	Remove a line item from the order
3	recalculateTotals()	void	Recalculate totals based on order items and VAT
4	canBeCancelled()	boolean	Check if order is in cancellable state
5	cancel()	void	Set order status to CANCELLED if allowed
6	confirm()	void	Change order status to CONFIRMED from PENDING
7	getTotalItemCount()	int	Return total number of product units across all line items
8	hasRushItems()	boolean	Check if any item in the order is flagged for rush delivery
9	onCreate()	@PrePersist	Set creation and update timestamps when order is first saved
10	onUpdate()	@PreUpdate	Update timestamp when order is modified

a. Parameter

addOrderItem(orderItem: OrderItem): void

- Adds item to orderItems, updates back-reference, and recalculates totals
- Return: void

removeOrderItem(orderItem: OrderItem): void

- Removes item from orderItems, clears reference, updates totals
- Return: void

recalculateTotals(): void

- Recomputes totalBeforeVat and totalAfterVat based on item prices and VAT
- Return: void

canBeCancelled(): boolean

- Return: true if status is PENDING or CONFIRMED, else false

cancel(): void

- Change status to CANCELLED if cancellable; otherwise, throws exception
- Return: void

confirm(): void

- Transition order from PENDING → CONFIRMED
- Return: void

getTotalItemCount(): int

- Return: Sum of quantities in orderItems

hasRushItems(): boolean

- Return: true if any OrderItem has rushOrder = true

b. Exception

- cancel() :Throws IllegalStateException if status is not PENDING or CONFIRMED
- confirm(): Throws IllegalStateException if status is not PENDING

c. Method

addOrderItem()/removeOrderItem()

- Purpose: Modify list of order lines dynamically
- Usage: Cart management, order updates

recalculateTotals()

- Purpose: Ensure totalBeforeVat and totalAfterVat reflect current state
- Usage: Called after order item change

cancel()

- Purpose: Invalidate an order
- Usage: Customer cancels or admin rejects

confirm()

- Purpose: Lock-in order for fulfillment
- Usage: After validation or payment

hasRushItems()

- Purpose: Determine if order includes fast shipping items
- Usage: Adjust delivery logic and fees

d.State

- If status == PENDING or CONFIRMED → Can be cancelled
- If status == CANCELLED → Order cancelled
- If status == PENDING → Can be confirmed
- If status != PENDING → confirm() → Throws exception
- If any orderItem.rushOrder == true → Order contains rush delivery items
- If order just created → createdAt, updatedAt auto-set via @PrePersist
- If order updated → updatedAt refreshed via @PreUpdate

4.3.3.9 DeliveryInformation

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	deliveryId	Long	Auto-generated	Unique identifier for delivery info (primary key)
2	name	String	-	Full name of the recipient
3	phone	String	-	Recipient's phone number
4	email	String	-	Recipient's email address
5	address	String (TEXT)	-	Street address (excluding ward and province)
6	ward	String	-	Ward or district of delivery address
7	province	String	-	Province or city of delivery address
8	deliveryMessage	String (TEXT)	null	Optional delivery note/instruction

9	deliveryFee	Integer	0	Fee for delivery (based on rules/weight)
---	-------------	---------	---	--

Table 2. Operation design

#	Name	Type	Description (purpose)
1	createDeliveryInfo(...)	void	Initialize all required delivery fields
2	getFullAddress()	String	Concatenate address, ward, and province into a full formatted string
3	isComplete()	boolean	Check if all required fields are present and non-empty
4	isValidPhone()	boolean	Validate phone number format (basic)
5	isValidEmail()	boolean	Validate email address format (basic)

a. Parameter

createDeliveryInfo(...)

- name: Full name of the recipient
- phone: Contact number
- email: Email address
- address: Street/house details
- ward: Ward or district
- province: Province or city
- deliveryMessage: Optional note for courier
- Return: void

getFullAddress(): String

- Return: Formatted string like "123 A St, Ward 5, HCM"

isComplete(): boolean

- Return: true if all required fields are filled, otherwise false

isValidPhone(): boolean

- Return: true if phone is 10–11 digits and valid format

isValidEmail(): boolean

- Return: true if matches standard email regex

b. Exception

- createOrder(): IllegalArgumentException if cart is empty, null, invalid product, or insufficient stock
- confirmOrder(), cancelOrder(), updateOrderStatus() : RuntimeException if order not found
- addRushDelivery() RuntimeException if order item not found. IllegalStateException if product doesn't support rush delivery

c. Method

createOrder()

- Purpose: Process cart into saved order + invoice
- Usage: Called when user submits a cart and delivery info

confirmOrder()

- Purpose: Change order state from PENDING to CONFIRMED
- Usage: Admin or automated trigger

cancelOrder()

- Purpose: Set order status to CANCELLED, restore stock
- Usage: Triggered by customer or timeout

addRushDelivery()

- Purpose: Flag rush delivery for 2-hour dispatch
- Usage: Called when customer opts-in per item basis

getRecentOrders()

- Purpose: Get orders placed in the last X days
- Usage: Dashboard, metrics, recent activities

getTotalRevenue()

- Purpose: Get money value from orders (not invoices)
- Usage: Reporting

d.State

- If cartItems == null || empty → Cart is invalid → Exception
- If any product is null in cart → Invalid cart item
- If requestedQuantity > product.stock → Stock insufficient → Exception
- If order.status == PENDING → Awaiting confirmation
- If order.status == CONFIRMED → Confirmed and ready
- If order.status == CANCELLED → Cancelled and logged

- If `product.rushOrderSupported == false` in `addRushDelivery()` → Rush delivery not available
- If `orderId` not found → `RuntimeException` (invalid line)

4.3.3.10 AuditLog

Table 1. Attribute design

#	Name	Data Type	Default Value	Description
1	auditLogId	Long	Auto-generated	Unique identifier for the audit log entry.
2	actionName	String	null	Name of the action performed (e.g., "User updated order").
3	recordedAt	LocalDateTime	Now (on insert)	Timestamp when the action occurred.
4	note	String (TEXT)	null	Optional description or comment about the action.
5	userId	Long	null	ID of the user who performed the action.
6	entityType	String	null	Type of entity affected (e.g., "Product", "Order").
7	entityId	Long	null	ID of the entity affected.
8	actionType	Enum (ActionType)	null	Type of action performed (e.g., CREATE, DELETE, LOGIN).

Table 2. Operation design

#	Name	Type	Description (Purpose)
1	<code>create(String, String)</code>	AuditLog	Creates a basic audit log with an action name and note.
2	<code>create(String, String, Long, ActionType)</code>	AuditLog	Creates an audit log with user context and action type.

3	<code>createForEntity(String, String, Long, ActionType, Long)</code>	<code>AuditLog</code>	Creates an audit log for actions on specific entities.
4	<code>getFormattedTimestamp()</code>	<code>String</code>	Returns the recordedAt timestamp as a formatted string.
5	<code>onCreate()</code>	<code>void</code>	JPA lifecycle callback to auto-set recordedAt if not provided.

a. Parameter & Return

- **create(String actionName, String note): AuditLog**
 - **Parameters:** actionName – the action performed, note – optional description.
 - **Return:** A new AuditLog instance with recordedAt set to now.
- **create(String actionName, String note, Long userId, ActionType actionType): AuditLog**
 - **Parameters:**
 - actionName: Name of the action
 - note: Additional note
 - userId: ID of the user
 - actionType: Type of the action (from enum)
 - **Return:** A new AuditLog instance with user and action type context.
- **createForEntity(String actionName, String entityType, Long entityId, ActionType actionType, Long userId): AuditLog**
 - **Parameters:**
 - actionName: Name of the action
 - entityType: Name of the entity (e.g., Product, Order)
 - entityId: ID of the entity
 - actionType: Enum value representing the type of action
 - userId: ID of the user performing the action

- **Return:** A complete AuditLog instance for entity-specific operations.
- **getFormattedTimestamp(): String**
 - **Return:** A string representation of recordedAt, or "Unknown" if null.
- **onCreate(): void**
 - **Return:** None.
 - **Effect:** Automatically sets recordedAt to current time if not already set, before persisting.

b. Exception

The AuditLog class does **not throw any custom exceptions**. All static methods safely initialize the object, and onCreate() ensures recordedAt is always set.

c. Method Details

create(String actionName, String note): AuditLog

Purpose: Creates a simple audit log entry with just an action name and a descriptive note.

Usage: Used for generic logging purposes where user or entity context is not required. Automatically sets recordedAt to the current timestamp.

- **create(String actionName, String note, Long userId, ActionType actionType): AuditLog**

Purpose: Creates an audit log entry with additional user and action type context.

Usage: This version is used when the identity of the acting user and the nature of the action (e.g., LOGIN, DELETE) needs to be tracked.
- **createForEntity(String actionName, String entityType, Long entityId, ActionType actionType, Long userId): AuditLog**

Purpose: Creates a detailed audit log that references a specific entity (e.g., product, order) and includes the user and action type.

Usage: Recommended for entity-related actions (e.g., "DELETE Product with ID: 7"). Auto-generates a descriptive note from the input.
- **getFormattedTimestamp(): String**

Purpose: Retrieves a string-formatted timestamp of when the action was recorded.

Usage: Used for displaying the time of the log entry. Returns "Unknown" if recordedAt is not set.
- **onCreate(): void**

Purpose: Ensures recordedAt is initialized before the entity is persisted.

Usage: Invoked automatically by JPA when a new entity is saved. If recordedAt is null, it sets it to LocalDateTime.now().

4.3.3.11 DailyOperationTracker

Table 1. Attribute design

#	Name	Data Type	Default Value	Description
1	operationId	Long	Auto-generated	Unique identifier for the operation tracking record.
2	userId	Long	null (required)	ID of the user whose operations are being tracked.
3	operationDate	LocalDate	null (required)	The specific date of the operation being recorded.
4	operationCount	Integer	0	Number of operations performed by the user on that date.
5	isEditing	Boolean	false	Indicates whether the user is currently in an editing session.
6	lastEditStarted	LocalDateTime	null	Timestamp of when the user last began editing.
7	createdAt	LocalDateTime	Auto-set	Timestamp when this record was created.
8	updatedAt	LocalDateTime	Auto-set	Timestamp when this record was last updated.

Table 2. Operation design

#	Name	Type	Description (Purpose)
1	onCreate()	void	JPA lifecycle callback that sets createdAt and updatedAt when inserted.
2	onUpdate()	void	JPA lifecycle callback that updates updatedAt before an update is persisted.

a. Parameter & Return

- **onCreate(): void**
 - **Return:** None.

- **Effect:** Automatically sets both createdAt and updatedAt to the current timestamp before the entity is persisted for the first time.
- **onUpdate(): void**
 - **Return:** None.
 - **Effect:** Automatically sets updatedAt to the current timestamp right before the entity is updated.

b. Exception

The DailyOperationTracker class does **not throw any custom exceptions**. All fields are validated via JPA annotations (nullable = false) and are assumed to be used within the context of a managed persistence layer.

c. Method Details

- **onCreate(): void**

Purpose: Initializes audit-related timestamps when a new DailyOperationTracker entity is created.

Usage: This method is automatically triggered by the JPA provider prior to inserting the entity into the database. It assigns the current timestamp to both createdAt and updatedAt to track when the record was created.
- **onUpdate(): void**

Purpose: Updates the updatedAt field whenever the entity is modified.

Usage: Invoked automatically by JPA before updating the record in the database. Ensures updatedAt reflects the latest modification time.

4.3.3.12 User

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	userId	Long	Auto-generated	Primary key ID of the user
2	password	String	null	User password (should be stored encrypted)
3	name	String	null	Full name of the user
4	email	String	null	Unique email address
5	phone	String	null	Phone number
6	role	UserRole (Enum)	CUSTOMER	User role: ADMIN, CUSTOMER, MANAGER, or EMPLOYEE
7	province	String	null	Province part of the address
8	district	String	null	District part of the address

9	ward	String	null	Ward part of the address
10	address	String	null	Full address (street etc.)
11	registrationDate	LocalDateTime	Now if null	Timestamp when user registers
12	salary	Double	0	Monthly salary (used for EMPLOYEE role)
13	isActive	Boolean	TRUE	Whether the account is active

Table 2. Operation design

#	Name	Type	Description (purpose)
1	onCreate()	void	Sets registrationDate to now if it is null during persistence
2	isAdmin()	boolean	Returns true if the user's role is ADMIN
3	canManageProducts()	boolean	Returns true if the role is ADMIN or MANAGER
4	getDisplayName()	String	Returns name if present, otherwise falls back to email

a. Parameter

isAdmin(): boolean

- Return: true if the user's role is ADMIN, otherwise false.

canManageProducts(): boolean

- Return: true if the user is an ADMIN or MANAGER, otherwise false.

getDisplayName(): String

- Return: User's name if available, otherwise returns their email.

b. Exception

No explicit exceptions are thrown in the class. All logic is conditional and guarded.

c. Method

isAdmin(): boolean

- Purpose: Checks whether the user has administrator privileges.
- Usage: Returns true only if role == ADMIN.

canManageProducts(): boolean

- Purpose: Checks if the user is allowed to manage products.
- Usage: Returns true if the role is ADMIN or MANAGER.

getDisplayName(): String

- Purpose: Provides a fallback mechanism for displaying a user-friendly name.
- Usage: Uses name if available, else email.

4.3.3.13 AuthController

Table 1. Attribute design

#	Name	Data Type	Description
1	authService	AuthService	Service component used for user authentication and data access.
2	userMapper	UserMapper	Mapper component used to convert between UserDTO and UserProfileDTO.

Table 2. Operation design

#	Name	Type	Description (Purpose)
1	login()	ResponseEntity<ApiResponse<UserProfileDTO>>	Authenticates a user and stores session data.
2	logout()	ResponseEntity<ApiResponse<Void>>	Invalidates the user session and logs the user out.
3	getCurrentUser()	ResponseEntity<ApiResponse<UserDTO>>	Retrieves the currently authenticated user's data from session.

a. Parameter & Return

- **login(Map<String, String> loginRequest, HttpSession session): ResponseEntity<ApiResponse<UserProfileDTO>>**
 - **Parameters:**
 - loginRequest: Contains email and password fields.
 - session: HTTP session object used to store authenticated user data.
 - **Return:** A response entity containing the authenticated user's profile data or an error message.
- **logout(HttpSession session): ResponseEntity<ApiResponse<Void>>**
 - **Parameters:**
 - session: HTTP session object to be invalidated.
 - **Return:** A success message if logout is successful, or an error message if it fails.
- **getCurrentUser(HttpSession session): ResponseEntity<ApiResponse<UserDTO>>**
 - **Parameters:**
 - session: HTTP session object containing the authenticated user's ID.
 - **Return:** The UserDTO of the currently logged-in user, or an unauthorized/error response.

b. Exception

This controller **handles exceptions internally** using try-catch blocks. On exceptions, it returns 500 Internal Server Error responses using the `ApiResponse.internalError()` method. Specific validation errors (like missing credentials) return 400 Bad Request.

c. Method Details

- **login(Map<String, String> loginRequest, HttpSession session)**

Purpose: Authenticates a user using the provided email and password.

Usage: This method validates the user's credentials using the `AuthService`. If valid, it sets user-related attributes (`userId`, `userEmail`, `userRole`) into the session and returns the `UserProfileDTO`. If invalid, it returns an unauthorized response. Handles missing fields and internal errors gracefully.
- **logout(HttpSession session)**

Purpose: Logs out the currently authenticated user by invalidating the session.

Usage: Clears the user's session data and returns a success response. Ensures that the session is invalidated to prevent reuse.
- **getCurrentUser(HttpSession session)**

Purpose: Retrieves the authenticated user's data based on the session's `userId`.

Usage: If a `userId` is present in the session, it fetches the corresponding `UserDTO` via `AuthService`. If no user is found or not logged in, it returns a 401 Unauthorized response.

4.3.3.14 InvoiceController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	<code>invoiceService</code>	<code>InvoiceService</code>	Injected	Service layer for invoice business logic
2	<code>orderMapper</code>	<code>OrderMapper</code>	Injected	Mapper to convert between Invoice and InvoiceDTO

Table 2. Operation design

#	Name	Type	Description (purpose)
1	<code>getAllInvoices()</code>	<code>ResponseEntity<ApiResponse<List<InvoiceDTO>>></code>	Retrieve all invoices
2	<code>getInvoiceById(id)</code>	<code>ResponseEntity<ApiResponse<InvoiceDTO>></code>	Retrieve an invoice by its ID
3	<code>getInvoiceByTransactionId()</code>	<code>ResponseEntity<ApiResponse<InvoiceDTO>></code>	Get invoice by VNPay transaction ID
4	<code>getInvoiceByOrderId()</code>	<code>ResponseEntity<ApiResponse<InvoiceDTO>></code>	Get invoice by order ID
5	<code>createInvoice(request)</code>	<code>ResponseEntity<ApiResponse<InvoiceDTO>></code>	Create a new invoice for an order
6	<code>updateInvoice(id, request)</code>	<code>ResponseEntity<ApiResponse<InvoiceDTO>></code>	Update an invoice (not fully implemented)
7	<code>processPayment(id, request)</code>	<code>ResponseEntity<ApiResponse<InvoiceDTO>></code>	Mark an invoice as paid
8	<code>getInvoicesByPaymentStatus()</code>	<code>ResponseEntity<ApiResponse<List<InvoiceDTO>>></code>	Filter invoices by PaymentStatus
9	<code>getPendingInvoices()</code>	<code>ResponseEntity<ApiResponse<List<InvoiceDTO>>></code>	Get all invoices with status PENDING
10	<code>getPaidInvoices()</code>	<code>ResponseEntity<ApiResponse<List<InvoiceDTO>>></code>	Get all invoices with status PAID
11	<code>getOverdueInvoices()</code>	<code>ResponseEntity<ApiResponse<List<InvoiceDTO>>></code>	Get invoices that are overdue for payment (over 30 days)
12	<code>getInvoicesByPaymentM</code>	<code>ResponseEntity<ApiResponse</code>	Get invoices filtered by payment

	ethod()	<List<InvoiceDTO>>>	method (not implemented)
13	getInvoicesForDateRange() ()	ResponseEntity<ApiResponse <List<InvoiceDTO>>>	Get invoices created within a specific date range (not implemented)
14	getTotalRevenue()	ResponseEntity<ApiResponse <Long>>	Calculate total revenue over a date range
15	getPaymentStatistics()	ResponseEntity<ApiResponse <List<Object[]>>>	Get aggregated statistics for payment methods
16	refundInvoice(id)	ResponseEntity<ApiResponse <InvoiceDTO>>	Mark an invoice as refunded with optional reason
17	getInvoiceCount()	ResponseEntity<ApiResponse <Long>>	Count the total number of invoices

a. Parameter

getInvoiceById(id: Long):

- id: Positive long – Invoice ID to retrieve
- Return: InvoiceDTO wrapped in ApiResponse

createInvoice(request: CreateInvoiceRequest):

- request.orderId: ID of order to create invoice for
- request.description: Description of invoice
- Return: Created invoice in DTO format

processPayment(id: Long, request: ProcessPaymentRequest):

- id: Invoice ID to update
- request.transactionId: VNPay transaction ID

request.paymentMethod: Payment method used

- Return: Updated invoice DTO

getInvoicesForDateRange(startDate, endDate: LocalDateTime):

- Start and end datetime range
- Return: List of invoices in date range (currently returns empty)

getTotalRevenue(startDate, endDate: LocalDateTime):

- Start and end datetime
- Return: Total revenue in VND for given period

b. Exception

- `getInvoiceById(id: Long)` : Returns 404 NOT FOUND if invoice not found
- `createInvoice()` / `processPayment()` : May throw exceptions from InvoiceService (e.g., invoice not found)
- `getInvoicesByPaymentStatus(status: String)` : Returns 400 BAD REQUEST if status is not valid enum
- `updateInvoice(id)`: Throws `RuntimeException` if invoice not found (temporary implementation)

c. Method

`getAllInvoices()`

- Purpose: Retrieve all invoices
- Usage: Admin/statistics view

`createInvoice()`

- Purpose: Create a new invoice for a given order
- Usage: Called when placing a new order

`processPayment()`

- Purpose: Mark invoice as paid with VNPay transaction info
- Usage: Used after payment confirmation

`getInvoicesByPaymentStatus()`

- Purpose: Filter invoices based on payment status
- Usage: Report grouping (PAID, FAILED, etc.)

`getOverdueInvoices()`

- Purpose: Identify invoices overdue for >30 days
- Usage: Monitoring unpaid invoices

`getPaymentStatistics()`

- Purpose: Aggregate statistics on payment methods
- Usage: Display insights on user payment behavior

d. State

- If invoice not found by id → HTTP 404 Not Found
- If transactionId not found → HTTP 404 Not Found
- If status not valid enum → HTTP 400 Bad Request
- If `invoiceService.markAsPaid()` succeeds → Status changed to PAID
- If `invoiceService.refundPayment()` succeeds → Status changed to REFUNDED
- If `getOverdueInvoices()` returns items → Invoice created > 30 days ago and unpaid
- If `getAllInvoices().size() == 0` → No invoices in system

4.3.3.15 OrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderService	OrderService	Injected	Service that handles business logic related to order operations
2	productService	ProductService	Injected	Service used to fetch product details when building cart items
3	orderMapper	OrderMapper	Injected	Mapper for converting between OrderItemList entities and DTOs

Table 2. Operation design

#	Name	Type	Description (purpose)
1	createOrder()	ResponseEntity	Create a new order with cart items and delivery info
2	getAllOrders()	ResponseEntity	Fetch all orders
3	getOrderById()	ResponseEntity	Fetch a specific order by ID
4	getOrdersByStatus()	ResponseEntity	Fetch orders filtered by status
5	getOrdersByCustomerEmail()	ResponseEntity	Fetch orders made by a specific customer via email
6	confirmOrder()	ResponseEntity	Mark an order as confirmed
7	cancelOrder()	ResponseEntity	Cancel an order with an optional reason
8	updateOrderStatus()	ResponseEntity	Manually update the status of an order
9	getPendingOrders()	ResponseEntity	Retrieve orders currently marked as PENDING
10	getRecentOrders()	ResponseEntity	Retrieve orders created within the past N days
11	getOrderStatistics()	ResponseEntity	Retrieve statistics grouped by status/type/customer
12	getTotalRevenue()	ResponseEntity	Calculate total revenue from orders in a given time range
13	getOrdersWithRushDelivery()	ResponseEntity	Fetch orders that have at least one rush delivery line
14	getCancellableOrders()	ResponseEntity	Get orders that are eligible for cancellation
15	addRushDelivery()	ResponseEntity	Add rush delivery option to a specific order line

a. Parameter

createOrder(request: CreateOrderRequest):

- request.cartItems: List of items (productId, quantity)
- request.deliveryInfo: Recipient info and address
- Return: DTO of newly created order

getOrderById(id: Long):

- id: Order ID to retrieve
- Return: Corresponding order if found

cancelOrder(id: Long, reason: String):

- id: ID of order to cancel
- reason: Optional cancellation reason
- Return: Cancelled order DTO

getRecentOrders(days: int):

- days: Time range for “recent” orders (default = 30)
- Return: Orders created in last days days

getTotalRevenue(startDate, endDate: LocalDateTime):

- Time window for calculating total revenue
- Return: Revenue in VND

addRushDelivery(orderLineId: Long, instructions: String):

- orderLineId: The ID of the order line to mark as rush delivery
- instructions: Optional delivery note
- Return: Updated order line DTO

b. Exception

- getOrderById(id: Long) :Returns 404 if not found
- getOrdersByStatus(status: String): Returns 400 if status is not a valid enum
- createOrder(): Throws IllegalArgumentException if product ID in cart is invalid
- updateOrderStatus() : Returns 400 if status is invalid enum

c. Method

createOrder()

- Purpose: Place new order from cart items
- Usage: Converts productId + quantity → OrderService.CartItem, maps delivery info

confirmOrder()

- Purpose: Marks the order as confirmed by system or manager
- Usage: Transition from PENDING → CONFIRMED

cancelOrder()

- Purpose: Cancel order due to customer or system reason
- Usage: Set status = CANCELLED, attach reason

addRushDelivery()

- Purpose: Add 2-hour rush delivery to a line item
- Usage: Add fee, flag eligibility, and set delivery time window

getTotalRevenue()

- Purpose: Sum of all paid orders in a date range
- Usage: Useful for finance/statistics

getOrdersByCustomerEmail()

- Purpose: Fetch customer-specific order history
- Usage: Used in user dashboard

d.State

- If status == PENDING → Waiting for approval
- If status == CONFIRMED → Order confirmed
- If status == CANCELLED → Order cancelled
- If status == DELIVERED → Order completed
- If recentOrders(days = 30) → Orders created within the last 30 days
- If rushDelivery = true on any item → Order includes rush deliver
- If order not found → HTTP 404 Not Found
- If invalid status passed in path variable → HTTP 400 Bad Request

4.3.3.16 ProductController

Table 1. Attribute design

#	Name	Data Type	Description
1	productService	ProductService	Service class that handles product-related business logic.
2	productMapper	ProductMapper	Mapper class to convert between entity and DTO representations.

Table 2. Operation design

#	Name	Type	Description (Purpose)
1	getAllProducts()	ResponseEntity<ApiResponse<List<ProductDTO>>>>	Retrieves all products.
2	getProductById(Long)	ResponseEntity<ApiResponse<ProductDTO>>>	Retrieves a product by its ID.
3	getProductByBarcode(String)	ResponseEntity<ApiResponse<ProductDTO>>>	Retrieves a product using its barcode.
4	getProductsByType(String)	ResponseEntity<ApiResponse<List<ProductDTO>>>>	Retrieves products filtered by product type.
5	searchProducts(...)	ResponseEntity<ApiResponse<List<ProductDTO>>>>	Searches products by title, type, price range, and stock availability.
6	createProduct(ProductDTO)	ResponseEntity<ApiResponse<ProductDTO>>>	Creates a new generic product.
7	updateProduct(Long, ProductDTO)	ResponseEntity<ApiResponse<ProductDTO>>>	Updates product information by ID.
8	deleteProduct(Long)	ResponseEntity<ApiResponse<Void>>>	Deletes a product by its ID.
9	updateStock(Long, int)	ResponseEntity<ApiResponse<ProductDTO>>>	Adjusts the quantity in stock for a product.

10	checkAvailability(Long, int)	ResponseEntity<ApiResponse<Boolean>>	Checks if sufficient quantity is available.
11	getLowStockProducts(int)	ResponseEntity<ApiResponse<List<ProductDTO>>>	Gets products with stock lower than a threshold.
12	getRushOrderProducts()	ResponseEntity<ApiResponse<List<ProductDTO>>>	Retrieves products that support rush order shipping.
13	getAllBooks()	ResponseEntity<ApiResponse<List<BookDTO>>>	Retrieves all book-type products.
14	getAllCDs()	ResponseEntity<ApiResponse<List<CDDTO>>>	Retrieves all CD-type products.
15	getAllDVDs()	ResponseEntity<ApiResponse<List<DVDDTO>>>	Retrieves all DVD-type products.
16	createBook(BookDTO)	ResponseEntity<ApiResponse<BookDTO>>	Creates a new book product.
17	createCD(CDDTO)	ResponseEntity<ApiResponse<CDDTO>>	Creates a new CD product.
18	createDVD(DVDDTO)	ResponseEntity<ApiResponse<DVDDTO>>	Creates a new DVD product.

a. Parameter & Return (Selected)

- **getProductById(Long id)**
 - **Parameters:** Product ID (must be positive).
 - **Returns:** ProductDTO if found; otherwise, a 404 NOT FOUND response.
- **searchProducts(String title, String type, Integer minPrice, Integer maxPrice, Boolean inStock)**
 - **Parameters:** Optional filters for search.
 - **Returns:** List of products matching the search criteria.
- **createProduct(ProductDTO productDTO)**
 - **Parameters:** A valid product DTO with all required fields.
 - **Returns:** The saved product in DTO form with 201 CREATED status.
- **updateStock(Long id, int quantity)**
 - **Parameters:** Product ID and the quantity change (positive or negative).
 - **Returns:** Updated product with new quantity in stock.
- **checkAvailability(Long id, int quantity)**
 - **Parameters:** Product ID and desired quantity.
 - **Returns:** Boolean value indicating if the requested quantity is available.
- **createBook(BookDTO bookDTO) / createCD(CDDTO cdDTO) / createDVD(DVDDTO dvdDTO)**
 - **Parameters:** DTO specific to each product type.
 - **Returns:** The saved typed product entity.

b. Exception

This controller handles:

- @Valid validation errors via @Validated annotation.
- Missing products return 404 Not Found.
- All methods log their actions using Slf4j.
- Internal exceptions are not explicitly caught here (assumed to be handled globally via exception handler).

c. Method Details

- **getAllProducts()**
Purpose: Fetches all products in the system.
Usage: Converts all Product entities to ProductDTO and returns them.
- **getProductById(Long id)**
Purpose: Retrieves product information by unique ID.
Usage: Calls productService.getProductById(), and maps result to ProductDTO.
- **getProductByBarcode(String barcode)**
Purpose: Finds a product using its unique barcode.
Usage: Useful in scanning systems or quick lookups.
- **searchProducts(...)**
Purpose: Filters products based on title, type, price range, and stock status.
Usage: Flexible searching mechanism for users or admin panel.
- **updateStock(Long id, int quantity)**
Purpose: Adjusts stock levels.
Usage: Used when placing or canceling orders, or restocking inventory.
- **getLowStockProducts(int threshold)**
Purpose: Monitors inventory to detect products that are nearly out of stock.
Usage: Default threshold is 10, but can be adjusted via query param.
- **getRushOrderProducts()**
Purpose: Retrieves only products eligible for fast delivery.
Usage: Typically filtered by a specific flag or product metadata.
- **createBook() / createCD() / createDVD()**
Purpose: Creates new instances of specific product subtypes.
Usage: Converts the DTO to an entity, saves it via productService, then maps it back to DTO.

4.3.3.17 UserController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	userService	UserService	Injected	Service used to handle business logic for users
2	userMapper	UserMapper	Injected	Mapper for converting between User and UserDTO

Table 2. Operation design

#	Name	Type	Description (purpose)
1	getAllUsers()	ResponseEntity	Retrieve all users
2	getUserById(Long id)	ResponseEntity	Get user by ID

	g id)		
3	getUserByEmail(String email)	ResponseEntity	Get user by email
4	createUser(UserDTO)	ResponseEntity	Create a new user
5	updateUser(Long, UserDTO)	ResponseEntity	Update full user information
6	deleteUser(Long)	ResponseEntity	Delete user by ID
7	searchUsers(...)	ResponseEntity	Search users by name/email/phone
8	getUsersByNameContaining()	ResponseEntity	Get users whose name contains a given string
9	checkEmailExists(String)	ResponseEntity	Check if email exists
10	checkPhoneExists(String)	ResponseEntity	Check if phone exists (stubbed with false)
11	updateUserProfile(...)	ResponseEntity	Update user's name and phone via PATCH
12	updateUserEmail(...)	ResponseEntity<ApiResponse<UserDTO>>	Update user's email via PATCH
13	getUserCount()	ResponseEntity<	Get total number of active users

a. Parameter

getUserById(Long id)

- id: User ID to retrieve.
- Return: A response containing user data if found.

createUser(UserDTO userDTO)

- userDTO: The user data to be created.
- Return: The newly created user wrapped in response.

updateUser(Long id, UserDTO userDTO)

- id: User ID to update.
- userDTO: Updated user data.
- Return: The updated user.

updateUserProfile(Long id, UpdateUserProfileRequest request)

- id: ID of user.
- request: Name and phone to be updated.
- Return: UserDTO with updated fields.

updateUserEmail(Long id, String email)

- id: ID of user.
- email: New email to set.
- Return: UserDTO with updated email.

searchUsers(String name, String email, String phone)

- Optional query parameters.
- Return: List of users matching any of the provided fields.

b. Exception

Explicit exception only thrown in: updateUserProfile(...), updateUserEmail(...)

- RuntimeException: If user with given ID is not found.

c. Method

getUserById(Long id)

- Purpose: Retrieve user by ID.
- Usage: If not found, return 404 with message.

createUser(UserDTO userDTO)

- Purpose: Create new user in the system.
- Usage: Requires valid fields; maps DTO to entity and persists it.

updateUser(Long id, UserDTO userDTO)

- Purpose: Update all fields of a user.
- Usage: Maps DTO, overwrites by ID.

updateUserProfile(...)

- Purpose: Update only name and phone.
- Usage: Fetch user by ID, update partial fields, save.

updateUserEmail(...)

- Purpose: Update only email.
- Usage: Fetch user by ID, update email, return updated user.

checkEmailExists(...), checkPhoneExists(...)

- Purpose: Utility endpoints to check if values exist.
- Usage: Email checked via userService; phone is hardcoded false.

4.3.3.18 AuditLogService

Table 1. Attribute design

#	Name	Data Type	Description
1	auditLogRepository	AuditLogRepository	Repository for performing database operations on AuditLog records.

Table 2. Operation design

#	Name	Return Type	Description (Purpose)
1	logAction(String, String)	AuditLog	Logs a simple action with a message.
2	logAction(String, String, Long, ActionType)	AuditLog	Logs an action with user context.
3	logAction(String, String, Long, ActionType, Long)	AuditLog	Logs an action on a specific entity.
4	getAllLogs()	List<AuditLog>	Retrieves all audit logs, sorted by timestamp descending.
5	getLogsByUser(Long)	List<AuditLog>	Retrieves audit logs created by a specific user.
6	getLogsByActionType(ActionType)	List<AuditLog>	Retrieves logs filtered by action type.

7	getLogsForEntity(String, Long)	List<AuditLog>	Retrieves logs related to a specific entity type and ID.
8	getRecentLogs()	List<AuditLog>	Retrieves logs recorded within the last 24 hours.
9	getLogsByDateRange(LocalDateTime, LocalDateTime)	List<AuditLog>	Retrieves logs within a specific date range.
10	searchLogs(String)	List<AuditLog>	Performs a full-text or keyword-based search in audit logs.
11	getActionStatistics()	List<Object[]>	Retrieves action counts grouped by action type.
12	getUserActivityStatistics()	List<Object[]>	Retrieves aggregated statistics about user activity.
13	getSystemLogs()	List<AuditLog>	Retrieves system-generated logs (i.e., without user context).
14	getUserLogs()	List<AuditLog>	Retrieves user-generated logs (i.e., logs with userId present).
15	countUserActionsInDateRange(Long, LocalDateTime, LocalDateTime)	long	Counts how many actions a user performed in a given time range.
16	getDailyActivityStatistics(int)	List<Object[]>	Retrieves daily activity trends for a specified number of past days.
17	getCriticalActions()	List<AuditLog>	Retrieves actions classified as critical (CREATE, UPDATE, DELETE).
18	logUserLogin(Long, String)	AuditLog	Logs a user login event.

19	logUserLogout(Long, String)	AuditLog	Logs a user logout event.
20	logPayment(Long, Long, String, String)	AuditLog	Logs a payment transaction (success, failed, etc.).
21	logOrderAction(Long, Long, String, String)	AuditLog	Logs an order-related action performed by a user.

a. Parameter & Return (Selected)

- **logAction(String actionName, String note)**
 - **Parameters:** Name of the action and a descriptive note.
 - **Returns:** A persisted AuditLog object.
- **logAction(String, String, Long, ActionType)**
 - **Parameters:** Name, note, user ID, and type of action.
 - **Returns:** A log entry with user context.
- **getLogsByDateRange(LocalDateTime start, LocalDateTime end)**
 - **Parameters:** Start and end timestamps.
 - **Returns:** All logs recorded between the specified timestamps.
- **getDailyActivityStatistics(int days)**
 - **Parameters:** Number of past days to analyze.
 - **Returns:** List of aggregated daily activity statistics.
- **countUserActionsInDateRange(Long userId, LocalDateTime start, LocalDateTime end)**
 - **Parameters:** User ID and date range.
 - **Returns:** Number of actions the user performed during the specified period.

b. Exception

This service:

- Assumes the repository layer handles data validation.
- Uses Spring's `@Transactional` to ensure data consistency.
- Logs debugging info using `Slf4j`.

Exceptions are not thrown directly but would bubble up to be handled by a controller or global handler.

c. Method Details

logAction(...)

- **Purpose:** Records actions in the audit log.
- **Usage:** Called whenever an event such as login, update, or payment occurs.

getAllLogs()

- **Purpose:** Retrieve every log for admin or monitoring dashboards.
- **Usage:** Could be used in log viewers or audit compliance tools.

getRecentLogs()

- **Purpose:** Helps monitor recent activities for incident response.
- **Usage:** Displays logs within the last 24 hours.

searchLogs(String term)

- **Purpose:** Keyword search across action name, notes, etc.
- **Usage:** Used in admin dashboards for quick filtering.

getActionStatistics() / getUserActivityStatistics()

- **Purpose:** Provides summarized insight.
- **Usage:** Useful for generating charts and reports.

logUserLogin(...) / logUserLogout(...)

- **Purpose:** Track user authentication behavior.
- **Usage:** Often used for security and session management tracking.

logPayment(...)

- **Purpose:** Track all payment attempts and statuses.
- **Usage:** Supports debugging payment gateway issues and reconciling transactions.

getCriticalActions()

- **Purpose:** Filters out actions like CREATE, UPDATE, and DELETE.
- **Usage:** Helps audit sensitive operations.

4.3.3.19 AuthService

Table 1. Attribute design

#	Name	Data Type	Description
1	userService	UserService	Provides access to user-related business logic and data.
2	userMapper	UserMapper	Converts between User entity and UserDTO.

Table 2. Operation design

#	Name	Return Type	Description
1	validateLogin(String, String)	UserDTO	Validates user credentials (email and password).
2	getCurrentUser(Long)	UserDTO	Retrieves details of a user by ID if the user is active.
3	isManager(Long)	boolean	Checks whether the given user is a manager.
4	canManageProducts(Long)	boolean	Checks if the user can manage products (must be either admin or manager).

a. Parameter & Return

- **validateLogin(String email, String password)**
 - **Parameters:** User's email and plain-text password.
 - **Returns:** A UserDTO object if credentials are valid; otherwise, returns null.
- **getCurrentUser(Long userId)**
 - **Parameters:** User ID.

- **Returns:** UserDTO if the user is found and active; otherwise null.
- **isManager(Long userId)**
 - **Parameters:** User ID.
 - **Returns:** true if the user has the role "MANAGER", otherwise false.
- **canManageProducts(Long userId)**
 - **Parameters:** User ID.
 - **Returns:** true if the user has role "ADMIN" or "MANAGER"; otherwise false.

b. Exception Handling

- All methods catch internal exceptions and:
 - Either throw a RuntimeException with contextual information (e.g., validateLogin, getCurrentUser)
 - Or return false for Boolean methods to ensure the system doesn't crash during authorization checks.

c. Method Details

validateLogin(...)

- Purpose: To validate user credentials during login.
- Logic: Checks email and password against stored user data; returns user info if authenticated.
- Use Case: Called by AuthController to initiate a login session.

getCurrentUser(...)

- Purpose: To retrieve the currently logged-in user based on session-stored user ID.
- Logic: Returns the user if they exist and are active.
- Use Case: Session management and access control.

isManager(...)

- Purpose: Role-checking method for managerial authorization.
- Logic: Gets the current user and checks if their role is "MANAGER".

canManageProducts(...)

- Purpose: Ensures a user has privileges to manage products in the system.
- Logic: Returns true if the user is either an "ADMIN" or "MANAGER".

4.3.3.20 InvoiceService

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	invoiceRepositor	InvoiceRepository	Injected	Repository interface for

	y			interacting with the invoice table
2	orderRepository	OrderItemListRepository	Injected	Repository interface for accessing order data
3	auditLogService	AuditLogService	Injected	Service used to log audit actions (e.g. create, pay, refund invoice)

Table 2. Operation design

#	Name	Type	Description (purpose)
1	getAllInvoices()	List<Invoice>	Get all invoices sorted by creation date
2	getInvoiceById(id)	Optional<Invoice>	Find invoice by ID
3	getInvoiceByOrderId(orderId)	Optional<Invoice>	Find invoice by associated order ID
4	createInvoice(orderId, desc)	Invoice	Create new invoice for a given order
5	markAsPaid(id, txnId, method)	Invoice	Update invoice status to PAID and save transaction ID
6	markAsFailed(id, reason)	Invoice	Mark payment attempt as failed
7	getInvoicesByStatus(status)	List<Invoice>	Filter invoices by payment status
8	getPaidInvoices()	List<Invoice>	Get invoices where status is PAID
9	getPendingInvoices()	List<Invoice>	Get invoices where status is PENDING
10	getFailedInvoices()	List<Invoice>	Get invoices where status is FAILED
11	getInvoiceByTransactionId()	Optional<Invoice>	Find invoice by VNPay transaction ID
12	getTotalPaidRevenue()	Long	Get total revenue from paid invoices
13	getRevenueByDateRange(start, end)	Long	Calculate revenue within a specific time window
14	getOverdueInvoices(days)	List<Invoice>	Fetch invoices overdue by X days (unpaid & old)
15	getPaymentMethodStatistics()	List<Object[]>	Return aggregate statistics on payment method usage
16	getDailyRevenueStatistics(days)	List<Object[]>	Return daily revenue trends for last X days
17	countInvoicesByStatus(status)	long	Count invoices matching a given payment status
18	processPayment(id, method, data)	Invoice	Handle payment logic and simulate integration with gateway
19	refundPayment(id, reason)	Invoice	Mark invoice as refunded

a. Parameter

`createInvoice(orderId: Long, description: String):`

- Creates invoice for an order
- Return: Newly created Invoice object

`markAsPaid(invoiceId, transactionId, paymentMethod):`

- Marks invoice as PAID, records payment info
- Return: Updated invoice

`markAsFailed(invoiceId, reason):`

- Marks payment attempt as failed
- Return: Invoice with status FAILED

`getRevenueByDateRange(startDate, endDate):`

- Computes revenue between two timestamps
- Return: Long representing revenue amount

`refundPayment(invoiceId, reason):`

- Set status to REFUNDED and logs reason
- Return: Refunded invoice

b. Exception

`createInvoice(orderId):`

- Throws `RuntimeException` if order not found
- Throws `IllegalStateException` if invoice already exists for order

`markAsPaid(invoiceId):`

- Throws `RuntimeException` if invoice not found
- Throws `IllegalStateException` if invoice already marked as paid

`refundPayment(invoiceId):`

- Throws `RuntimeException` if invoice not found
- Throws `IllegalStateException` if invoice not paid yet

`processPayment(invoiceId):`

- On error, logs failure, saves status as FAILED, and throws `RuntimeException`

c. Method

createInvoice()

- Purpose: Create a new invoice linked to a valid order
- Usage: Called when order is confirmed and invoice needs to be tracked

markAsPaid()

- Purpose: Mark an invoice as paid with VNPay transaction
- Usage: Used when payment is completed successfully

processPayment()

- Purpose: Simulate integration with payment provider
- Usage: Generates mock transaction ID, saves payment status

getOverdueInvoices(days)

- Purpose: Identify invoices created before X days that are still unpaid
- Usage: Used for financial review or notification system

refundPayment()

- Purpose: Mark invoice as refunded and log reason
- Usage: Used by admin when issuing a refund

getPaymentMethodStatistics()

- Purpose: Aggregate usage count of each payment method
- Usage: Useful for reporting and business insights

d.State

- If invoice not found → Throws RuntimeException
- If invoice already exists for order → Throws IllegalStateException
- If invoice is already paid → State: PAID, further payments blocked
- If invoice is unpaid after N days → Considered overdue
- If invoice is successfully refunded → Status = REFUNDED
- If payment fails in processPayment() → Status = FAILED, reason is logged
- If auditLogService.logPayment(...) called → State change is tracked/audited

4.3.3.21 OrderService

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderItemListRepository	OrderItemListRepository	Injected	Access order list (order-level data)

2	orderItemRepository	OrderItemRepository	Injected	Access individual order items
3	productRepository	ProductRepository	Injected	Used to fetch and update product stock
4	deliveryRepository	DeliveryInformationRepository	Injected	Used to save delivery address/info
5	invoiceRepository	InvoiceRepository	Injected	Create invoice after successful order
6	auditLogService	AuditLogService	Injected	Logs all actions related to orders, payment, and delivery

Table 2. Operation design

#	Name	Type	Description (purpose)
1	createOrder(cartItems, deliveryInfo)	OrderItemList	Create order, reduce stock, save delivery, log, and create invoice
2	getOrderById(id)	Optional<OrderItemList>	Retrieve order by its unique ID
3	getAllOrders()	List<OrderItemList>	Return all orders sorted by created time
4	getOrdersByStatus(status)	List<OrderItemList>	Retrieve orders filtered by status enum
5	getOrdersByCustomerEmail(email)	List<OrderItemList>	Fetch orders made by a customer via email
6	confirmOrder(id)	OrderItemList	Confirm an order and update status
7	cancelOrder(id, reason)	OrderItemList	Cancel order, restore stock, and log
8	updateOrderStatus(id, status)	OrderItemList	Set a custom status for the order (e.g. shipped, delivered)
9	getPendingOrders()	List<OrderItemList>	Return all orders currently in pending status
10	getRecentOrders(days)	List<OrderItemList>	Orders created in the last X days
11	getOrderStatistics()	List<Object[]>	Aggregate order count, revenue by status or time
12	getTotalRevenue(start, end)	Long	Total value of all orders in a time range
13	getOrdersWithRushDelivery()	List<OrderItemList>	Filter orders containing rush delivery items

14	getCancellableOrders()	List<OrderItem>	Orders still eligible for cancellation
15	addRushDelivery(orderItemId, instructions)	OrderItem	Add rush delivery option to a specific order line
16	validateCartItems(cartItems)	void	Ensures all cart items are valid before placing an order

a. Parameter

createOrder(cartItems: List<CartItem>, deliveryInfo: DeliveryInformation):

- cartItems: Products and quantity
- deliveryInfo: Shipping info (name, phone, address...)
- Return: Created order with generated ID and invoice

confirmOrder(orderId: Long):

- Confirms the order and logs action
- Return: Updated order object

cancelOrder(orderId: Long, reason: String):

- Cancels order and restores inventory
- Return: Order object with CANCELLED status

updateOrderStatus(orderId: Long, newStatus: OrderStatus):

- Set custom status (e.g. SHIPPED) for order
- Return: Updated order

addRushDelivery(orderItemId: Long, instructions: String):

- Mark a line item as rush order
- Return: Updated OrderItem object

b. Exception

createOrder() : IllegalArgumentException if cart is empty, null, invalid product, or insufficient stock

confirmOrder(), cancelOrder(), updateOrderStatus() : RuntimeException if order not found

addRushDelivery()

- RuntimeException if order item not found
- IllegalStateException if product doesn't support rush delivery

c. Method

createOrder()

- Purpose: Process cart into saved order + invoice
- Usage: Called when user submits a cart and delivery info

confirmOrder()

- Purpose: Change order state from PENDING to CONFIRMED
- Usage: Admin or automated trigger

cancelOrder()

- Purpose: Set order status to CANCELLED, restore stock
- Usage: Triggered by customer or timeout

addRushDelivery()

- Purpose: Flag rush delivery for 2-hour dispatch
- Usage: Called when customer opts-in per item basis

getRecentOrders()

- Purpose: Get orders placed in the last X days
- Usage: Dashboard, metrics, recent activities

getTotalRevenue()

- Purpose: Get money value from orders (not invoices)
- Usage: Reporting

d.State

- If cartItems == null || empty → Cart is invalid → Exception
- If any product is null in cart → Invalid cart item
- If requestedQuantity > product.stock → Stock insufficient → Exception
- If order.status == PENDING → Awaiting confirmation
- If order.status == CONFIRMED → Confirmed and ready
- If order.status == CANCELLED → Cancelled and logged
- If product.rushOrderSupported == false in addRushDelivery() → Rush delivery not available
- If orderId not found → RuntimeException (invalid line)

4.3.3.22 ProductService

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productRepository	ProductRepository	Injected	Main repository for all product types
2	bookRepository	BookRepository	Injected	Repository for Book entities
3	cdRepository	CDRepository	Injected	Repository for CD entities
4	dvdRepository	DVDRepository	Injected	Repository for DVD entities
5	auditLogService	AuditLogService	Injected	For logging product-related actions

Table 2. Operation design

#	Name	Type	Description (purpose)
1	getAllProducts()	List<Product>	Fetch all products from DB
2	getProductById(Long id)	Optional<Product>	Find product by ID
3	getProductByBarcode(String barcode)	Optional<Product>	Find product by barcode
4	getProductsByType(String type)	List<Product>	Get products filtered by type
5	searchProducts(String, String, Integer, Integer, Boolean)	List<Product>	Search with filters
6	saveProduct(Product product)	Product	Validate & save Book, CD, or DVD
7	updateProduct(Long id, Product updated)	Product	Update a product by ID
8	deleteProduct(Long id)	void	Delete product by ID
9	updateStock(Long productId, int quantity)	Product	Add or subtract stock
10	checkAvailability(Long productId, int quantity)	boolean	Check if stock is enough
11	getLowStockProducts(int threshold)	List<Product>	Products with quantity below threshold
12	getRushOrderPro	List<Product>	Products marked for rush delivery

	ducts()		
13	validateProduct(Product)	void	Internal validation method
14	getProductCountByType(String type)	long	Count products by type
15	getAllBooks()	List<Book>	Fetch all books
16	getAllCDs()	List<CD>	Fetch all CDs
17	getAllDVDs()	List<DVD>	Fetch all DVDs

a. Parameters

getProductById(Long id)

- id: ID of the product to retrieve.
- **Return:** An Optional<Product> containing the product if found.

getProductByBarcode(String barcode)

- barcode: Unique barcode identifier.
- **Return:** An Optional<Product> if the barcode exists.

getProductsByType(String type)

- type: The type of product (book, cd, dvd).
- **Return:** List of products of the given type.

searchProducts(String title, String type, Integer minPrice, Integer maxPrice, Boolean inStock)

- title: (Optional) Keyword in product title.
- type: (Optional) Product type.
- minPrice: (Optional) Minimum price.
- maxPrice: (Optional) Maximum price.
- inStock: (Optional) If true, only return items with stock > 0.
- **Return:** List of products matching the criteria.

saveProduct(Product product)

- product: The product object to be created. Supports polymorphism (Book, CD, DVD).

- **Return:** The saved product with generated ID.

updateProduct(Long id, Product updatedProduct)

- id: Product ID to update.
- updatedProduct: New product details.
- **Return:** The updated product after saving.

deleteProduct(Long id)

- id: ID of the product to delete.
- **Return:** void (operation logs deletion via AuditLogService).

updateStock(Long productId, int quantity)

- productId: ID of product to update.
- quantity: Quantity to add or subtract (negative means reduce).
- **Return:** Updated product with new stock quantity.

checkAvailability(Long productId, int requestedQuantity)

- productId: ID of the product.
- requestedQuantity: Quantity requested for order.
- **Return:** true if available stock $\geq$ requested, else false.

getLowStockProducts(int threshold)

- threshold: Stock threshold.
- **Return:** List of products where stock $<$ threshold.

getRushOrderProducts()

- **Return:** List of products where rushOrderSupported = true.

getProductCountByType(String type)

- type: Product type (book, cd, dvd).
- **Return:** Total count of that type.

getAllBooks(), getAllCDs(), getAllDVDs()

- **Return:** All items of the respective subclass.

b. Exception

- `saveProduct(...)`, `updateProduct(...)`, `deleteProduct(...)`, `updateStock(...)`
 - `RuntimeException`: If product not found with given ID.
- `saveProduct(...)`:
 - `IllegalArgumentException`: If required fields are missing or invalid (title, price, barcode duplication, negative quantity).

c. Method Purpose and Usage

`getProductById(Long id)`

- **Purpose**: Retrieve a product by its unique database ID.
- **Usage**: Calls `productRepository.findById(id)` to get an `Optional<Product>`. Used in APIs where the client needs to fetch product details.

`getProductByBarcode(String barcode)`

- **Purpose**: Find a product using its barcode.
- **Usage**: Useful for scanning products at checkout or inventory systems. Calls `productRepository.findByBarcode(barcode)`.

`getProductsByType(String type)`

- **Purpose**: Retrieve all products of a specific type (Book, CD, DVD).
- **Usage**: Filters based on the discriminator column. Often used to display or manage items by category.

`searchProducts(String title, String type, Integer minPrice, Integer maxPrice, Boolean inStock)`

- **Purpose**: Perform a flexible multi-criteria product search.
- **Usage**: Calls a custom query in `productRepository` to return results based on a combination of filters. Supports advanced filtering on UI.

`saveProduct(Product product)`

- **Purpose**: Persist a new product in the system.
- **Usage**: Automatically determines subtype (Book, CD, or DVD) and saves using the corresponding repository. Also performs validation and logs audit.

`updateProduct(Long id, Product updatedProduct)`

- **Purpose:** Update base product fields (title, price, barcode, etc.).
- **Usage:** Retrieves the existing product, applies updates, and saves. Does not update subtype-specific fields.

`deleteProduct(Long id)`

- **Purpose:** Delete a product permanently.
- **Usage:** Finds the product by ID, deletes it from the repository, and logs the delete operation to AuditLog.

`updateStock(Long productId, int quantity)`

- **Purpose:** Adjust inventory levels of a product.
- **Usage:** Uses `addStock()` if quantity is positive, or `reduceStock()` if negative. Saves the result and logs the action.

`checkAvailability(Long productId, int requestedQuantity)`

- **Purpose:** Determine if a product has enough quantity for a request.
- **Usage:** Calls `hasStock()` on the product instance. Returns boolean for use in ordering or cart validation logic.

`getLowStockProducts(int threshold)`

- **Purpose:** List all products below a specific inventory threshold.
- **Usage:** Calls `productRepository.findLowStockProducts(threshold)`. Useful for triggering restocking alerts.

`getRushOrderProducts()`

- **Purpose:** List products that support rush delivery.
- **Usage:** Used to allow fast-shipping filtering. Calls repository to find products with `rushOrderSupported = true`.

`getAllBooks(), getAllCDs(), getAllDVDs()`

- **Purpose:** Retrieve all products of a particular subclass.
- **Usage:** Calls individual repositories. Supports type-specific management features in UI.

`getProductCountByType(String type)`

- **Purpose:** Return the total number of products of a given type.

- **Usage:** Calls repository method using discriminator field. Used for dashboard statistics or analytics.

4.3.3.23 UserService

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	userRepository	UserRepository	null	Repository interface for performing DB operations on users.
2	auditLogService	AuditLogService	injected	Service used to log audit actions for create/update/delete.

Table 2. Operation design

#	Name	Type	Description (purpose)
1	getAllUsers()	List	Fetch all users from database.
2	getUserById(Long id)	Optional	Find user by ID.
3	getUserByEmail(String e)	Optional	Find user by email.
4	createUser(User u)	User	Save a new user after validation.
5	updateUser(Long, User)	User	Update user fields like name, role, etc.
6	deleteUser(Long id)	void	Soft delete user (mark inactive).
7	authenticateUser(..)	Optional	Validate login using email and password.
8	getUsersByRole(role)	List	Retrieve users based on role.
9	getActiveUsers()	List	Fetch users who are currently active.
10	getAdminUsers()	List	Retrieve users with ADMIN role.
11	getProductManagers()	List	Retrieve users with ADMIN or MANAGER role.
12	getCustomers()	List	Retrieve users with CUSTOMER role.
13	searchUsers(String term)	List	Search users by matching input term.

14	updatePassword(..)	User	Validate and update a user's password.
15	setUserActive(...)	User	Activate or deactivate a user account.
16	getUserCountByRole(role)	long	Count number of users in a given role.
17	getActiveUserCount()	long	Count number of active users.
18	validateUser(User user)	void (private)	Internal validation of user fields.

a, Parameter

getUserById(Long id)

- id: User ID to retrieve.
- **Return:** Optional of found User.

getUserByEmail(String email)

- email: Email to search.
- **Return:** Optional of found User.

createUser(User user)

- user: Object containing user data to be saved.
- **Return:** Persisted user.

updateUser(Long id, User updatedUser)

- id: User ID to update.
- updatedUser: Fields to be updated.
- **Return:** Updated user object.

deleteUser(Long id)

- id: ID of user to be deactivated.
- **Return:** None (void).

authenticateUser(String email, String password)

- email: User email.
- password: Raw password to check.
- **Return:** Optional of authenticated user.

searchUsers(String searchTerm)

- searchTerm: Value to search across name, email, phone.
- **Return:** List of matching users.

updatePassword(Long userId, String oldPassword, String newPassword)

- userId: ID of user.
- oldPassword: Existing password.
- newPassword: New password (validated).
- **Return:** User with updated password.

setUserActive(Long userId, boolean active)

- userId: ID of user.
- active: true = activate, false = deactivate.
- **Return:** User with updated status.

getUsersByRole(User.UserRole role)

- role: Role to filter (e.g., ADMIN).
- **Return:** List of users with that role.

getUserCountByRole(User.UserRole role)

- role: Count users for this role.
- **Return:** Number of users.

b. Exception

- **RuntimeException**
 - Thrown if user not found in updateUser, deleteUser, updatePassword, setUserActive.
- **IllegalArgumentException**
 - Thrown for validation failures (missing fields, invalid email, bad password, etc.).
- Most read-only functions are safe and do not throw explicitly unless result is missing.

c. Method

getAllUsers()

- **Purpose:** Retrieve all users in the system.
- **Usage:** Simple fetch of all records from the database using userRepository.findAll().

getUserById(Long id)

- **Purpose:** Find a user by their unique ID.
- **Usage:** Returns an Optional<User> object using userRepository.findById(id).

getUserByEmail(String email)

- **Purpose:** Fetch user by email address.
- **Usage:** Uses userRepository.findByEmail(email) to find a user. Common for login or validation.

createUser(User user)

- **Purpose:** Create and persist a new user in the database.
- **Usage:** Validates user fields, checks for email uniqueness, assigns defaults, saves, and logs the action.

updateUser(Long id, User updatedUser)

- **Purpose:** Update core user fields.
- **Usage:** Fetches user by ID, updates fields like name, phone, role, salary, and active status. Email is not updated.

deleteUser(Long id)

- **Purpose:** Soft-delete a user by marking them as inactive.
- **Usage:** Sets isActive = false and saves the change. Logs the deletion.

authenticateUser(String email, String password)

- **Purpose:** Check user credentials.
- **Usage:** Compares raw password (no hashing) and ensures user is active. Logs login attempts.

getUsersByRole(UserRole role)

- **Purpose:** Retrieve users by role (e.g., ADMIN, MANAGER).
- **Usage:** Queries userRepository.findByRole(...).

getActiveUsers()

- **Purpose:** List all active users.
- **Usage:** Returns users with isActive = true.

getAdminUsers()

- **Purpose:** Fetch users who are admins and active.
- **Usage:** Calls userRepository.findActiveAdmins().

getProductManagers()

- **Purpose:** Retrieve users eligible to manage products.
- **Usage:** Calls repository method for users with roles like ADMIN or MANAGER.

getCustomers()

- **Purpose:** Retrieve users with the CUSTOMER role.
- **Usage:** Calls userRepository.findCustomers().

searchUsers(String searchTerm)

- **Purpose:** Search users by a general search term.
- **Usage:** Delegates to a repository method that matches across name/email/phone.

updatePassword(Long userId, String oldPassword, String newPassword)

- **Purpose:** Change the user's password.
- **Usage:** Verifies old password, validates new password, updates field, and logs the change.

setUserActive(Long userId, boolean active)

- **Purpose:** Activate or deactivate a user.
- **Usage:** Sets isActive to the given boolean value and logs the change.

getUserCountByRole(UserRole role)

- **Purpose:** Get number of users with a specific role.
- **Usage:** Calls userRepository.countByRole(...).

getActiveUserCount()

- **Purpose:** Count all currently active users.
- **Usage:** Returns userRepository.countByIsActiveTrue().

validateUser(User user)

- **Purpose:** Ensure user object is valid before persistence.
- **Usage:** Checks non-null fields, validates email format and password length.

5. Design Considerations

<Describe issues which need to be addressed or resolved before attempting to devise a complete design solution. Remember that, you have to refactor your source code to strictly follow the final design>

5.1 Goals and Guidelines

The following goals, guidelines, principles, and priorities dominate the design of the AIMS system and its software components.

5.1.1 Performance Goals

1. Response Time Optimization

- **Goal:** Achieve maximum response time of 2 seconds under normal conditions and 5 seconds during peak hours
- **Rationale:** E-commerce applications require fast response times to prevent user frustration and cart abandonment
- **Implementation:**
 - Implement caching mechanisms for frequently accessed product data
 - Use connection pooling for database operations
 - Optimize database queries with proper indexing
 - Implement lazy loading for product images

2. Concurrent User Support

- **Goal:** Support up to 1,000 simultaneous users without performance degradation
- **Rationale:** Scale to meet business requirements for multiple simultaneous customers
- **Implementation:**
 - Thread-safe singleton patterns for shared resources
 - Session management with minimal memory footprint
 - Efficient connection handling and resource management

3. System Reliability

- **Goal:** 300 hours continuous operation without failure, 1-hour maximum recovery time
- **Rationale:** Minimize business disruption and revenue loss during outages
- **Implementation:**

- Robust exception handling and logging
- Automatic failover mechanisms
- Health monitoring and alerting systems

5.1.2 Modularity and Maintainability

- **Goal:** Create loosely coupled, highly cohesive components
- **Rationale:** Enable independent development, testing, and maintenance of system modules
- **Implementation:**
 - Clear separation between presentation, business logic, and data access layers
 - Use of dependency injection for component dependencies
 - Interface-based programming for core services

1. Scalability

- **Goal:** Design for future growth in product catalog, user base, and feature set
- **Rationale:** Accommodate business expansion without major architectural changes
- **Implementation:**
 - Abstract factory pattern for different media product types
 - Plugin architecture for payment gateway integration
 - Configurable business rules and pricing policies

2. Security First

- **Goal:** Protect sensitive customer and payment information
- **Rationale:** Build trust with customers and comply with data protection regulations
- **Implementation:**
 - Token-based authentication for user sessions
 - Encrypted password storage using BCrypt
 - Secure communication with payment gateways
 - Input validation and SQL injection prevention

5.1.3 User Experience Guidelines

1. Intuitive Navigation

- **Goal:** Minimize learning curve for new users
- **Rationale:** Reduce customer support burden and increase conversion rates

- **Implementation:**
 - Consistent UI patterns across all screens
 - Clear visual hierarchy and navigation breadcrumbs
 - Progressive disclosure of complex features
- 2. **Error Prevention and Recovery**
 - **Goal:** Prevent user errors and provide clear recovery mechanisms
 - **Rationale:** Reduce frustration and support requests
 - **Implementation:**
 - Real-time input validation with clear error messages
 - Confirmation dialogs for destructive actions
 - Automatic session recovery mechanisms

5.1.4 Development Guidelines

1. **Coding Standards**
 - **Naming Conventions:** CamelCase for classes, camelCase for methods and variables
 - **Package Structure:** Organized by feature/module (e.g., com.aims.product, com.aims.order)
 - **Code Documentation:** Javadoc comments for all public classes and methods
 - **Version Control:** Feature branch workflow with pull request reviews
2. **Quality Assurance**
 - **Unit Test Coverage:** Minimum 80% coverage for business logic classes
 - **Integration Testing:** Comprehensive testing of API integrations
 - **Performance Testing:** Load testing to verify concurrent user requirements
 - **Security Testing:** Regular vulnerability scans and penetration testing
3. **Database Design Guidelines**
 - **Normalization:** 3NF normalization with denormalization only for proven performance benefits
 - **Indexing Strategy:** Indexes on all foreign keys and frequently searched columns
 - **Transaction Management:** ACID compliance for all financial operations
 - **Audit Trails:** Complete logging of all data modifications

5.1.5 Integration Guidelines

1. **External Service Integration**
 - **Goal:** Seamless integration with third-party services (VNPay, Email)

- **Rationale:** Leverage existing services while maintaining system independence
- **Implementation:**
 - Adapter pattern for external service interfaces
 - Retry mechanisms with exponential backoff
 - Circuit breaker pattern for fault tolerance

2. API Design

- **Goal:** Consistent and versioned API interfaces for future integration
- **Rationale:** Enable future mobile applications or third-party integrations
- **Implementation:**
 - RESTful principles for internal service communication
 - JSON for data exchange with clear schema definitions
 - API versioning strategy for backward compatibility

5.1.6 Business Logic Guidelines

1. Business Rule Centralization

- **Goal:** Centralize business rules in service layer
- **Rationale:** Ensure consistency across different user interfaces and future integrations
- **Implementation:**
 - Rule engine pattern for configurable business rules
 - Service facade pattern for complex business operations
 - Strategy pattern for different pricing and delivery calculations

2. Transaction Handling

- **Goal:** Ensure data consistency across all business operations
- **Rationale:** Prevent data corruption and maintain inventory accuracy
- **Implementation:**
 - Unit of Work pattern for complex operations
 - Compensation transactions for external service failures
 - Event sourcing for audit and recovery capabilities

These goals and guidelines form the foundation for making design decisions throughout the development process, ensuring the system meets both functional requirements and quality attributes.

5.2 *Architectural Strategies*

The following architectural strategies provide insight into the key design decisions and abstractions used in the AIMS system architecture.

5.2.1 Layered Architecture Strategy

Decision: Implement a 3-tier layered architecture with clear separation of concerns

Reasoning:

- Supports modularity and maintainability goals
- Enables independent development and testing of layers
- Facilitates future enhancements without affecting other layers

Implementation:

- **Presentation Layer:** User interface components (Views, Controllers)
- **Business Logic Layer:** Service classes, business rules, and domain models
- **Data Access Layer:** Repository pattern implementation, database access

Trade-offs:

- Higher initial development complexity vs. long-term maintainability benefits
- Potential performance overhead from layer transitions vs. clean separation of concerns

5.2.2 Technology Stack Selection

Decision: Java-based technology stack with SQLite database **Reasoning:**

- Platform independence for desktop deployment
- Robust ecosystem with extensive libraries
- Strong threading capabilities for concurrent user support
- Existing team expertise

Components:

- **Programming Language:** Java (for backend), JavaScript (for frontend)
- **Database:** SQLite
- **ORM Framework:** Spring Data JPA
- **UI Framework:** React.js

- **Build Tool:** Maven
- **Version Control:** Git

Trade-offs:

- Java's memory footprint vs. performance and ecosystem benefits

5.2.3 Module-Based Architecture

Decision: Organize system into functional modules with well-defined interfaces

Reasoning:

- Supports scalability goals for future feature additions
- Enables parallel development by different team members
- Facilitates plugin architecture for payment gateways

Key Modules:

- **Product Management Module:** Catalog, inventory, pricing
- **Order Processing Module:** Cart, checkout, delivery
- **User Management Module:** Authentication, authorization
- **Payment Integration Module:** VNPay interface, transaction handling
- **Notification Module:** Email services, order updates

Communication Strategy: Event-driven architecture within modules, direct method calls between layers

5.2.4 Data Management Strategy

Decision: Centralized database with distributed caching **Reasoning:**

- Ensures data consistency for financial transactions
- Supports concurrent user requirements through caching
- Simplifies backup and recovery procedures

Implementation Details:

- **Primary Storage:** SQLite — used as the main relational database for persisting application data
- **Caching:** In-memory cache (Ehcache) for product catalog

- **Session Management:** Handled by Spring Boot — most likely using HTTP session stored in memory by default.
- **Transaction Management:** Managed by Spring Framework — using Spring's `@Transactional` annotation to ensure ACID compliance for database operations.

Trade-offs:

- Single database vs. microservices approach (chosen for data consistency)
- Cache complexity vs. performance requirements

5.2.5 External Integration Strategy

Decision: Adapter pattern for external service integration **Reasoning:**

- Isolates system from changes in external APIs
- Enables easy switching of service providers
- Supports testing with mock implementations

External Services:

- **Payment Gateway:** VNPay Sandbox API
- **Email Service:** JavaMail API
- **Future Extensions:** Google Maps API for delivery tracking

Implementation:

- Interface-based service definitions
- Retry mechanisms with circuit breaker pattern
- Asynchronous processing for non-critical operations

5.2.6 Concurrency and Synchronization Strategy

Decision: Thread-safe implementations with optimistic locking **Reasoning:**

- Supports 1,000 concurrent users requirement
- Minimizes performance impact from locking
- Prevents race conditions in inventory management

Key Approaches:

- **Inventory Management:** Optimistic locking with version control
- **Session Management:** ThreadLocal storage for user context

- **Database Connections:** Connection pooling (HikariCP)
- **Async Operations:** CompletableFuture for email notifications

Critical Sections:

- Product price updates (limited to 2 per day)
- Order placement with inventory reservation
- Payment transaction processing

5.2.7 Error Detection and Recovery Strategy

Decision: Comprehensive exception handling with graceful degradation **Reasoning:**

- Meets 1-hour maximum recovery time requirement
- Maintains system availability during partial failures
- Provides clear error reporting for debugging

Implementation:

- **Global Exception Handler:** Catches uncaught exceptions
- **Compensation Transactions:** Rollback for failed operations
- **Circuit Breaker:** Prevents cascade failures in external services
- **Logging Framework:** SLF4J with Logback for audit trails

Recovery Mechanisms:

- Automatic retry for transient failures
- Manual intervention workflows for critical errors
- Data backup and restore procedures

5.2.8 Security Architecture Strategy

Decision: Token-based authentication with role-based access control **Reasoning:**

- Supports stateless operation for scalability
- Enables fine-grained permission management
- Integrates with existing Java security frameworks

Security Components:

- **Authentication:** Handled by the backend using Spring Security (or custom service) — users log in with credentials, which are validated against stored data

- **Authorization:** Role-based access control (RBAC) — user roles determine access to specific endpoints or UI components.
- **Data Encryption:** BCrypt for passwords, TLS for communication
- **Input Validation:** Input validation is handled on both sides:
 - **Frontend:** Basic form validation via React (e.g., required fields, formats).
 - **Backend:** Validation using Spring annotations (e.g., `@NotNull`, `@Email`) to protect data integrity and prevent malformed requests.

Security Layers:

- Network security (HTTPS)
- Application security (authentication/authorization)
- Data security (encryption at rest and in transit)

5.2.9 User Interface Strategy

Decision: Adopt a **component-based single-page application (SPA)** using React.js to ensure fast, responsive, and maintainable user experiences across the E-commerce platform.

UI Architecture:

- **Pages** are composed of reusable **UI components** (e.g., product cards, navigation bars, forms).
- State management is handled via React's built-in hooks (or may be extended with libraries like Redux or Context API).
- API communication with the Spring Boot backend is performed using `fetch` or `axios`, following RESTful conventions.
- Layout separation supports clear distinction between public pages (login, home, product listings) and protected/admin views.

UX Principles (modular component-based architecture):

- **Clarity:** Clean, intuitive layout with consistent navigation and clear call-to-action buttons.
- **Responsiveness:** Designed for usability on both desktop and mobile devices.
- **Feedback:** Real-time feedback for user actions (e.g., loading indicators, form validation messages).
- **Accessibility:** Ensures essential accessibility practices, such as keyboard navigation and semantic HTML elements, are followed where applicable.

5.2.10 Extensibility Strategy

Decision: Plugin architecture for future feature additions **Reasoning:**

- Supports long-term maintenance and enhancement
- Enables third-party integrations without core changes
- Facilitates A/B testing of new features

Extension Points:

- Payment gateway interfaces for new providers
- Delivery service integrations
- Reporting module for business analytics
- Product type extensions (digital media in future)

Configuration Management:

- External configuration files for business rules
- Environment-specific settings management
- Feature flags for gradual rollout

These architectural strategies collectively address the system's requirements while providing a foundation for future growth and maintenance. Each decision balances immediate needs against long-term sustainability, resulting in a robust and scalable e-commerce platform.

5.3 Design and Program Evaluation

5.3.1 Cohesion & SRP

Table 4. Cohesion & SRP of AIMS

#	<i>Class Name</i>	<i>PIC</i>	<i>Cohesion</i>	<i>SRP</i>	<i>Solution</i>
1	AddProductToCartControllerProduct(Book, CD, DVD, LP)	Vu Hai Dang-20225962	Communicational - Methods operate on the same data (carts and products) but serve multiple responsibilities. Contains validation,	No - Handles multiple concerns: 1) Product retrieval and validation 2) Cart retrieval	Extract services: 1) Create CartService with methods like getCart(), addProductToCart() 2) Create ProductService with getProduct(), checkProductAvailability()

			product checking, and cart modification logic .Functional Methods operate around the same concept (product data and stock), but mix multiple responsibilities.	and management 3) Inventory checking.No	3) Make controller focus only on coordinating these services.The class mixes entity definition, business logic (e.g., pricing, stock management), and lifecycle handling. Refactor as follows: 1. Move stock-related methods (hasStock, reduceStock, addStock) to a StockService. 2. Move calculateTotalPrice() to a separate utility or pricing service. 3. Keep the Product class as a pure data model (entity).
2	UpdateCartControllerAuditLog	Vu Hai Dang-20225962	Communicational - Similar to AddProductToCartController, duplicates logic but focuses on cart updates. Contains similar validation, retrieval, and modification logic.Communication 1 Methods work around the same data (audit entries), but combine data modeling, formatting, and object creation logic.	No - Contains multiple concerns: 1) Product retrieval, 2) Cart retrieval and management, 3) Quantity validationNo	Share services: 1) Use the same CartService and ProductService as AddProductToCartController. 2) Remove duplicated getProduct() and getCart() methods. 3)Focus controller solely on update operations logic.The class handles too many concerns: data persistence, business logic (log creation), and utility formatting. Refactor as follows: 1. Move the create(...) factory methods to an AuditLogFactory or AuditLogService. 2. Keep the entity clean with only fields and annotations. 3. Move getFormattedTimestamp() to a utility/helper class or a formatter.
3	ViewCartController	Vu Hai Dang-20225962	Communicational - Methods operate on cart data, but mix display and validation concerns. Contains product availability checking	No - Handles mixed concerns: 1) Cart retrieval/display 2) Product availability checks	Separate concerns: 1) Use shared CartService for cart retrieval, 2) Move product availability checking to InventoryService 3) Create dedicated ViewService if needed for

			mixed with cart retrieval and display logic	3) Price calculation.	display formatting 4) Focus controller solely on coordinating display operations.
4	ViewProductDetailsController	Vu Hai Dang-20225962	Functional - All methods work together with a clear, singular purpose: to retrieve and display product details. Methods are cohesive and focused on one task without mixing responsibilities.	Yes - Focused solely on product detail retrieval and display. The class has a single reason to change: if the way products are retrieved or displayed changes. Uses a clean pattern with requestToViewProductDetails and renderProductDetails.	Maintain design: Already well-designed with good cohesion and SRP compliance.
5	CartItem	Vu Hai Dang-20225962	Functional - All properties and methods focus on representing and managing a single cart item. Methods like changeQuantityOfProduct directly serve the cart item management purpose. Contains appropriate data and behavior for a cart item.	Yes - Represents and manages a single cart item entity. Contains only data and behaviors related to cart items (ID, quantity, price, product reference). Class would only change if cart item representation needs to change.	No changes needed: Class already demonstrates high cohesion and SRP compliance. Properly encapsulates cart item data and behavior. Could consider making immutable for additional safety if cart item properties shouldn't change after creation.
6	Product (and subclasses like Book, CD, DVD, LP Record)	Vu Hai Dang-20225962	Functional - Classes properly encapsulate product-specific behavior with appropriate inheritance hierarchy. Each subclass (Book, CD, DVD, LPRecord)	Yes - Each class in the hierarchy has clear responsibilities: base Product handles common attributes/behaviors	No changes needed: Class hierarchy follows good OO design with proper inheritance and encapsulation. Consider using factory pattern if product creation becomes more complex. Ensure serialization/deserialization is

			focuses on its specific product type while sharing common behavior through the base Product class.	ors, while subclasses handle specific product type concerns. Clear separation of concerns through inheritance.	handled appropriately if objects are persisted.
7	Cart	Vu Hai Dang-20225962	Functional - All methods focus on managing cart contents and operations (add, update, remove, calculate). Internal implementation details are encapsulated, and public methods all serve the central purpose of cart management.	Yes - Focused on cart operations with a cohesive set of methods for managing the shopping cart. Contains appropriate validation and operations for a shopping cart entity. Responsibilities are clearly defined around cart management.	No significant changes needed: Well-designed with good cohesion and SRP compliance. Consider adding events/observers if other components need to react to cart changes. Could extract a CartRepository interface if persistence becomes more complex.
8	HistoryUpdate	Le Dai Lam - 20225982	Functional – All methods (recordUpdate(), getUpdateHistory(), getLastUpdate(), deleteHistory()) operate on the same concept of update history.	Yes – Focused solely on update history management. Handles a single concern: tracking product update activities. The class would only change if the way update history is stored or retrieved changes.	Solution: No changes needed.
9	ProductController	Le Dai Lam - 20225982	Communicational – Methods (CreateProduct(), UpdateProduct(), ValidateProduct()) operate on product	No – Handles multiple concerns: 1) Product data validation 2) Business logic	Solution: 1) Extract a ProductValidationService. 2) Keep the controller focused only on calling services like

			data but involve two concerns: 1) Coordination 2) Validation logic	coordination (create/update) 3) Error handling and routing. These concerns should be separated into different classes/services.	ProductService.create() and update() after validation.
10	OrderController	Le Dai Lam - 20225982	Procedural – Contains too many responsibilities: approval, cancellation, refund, product checking, and data retrieval.	No – Handles multiple concerns: 1) Order approval and rejection 2) Order cancellation and refund 3) Order display and status rendering 4) Order-product checking logic. The class should be split into focused controllers for each use case.	Solution: 1) Split into OrderApprovalController, OrderQueryController, OrderCancellationController, etc. 2) Each controller should handle only one use case.
11	Order	Le Dai Lam - 20225982	Temporal – Combines UI logic (display methods), business logic (calculations), persistence (save()), and data (fields).	No – Handles mixed concerns: 1) Order data representation (ID, list of items, status) 2) UI logic (display approval status, render order screen) 3) Business logic (subtotal, VAT, shipping calculation) 4) Persistence (save/getById methods). Each of these concerns	Solution: 1) Retain only data fields in Order. 2) Move calculations to OrderService. 3) Move save/get to OrderRepository. 4) Move UI logic (e.g. DisplayApproveStatus) to a separate class or view/controller.

				should be placed in separate components.	
12	PaymentTransaction	Nguyen Minh Khoi-20226050	Logical – Class handles data (ID, amount, status), logic (evaluatePaymentResult), and persistence (save), making it a mix of responsibilities.	No – Combines: 1) Transaction data 2) Business logic (VNPay result simulation) 3) Persistence logic (save, saveRefundTransaction)	Split into: 1) PaymentTransaction (pure data) 2) PaymentService (evaluate and manage result) 3) TransactionRepository (save methods)
13	Invoice	Nguyen Minh Khoi-20226050	Logical – Contains pricing data, price/VAT/delivery calculations, persistence, and formatting logic, leading to multiple purposes.	No – Handles: 1) Business data 2) Price calculation logic 3) Persistence (save) 4) Invoice rendering	Split into: 1) Invoice as pure model 2) InvoiceCalculator 3) InvoiceRepository 4) Optional: InvoicePresenter
14	VNPay	Nguyen Minh Khoi-20226050	Functional – All methods and fields support a single responsibility: interacting with the VNPay payment gateway.	Yes – Clearly focused on external payment communication. Has a single reason to change: if VNPay interaction changes.	No changes needed.
15	PayOrderController	Nguyen Minh Khoi-20226050	Logical – Orchestrates multiple responsibilities like payment flow, exception handling, persistence, and diagnostic printing.	No – Handles: 1) Payment coordination 2) Persistence (tx/invoice save) 3) Logging & diagnostics	Refactor into: 1) PayOrderController (orchestration only) 2) PaymentService (handles VNPay + transaction result) 3) InvoiceService 4) Move logging to separate logger if needed.
16	PlaceRushOrderController	Bui Xuan Son - 20226065	Logical - The class handles multiple actions that could be grouped by a control structure or logic (e.g., "if X, validate; if Y, notify"). However, it's	Yes, handle UI event and using routines from 'service' layer to process the needed data	No changes needed.

			an anti-pattern when one class chooses between unrelated operations.		
17	UserController	Bui Xuan Son - 20226065	Logical - The class contains logic for different but related operations grouped in the same controller (create vs update logic vs policy checks), decided by code branches or method calls.	Yes, handle UI event and using routines from 'service' layer to process the needed data	No changes needed.
18	DeliveryInfo	Bui Xuan Son - 20226065	Functional - All fields and methods work together to represent a single concept: delivery information.	Yes, wrap all attributes of a delivery information and provide methods to initialize, retrieve and update	No changes needed.
19	CartRepository	Bui Xuan Son - 20226065	Communicational - The method uses shared data (cartItems, products) to produce output — they are processed together.	No – Handles: 1) Database connection 2) Execute cart-related queries	Build 'utility' class which work on database connection
20	UserRepository	Bui Xuan Son - 20226065	Communicational - The methods operate on shared data (users list).	No – Handles: 1) Database connection 2) Execute user-related queries	Build 'utility' class which work on database connection
21	CartService	Bui Xuan Son - 20226065	Functional - The method performs one specific function: mapping cart items to their delivery eligibility.	Yes, using services from 'repository' class to retrieve the needed data, processing them, making them convenient for related 'controller' classes	No changes needed.

22	UserService	Bui Xuan Son - 20226065	Functional - All methods serve the purpose of handling user-related operations.	Yes, using services from 'repository' class to retrieve the needed data, processing them, making them convenient for related 'controller' classes	No changes needed.
23	User	Bui Xuan Son - 20226065	Functional - All fields and methods are directly related to representing a user object.	Yes, wrap all attributes of an user information and provide methods to initialize, retrieve and update	No changes needed.
24	PlaceOrderController	Pham Thanh Nam - 20225989	Functional - All methods focus specifically on place order workflow with clear delegation pattern	No - Handles multiple concerns: 1) Cart validation 2) Delivery info validation 3) Inventory checking 4) Order creation 5) Invoice generation	Solution: 1) Keep the controller focused on orchestration only 2) Extract validation logic to dedicated validators 3) Move invoice generation to a dedicated service 4) Use dependency injection for all services

5.3.2 Coupling & Other SOLID

Explain in detail for each problem in the table. Each problem, you may provide a class diagram before and after the improvement.

Table 5. Coupling & other SOLID of AIMS

#	Problem	PIC	Location	Solution
1	Tight coupling between controllers and multiple services/repositories. Violates SRP and DIP.	Vũ Hải Đăng-20225962	AddProductToCartController	Extract services like CartService. Use dependency injection. Keep controllers thin and focused on coordination.

2	Tight coupling between controllers and multiple services/repositories. Violates SRP and DIP.	Vũ Hải Đăng-20225962	UpdateCartController	Extract services like CartService and ProductService. Use dependency injection. Keep controllers thin and focused on coordination.
3	Tight coupling between controllers and multiple services/repositories. Violates SRP and DIP.	Nguyễn Minh Khôi-20226050	OrderController	Extract services like CartService, ProductService, and OrderService. Use dependency injection. Keep controllers thin and focused on coordination.
4	Tight coupling between controllers and multiple services/repositories. Violates SRP and DIP.	Phạm Thành Nam-20225989	PlaceOrderController	Extract services like CartService, ProductService, and OrderService. Use dependency injection. Keep controllers thin and focused on coordination.
5	Entity classes (e.g., Product, AuditLog, Order) contain mixed concerns: logic, data, and UI.	Lê Đại Lâm-20225982	Product, AuditLog, Order, Invoice, PaymentTransaction	Split responsibilities: Keep entities pure, move logic to services (e.g., AuditLogService), and move UI or formatting logic to separate classes.
6	Tight coupling between controllers and internal entity models. Exposing raw entities via APIs risks leaking sensitive data and creates strong dependency.	Bùi Xuân Son-20226065	AddProductToCartController, UpdateCartController, ProductController, OrderController, PlaceOrderController	Introduce DTOs (e.g., ProductDTO, CartItemDTO, OrderDTO) to decouple API responses from entity classes. Use mappers to convert between entities and DTOs. This improves security, maintainability, and aligns with separation of concerns.

5.4 Design Patterns

After the refactoring process, the system has been significantly improved by decomposing business logic into dedicated services such as CartService, ProductService, OrderService, and AuditService. This restructuring aligns with the Single Responsibility Principle (SRP) and promotes high cohesion and low coupling across modules. In addition, the use of Data Transfer Object (DTO) patterns (e.g., ProductDTO,

OrderItemListDTO) enhances data encapsulation and ensures a clear separation between internal domain models and external API responses.

Controllers are now thin and act only as coordinators, delegating responsibilities to services—this reflects the application of the Facade Pattern, where complex subsystems are abstracted behind simple interfaces. The overall architecture also moves toward the principles of Clean Architecture, making the system easier to maintain, test, and extend. These improvements contribute to a modular, scalable, and robust design, well-suited for enterprise-level applications.